Static Analysis of Serverless Applications for Security

Giuseppe Raffa

Information Security Group
Royal Holloway, University of London

Thesis submitted for the degree of
Doctor of Philosophy

June 2025

Declaration

These doctoral studies were conducted under the supervision of Dr. Jorge Blasco Alís, Dr. Daniel O'Keeffe, and Dr. Santanu Kumar Dash.

I, Giuseppe Raffa, hereby declare that this thesis and the work presented in it are entirely my own. Where I have consulted the work of others, this is always clearly stated. This work has not been submitted for any other degree or award in any other university or educational establishment.

Giuseppe Raffa
June 2025

Acknowledgements

I would like to start by wholeheartedly thanking my supervisors, Dr. Jorge Blasco Alís, Dr. Daniel O'Keeffe, and Dr. Santanu Kumar Dash. At the beginning of my studies, I was told that working with a supervisory team can be a challenging task. In my case, it has simply been a privilege. You have shared with me your experience, knowledge, and different perspectives. I have learned a lot along the way, and I cannot thank you enough for your continuous support and encouragement.

I must also express my gratitude to the Centre for Doctoral Training (CDT) in Cyber Security at Royal Holloway for this wonderful opportunity. Thanks to their support, I have been able to present my research in four different countries, a truly career-changing experience. Being part of the CDT means also something else though. That there is always someone you can talk to. I really appreciate it, and I would like to thank the management committee, Claire, and Harpreet for their efforts.

These acknowledgements would not be complete if I did not mention my CDT cohort. We began our PhD journeys in 2020, the first year of the COVID pandemic. No, it was not easy, so a big thank you to Jamie, Dan, Steph, Sofia, Emma, and Elle for their commitment and dedication. You showed me that working together and having fun was still possible, despite the circumstances.

Finally, my family. My wife Anna has been very supportive and undoubtedly very patient over the last four years. I greatly appreciate this, and hope that the end of my PhD studies will also be the beginning of a new chapter in our lives. There is another important family member I have to mention though, and that is Sally, our cat. It is impossible to say how many times you have interrupted my work, nor will I ever be able to count the lines of code I have written with you sleeping on my lap. Your love has probably prolonged my PhD, but it was, and still is, love nevertheless. Thank you.

Abstract

The serverless computing paradigm has significantly changed how modern cloud applications are developed. This model allows developers to focus on the business logic of their applications while outsourcing to the cloud provider infrastructure details, such as machine provisioning. However, the serverless model also presents new security challenges. Among these, static analysis of application security, a fundamental part of the secure software development lifecycle, becomes more complex due to the presence of event-triggered code and the black-box nature of cloud services.

This thesis begins with the study of `AWSomePy`, a new dataset of 145 serverless applications. The objective of this part of the presented work is to identify real-world usage trends, as these can guide the development of novel static analysis tools. The application code-level analysis of `AWSomePy`, in particular, establishes which cloud services and APIs developers use most frequently. The dataset characterization, along with documented examples of serverless-specific vulnerabilities, inform the design of `CloudBench`, a suite of 40 security-oriented microbenchmarks for serverless computing. This is an important contribution of this research, as the existing serverless benchmarks are focused on performance evaluation and cost effectiveness.

This work ends by presenting `CloudFlow`, a novel framework to statically detect security-sensitive data flows in serverless applications. To achieve this, `CloudFlow` leverages the infrastructure definition provided by the developer to identify the events, permissions and entry points of an application. Using this information and custom models for events and cloud API calls, it instruments the application code, which can then be analysed with general-purpose methods for static analysis. The framework is evaluated against the `CloudBench` suite. Furthermore, 104 real-world applications selected from `AWSomePy` are analysed. The results show that `CloudFlow` passes all microbenchmarks, apart from three, and detects 11 code injection and information leakage vulnerabilities in the analysed applications.

Overall, this thesis shows that vulnerabilities exist in real-world serverless applications, and that it is possible to discover them using static analysis. While further research into serverless security frameworks is necessary, `CloudFlow` and `CloudBench` will hopefully constitute an important starting point.

Table of Contents

List of Figures xiii

List of Tables xv

Nomenclature xix

1 Introduction 1

1.1 Motivation 1
1.2 Research Objective and Questions 3
1.3 Thesis Structure 4
1.4 Answers to Research Questions 6
1.5 Contributions 9
1.6 Publications 10

2 Background 11

2.1 Serverless Computing Paradigm 11
2.1.1 Overview 12
2.1.2 Advantages and Disadvantages 13
2.2 Securing Serverless Applications 15
2.2.1 Critical Risks for Serverless Applications 16
2.2.2 Example of Vulnerable Serverless Application 18
2.2.3 Identity and Access Management 20
2.3 Infrastructure as Code 22
2.4 Overview of Assessed Code Analysis Frameworks 23
2.4.1 Frameworks for Event-driven Applications 24
2.4.2 Frameworks for Data Flow Analysis 26
2.5 Static Analysis of Python Code 28
2.5.1 Python Type System 29
2.5.2 Pysa Static Analyser 30

2.6 Chapter Summary 32

3 Characterization of the AWSomePy Dataset 35

3.1 Introduction 35
3.2 Dataset Generation 37
3.3 Dataset Analysis 40
3.3.1 Configuration and Architectural Analysis 40
3.3.2 Cloud Service and API Usage 42
3.4 Discussion 46
3.5 Related Work 48
3.6 Chapter Summary 50

4 CloudBench: A Novel Suite of Security-oriented Microbenchmarks 51

4.1 Introduction 51
4.2 Motivating Examples 52
4.2.1 Code Injection Example 53
4.2.2 Information Leakage Example 56
4.3 CloudBench 57
4.4 Related Work 61
4.5 Chapter Summary 62

5 CloudFlow: A Framework for the Detection of Security-sensitive Data Flows 63

5.1 Introduction 63
5.2 CloudFlow 64
5.2.1 Overview 64
5.2.2 Infrastructure Code Processing 66
5.2.3 Application Code Processing 67
5.2.4 CloudFlow Representation 68
5.2.5 Static Analysis Models 69
5.3 Evaluation 69
5.3.1 Results 69
5.4 Discussion 72
5.5 Related Work 74
5.6 Chapter Summary 77

6 Analysis of the AWSomePy Dataset with CloudFlow 79

6.1 Introduction 79

6.2 Dataset Triage 80
6.3 Security-sensitive Data Flows 81
6.4 Vulnerability Case Studies 84
6.5 Performance Evaluation 88
6.6 Discussion 89
6.7 Related Work 93
6.8 Chapter Summary 94

7 Conclusion 97
7.1 Future Work 97
7.2 Final Remarks 99
7.2.1 Evolution of the Serverless Paradigm 99
7.2.2 Recommendations 100

References 103

Appendix A CloudBench Characterization 117
A.1 Preliminary Microbenchmark Collection 117
A.2 CloudBench Inter-procedural Microbenchmarks 118

Appendix B CloudFlow Architecture and Evaluation 121
B.1 CloudFlow Architecture 121
B.2 Ablation Study Results 123

Appendix C AWSomePy Dataset Analysis 125
C.1 AWSomePy Dataset Triage 125
C.2 Identified Security-sensitive Data Flows 127

List of Figures

2.1 Key elements of a FaaS solution [33]. 12
2.2 Shared responsibility model [108]. 14
2.3 Example application with an event-driven data flow and a code injection vulnerability. The blocks AWS Lambda identify the three event-triggered handlers included in the application. 19
2.4 AWS-specific example of IAM policy [12]. 21
2.5 Example of declarative code for IaC [91, Ch. 4]. 23
2.6 Example of imperative code for IaC [91, Ch. 4]. 24
2.7 Pysa workflow. 31

3.1 Dataset generation process. The dashed lines indicate the customized steps compared to the Wonderless dataset. 37
3.2 Cumulative distribution of the lines of code in `AWSomePy` (cumulative fraction of repositories on the y axis). 43
3.3 Number of handlers vs lines of code in `AWSomePy`. 43
3.4 Number of events vs number of handlers in `AWSomePy`. 44

4.1 Example application with an event-driven data flow. The four event-triggered handlers are identified by the blocks AWS Lambda. 53
4.2 Microbenchmark `api-publish-wrong-bucket-key`. 56

5.1 Proposed static analysis pipeline for serverless applications (PEH=Permissions, Events and Handlers). 64

6.1 Analysed `AWSomePy` applications vs number of security-sensitive data flows. 82
6.2 Cumulative distribution of the detected security-sensitive data flows. 82
6.3 Number of security-sensitive data flows (vertical axis on the left) and LOC (vertical axis on the right) vs analysed `AWSomePy` applications sorted by their LOC. Note the logarithmic scale on the vertical axis on the right. 83

6.4 Total and CloudFlow-specific execution times vs LOC of the analysed AWSomePy repositories. The displayed values are the average of five CloudFlow executions. Note the logarithmic scale on the horizontal axis. 88
6.5 Cumulative distribution of the CloudFlow overhead. 89
6.6 Interquartile range, minimum, maximum, and median of the recorded CloudFlow overhead percentages. 90

B.1 CloudFlow modular architecture. 122
B.2 CloudFlow detailed workflow. 122

List of Tables

1.1 Mapping between research questions and core thesis chapters. 4

2.1 Critical risks for serverless applications according to the CSA [34]. 17
2.2 Summary of assessed frameworks for event-driven applications. 25
2.3 Summary of assessed frameworks for data flow analysis. 26

3.1 Summary of the dataset generation process. 39
3.2 Top eight plugins in `AWSomePy`. 41
3.3 Top eleven AWS services in `AWSomePy`. The column Occurrences reports the number of `boto3` client and resource objects instantiations within the relevant repositories. 45
3.4 Occurrences of the six most widely used APIs for the top three AWS services in `AWSomePy`. The occurrences of all the other detected APIs are aggregated in the entry *other*. 47
3.5 Occurrences of the six most widely used APIs for fourth (`ssm`) and the fifth (`sqs`) most frequently used AWS services in `AWSomePy`. The occurrences of all the other detected APIs are aggregated in the entry *other*. The `ssm` APIs `get_parameters_by_path` and `describe_instance_information` are abbreviated as `get_parameters_by_p` and `describe_instance_i`, respectively. 47

4.1 Summary of the `CloudBench` microbenchmark suite. This table specifies the categorization of the microbenchmarks, the included vulnerabilities, and the LOC range (CI=Code Injection, II=Information Integrity, IL=Information Leakage, LOC=Lines Of Code, NN=None). 57
4.2 Summary of the `CloudBench` microbenchmark suite. This table specifies the categorization of the microbenchmarks and the supported cloud services. . . 57

4.3 Data flow characterization. CloudBench inter-procedural microbenchmarks with S3 as target service (CI=Code Injection, II=Information Integrity, IL=Information Leakage, NN=None, VUL=Vulnerability). 59
4.4 Data flow characterization. CloudBench inter-procedural microbenchmarks with DynamoDB as target service (CI=Code Injection, II=Information Integrity, IL=Information Leakage, NN=None, VUL=Vulnerability). 59
4.5 Data flow characterization. CloudBench inter-procedural microbenchmarks with SQS as target service (CI=Code Injection, II=Information Integrity, IL=Information Leakage, NN=None, VUL=Vulnerability). 60
4.6 Data flow characterization. CloudBench inter-procedural microbenchmarks with SNS as target service (CI=Code Injection, II=Information Integrity, IL=Information Leakage, NN=None, VUL=Vulnerability). 60
4.7 Characterization of intra-procedural microbenchmarks (CI=Code Injection, II=Information Integrity, IL=Information Leakage, NN=None, TS=Target Service, VUL=Vulnerability). 60

5.1 Summary of CloudFlow evaluation (EVI=Environment Variable Inspection, INTER=Inter-procedural, INTRA=Intra-procedural, PLA=Plugin Analysis, REA=Resource Analysis, SA=Simple Applications). 70

6.1 Summary of the AWSomePy dataset triage. The categorized repositories are listed in the specified tables. 80
6.2 Summary of the security-sensitive data flows identified in the AWSomePy dataset. The table specifies the total number of data flows for each category along with the number of affected applications (App=Application, INTER=Inter-procedural, INTRA=Intra-procedural, IC=Injected Code). . . 84
6.3 Summary of the vulnerabilities detected in the AWSomePy dataset. The table specifies the total number of vulnerabilities for each category along with the number of affected applications. The column Models reports the number of CloudFlow-generated custom Pysa models. (App=Application, INTER=Inter-procedural, INTRA=Intra-procedural, IC=Injected Code). . . 84
6.4 Alternative mechanisms to define permissions identified in our dataset (arranged alphabetically). 91
6.5 Cloud APIs supported by CloudFlow and corresponding AWS services. . . 91
6.6 AWSomePy files with asynchronous constructs (ACO=Asynchronous Constructs Occurrences). 92

A.1 Preliminary collection of microbenchmarks used to develop the `CloudBench` suite (INTER=Inter-procedural, INTRA=Intra-procedural). 117
A.2 Cloud service characterization. `CloudBench` inter-procedural microbenchmarks with S3 as target service (DDB=DynamoDB, TS=Target Service, ✔=Yes, ✗=No). 118
A.3 Cloud service characterization. `CloudBench` inter-procedural microbenchmarks with DynamoDB as target service (DDB=DynamoDB, TS=Target Service, ✔=Yes, ✗=No). 119
A.4 Cloud service characterization. `CloudBench` inter-procedural microbenchmarks with SQS as target service (DDB=DynamoDB, TS=Target Service, ✔=Yes, ✗=No). 119
A.5 Cloud service characterization. `CloudBench` inter-procedural microbenchmarks with SNS as target service (DDB=DynamoDB, TS=Target Service, ✔=Yes, ✗=No). 119

B.1 Microbenchmark-level results of `CloudFlow` ablation study. The column CloudFlow specifies the CloudFlow evaluation results (EVI=Environment Variable Inspection, PLA=Plugin Analysis, REA=Resource Analysis, ✔=Decisive feature, ✗=Non decisive feature). 123

C.1 `AWSomePy` repositories categorized as Template (TEM). 126
C.2 `AWSomePy` repositories categorized as `CloudFlow`-incompatible (CFI). . . . 126
C.3 `AWSomePy` repositories categorized as Pysa-incompatible (PSI). 127
C.4 `AWSomePy` repositories categorized as infrastructure code inconsistent (ICI). 127
C.5 `AWSomePy` repositories categorized as application code inconsistent (ACI). . 127
C.6 `AWSomePy` repositories with security-sensitive data flows. 128

Nomenclature

Acronyms / Abbreviations

ABAC Attribute-based Access Control

API Application Programming Interface

ARN Amazon Resource Name

AST Abstract Syntax Tree

AWS Amazon Web Services

CD Continuous Delivery

CDK Cloud Development Kit

CDN Content Delivery Network

CG Callgraph

CI Continuous Integration

CLI Command Line Interface

CSA Cloud Security Alliance

DAST Dynamic Application Security Testing

DB Database

DNS Domain Name System

DoW Denial of Wallet

DSL Domain-specific Language

FaaS Function as a Service

GUI Graphical User Interface

HTTP Hypertext Transfer Protocol

IaaS Infrastructure as a Service

IaC Infrastructure as Code

IAM Identity and Access Management

IT Information Technology

JSON JavaScript Object Notation

LOC Lines of Code

MFA Multi-factor Authentication

NLP Natural Language Processing

OS Operating System

PaaS Platform as a Service

RBAC Role-based Access Control

S3 Simple Storage Service

SAST Static Application Security Testing

SDK Software Development Kit

SQL Structured Query Language

URL Uniform Resource Locator

VCS Version Control System

VM Virtual Machine

WAF Web Application Firewall

XML Extensible Markup Language

YAML YAML Ain't Markup Language

Chapter 1

Introduction

In this chapter, we illustrate the motivation, the research questions and the structure of this thesis. Taking into account the key findings, the answers to the research questions are also provided. The chapter ends by presenting the contributions of this work and the published papers that constitute its basis.

1.1 Motivation

In recent years, a growing number of enterprises have adopted the serverless computing paradigm to develop a variety of cloud-based applications. Serverless platforms are an attractive option because they simplify the development process by reducing operational overheads. In the serverless model, cloud providers, e.g., Amazon [6], Microsoft [89], and Google [61], are responsible for provisioning, scaling, and maintaining the underlying infrastructure. Furthermore, numerous backend services [94, 84], such as database management systems and data storage solutions, facilitate the implementation of a diverse range of applications, including image processors, web and chatbot services. Serverless computing is also a viable option for the execution of complex scientific workflows [42].

However, recent data breaches [104–106], academic research [35, 31] and reports by industry experts [34, 100] show that securing serverless applications is challenging. A recent report by the Cloud Security Alliance (CSA) [34], in particular, highlights that such applications are affected by a combination of well-studied and serverless-specific risks (§2.2.1). Code injection and broken authentication are included in the first category, whereas financial resource exhaustion and cross-execution data persistency are prime examples of the second. Importantly, even when the concepts underlying a given risk are consolidated, the characteristics of the serverless paradigm, described in §2.1.1, provide a different context. Differently from applications that rely on cloud infrastructure solely for the implementation of their

backend functionality, in fact, serverless applications usually leverage several cloud-based services. This implies that they receive inputs from a variety of sources and interact with the required services through a wide range of internal and external endpoints. Such applications, therefore, typically feature a large attack surface. As a result, developers, who are tasked with securing their applications to comply with the well-known shared responsibility model [75, Ch. 9], face considerable challenges.

To improve the cloud provider-developed tool ecosystems [144, 86], researchers have proposed frameworks focused on dynamic information flow [4, 38, 69], customized access control models [131] and provenance [39]. Moreover, model-based techniques for integration-level testing of serverless applications have also been explored [168–171]. Notably though, these frameworks and testing techniques require the deployment of additional code and runtime information, respectively. All such research contributions, therefore, are focused on dynamic analysis techniques that rely on the execution of the inspected application in the cloud environment.

Dynamic analysis is widely adopted in software engineering, as it allows observing the behaviour of running code. Although this, in general, facilitates the development process, the effectiveness of such techniques critically depends on how the application under test is stimulated. In the specific context of serverless computing, this means that if the chosen inputs do not trigger certain data flows, then detecting security vulnerabilities associated with them will not be possible. Thus, dynamic analysis approaches can lead to the deployment of applications with security flaws. Furthermore, given that all serverless providers adopt a pay-per-usage billing model (§2.1.2), executing code for testing purposes implies incurring additional costs.

Static analysis, by contrast, enables developers to inspect an application *prior* to its deployment. While undoubtedly advantageous, analysing serverless applications statically is challenging for three reasons, which are an important starting point for this thesis:

- **C1:** Serverless applications are *event-driven*, i.e., they consist of a set of short-running stateless functions triggered, for example, by HTTP requests and database updates. This makes static analysis harder because functions are also connected via cloud interactions that are not part of the synchronous execution of the application.

- **C2:** Application permissions can be configured using a wide range of mechanisms, which, as illustrated in §2.2.3, are generally rather sophisticated.

- **C3:** The code that implements the platform services is proprietary and cannot be analysed by the developer. This implies that models and approximations are needed.

While taking important first steps, previous works on call graphs [96, 95] and performance modelling [165], further discussed in §5.5, only partially address these challenges, as they require developers to manually model the complex interactions with backend services, and do not fully take into account how the cloud infrastructure is defined.

To address the shortcomings of the available studies and the above challenges, further explained in §2.2.2 and §4.2.1 with the support of practical examples, this work seeks to contribute to the body of knowledge on software static analysis by proposing a serverless-specific approach to detecting security-sensitive data flows. Their identification, in fact, helps to address some of the most critical risks for serverless applications, such as code injection and information leakage [34, 100].

As illustrated in the remainder of this chapter, this thesis starts with the characterization of real-world applications, identifies new reference microbenchmarks, and ends by presenting a novel, comprehensively evaluated static analysis framework.

1.2 Research Objective and Questions

The overall objective of this thesis is to advance the current static analysis techniques in order to identify security vulnerabilities in serverless applications. We break down this objective into four research questions, which are introduced in this section.

Since serverless applications rely on APIs that access cloud-based services, modelling the functionality offered by these is undoubtedly important. Ignoring the effect of such APIs, in fact, would result in an incomplete analysis. The number of APIs and services available on modern serverless platforms is very high though. Consequently, modelling all of them, especially as part of a research study, is unattainable. The following question is therefore legitimate:

- **RQ1:** What are the cloud-based services and the APIs most frequently used in real-world serverless applications?

In addition, comprehensive evaluations of security tools typically include microbenchmarks [7, 146]. The key advantage of this approach is that microbenchmarks, such as those included in the Android-focused DroidBench suite [150], provide a standardized set of tests. These can be used to assess the effectiveness of new frameworks and to compare them with the existing ones. Since achieving our overall research objective implies evaluating a novel analysis approach, this question needs to be considered[1]:

[1]The difference between benchmarks and microbenchmarks is clarified in §4.1. This does not affect the rationale behind RQ2, since both benchmarks and microbenchmarks provide standardized references to evaluate novel frameworks.

- **RQ2:** How can we design security-oriented serverless microbenchmarks?

The previously introduced challenges and the limitations of the available studies (§1.1) motivate the need for serverless-specific static analysis approaches. These, given their complexity, will consist of multiple, task-focused steps, which need to be suitably chained together into an effective pipeline. The latter is instrumental in achieving our objective, thus prompting this research question:

- **RQ3:** How can a serverless-specific static analysis pipeline be structured?

Finally, considering the absence of published papers including large security-focused analyses of real-world serverless applications, we reckon that bridging this gap in the literature is crucial. Gaining insight into the types of security vulnerability that can affect such applications, in fact, can improve the current implementation guidelines and reshape software development workflows. Moreover, inspecting a large dataset with a new analysis framework allows evaluating the latter more comprehensively, as the coverage and scope of a microbenchmark suite are inevitably limited. All this gives rise to the following question:

- **RQ4:** Are real-world serverless applications affected by vulnerabilities that can be identified statically?

Note that the above research questions are mapped to the thesis chapters in §1.3.

1.3 Thesis Structure

This thesis consists of three constituent parts, which are described below. Furthermore, to help the reader understand the structure of this work, the research questions introduced in §1.2 are mapped to their respective chapters in Table 1.1.

Table 1.1 Mapping between research questions and core thesis chapters.

Research Question	Ch. 3	Ch. 4	Ch. 5	Ch. 6
RQ1	✔			
RQ2		✔		
RQ3			✔	
RQ4				✔

Preliminaries. This part of the thesis includes Chapter 1 and 2. The latter introduces the serverless computing model along with the serverless security concepts that underpin this

research. The chapter also provides an overview of existing code analysis frameworks, and ends with a description of the language-specific tools used to implement `CloudFlow`.

Static Analysis Framework Development. All the core chapters of the thesis, i.e., Chapters 3, 4, 5, and 6, are included in this part. As further elucidated below, their chief objective is to illustrate how we informed the design and the evaluation of `CloudFlow`, the proposed static analysis framework. We also note that all such chapters include a related work section (i.e., §3.5, §4.4, §5.5, and §6.7), where we review previous research contributions.

- Chapter 3 presents the study of `AWSomePy`, a collection of applications developed in Python for the Amazon Web Services (AWS) platform. The main motivation of this chapter is that some of the security-related challenges posed by the serverless computing paradigm can be addressed by analysing real-world applications to identify key trends. These, in fact, can guide the development of novel models and tools. The results of the dataset characterization were used to design our microbenchmarks. `AWSomePy` is publicly available to support future research work.

- Chapter 4 introduces `CloudBench`, a novel suite of 40 security-oriented serverless microbenchmarks. Previous research shows that security tools can be tested and compared effectively using microbenchmarks [7]. The existing serverless benchmarks, however, are designed to assess platform performance [164, 87] and cost effectiveness [16], and cannot, therefore, be used to evaluate security tools. `CloudBench`, which includes intra-procedural and inter-procedural microbenchmarks, is used to evaluate our static analysis framework.

- Chapter 5 is focused on `CloudFlow`, a novel framework to statically detect security-sensitive data flows in serverless applications. To address the complexity caused by event-triggered code, multiple application permission mechanisms, and the black-box nature of cloud services, `CloudFlow` is based on a serverless-specific analysis pipeline. The chapter describes the latter, provides details on the framework implementation, and presents the results of the microbenchmark-based evaluation.

- Chapter 6 illustrates the analysis of `AWSomePy` conducted with the proposed static analysis framework. The principal aim of this analysis is to identify security vulnerabilities that affect real-world serverless applications. Moreover, it allows us to assess the quality and the robustness of the `CloudFlow` design, thus completing its evaluation. The chapter begins by describing the initial dataset triage, and it then illustrates the detected code injection and information leakage vulnerabilities.

Conclusion. Chapter 7 constitutes the last part of this thesis. Building on the answers to the research questions, provided in §1.4, and the limitations of the proposed framework, the chapter identifies areas for future work. It also includes our final remarks on the expected evolution of the serverless model, and the security and static analysis-related implications that this might have. Finally, we share the experience gained during this project by providing our recommendations to researchers and industry practitioners.

Appendices. This thesis also includes a set of appendices with supplementary information. Specifically, Appendix A provides additional details on `CloudBench`. In contrast, Appendix B is focused on the architecture and the microbenchmark-based evaluation of the proposed framework. Finally, Appendix C is dedicated to the analysis of the `AWSomePy` dataset performed with `CloudFlow`.

1.4 Answers to Research Questions

In this section, starting from the key findings of this work, we provide the answers to the research questions introduced in §1.2.

RQ1. The first core chapter of this thesis (Chapter 3) is dedicated to `AWSomePy`, a new dataset of 145 serverless applications compatible with the AWS platform. As summarized in Table 3.3, the application code-level analysis of `AWSomePy` reveals that the two most frequently used cloud-based services are `s3`, a data storage solution, and the NoSQL database `dynamodb`. This result confirms that serverless functions, due to their stateless and ephemeral nature, critically rely on these types of services to store reusable information. Notably though, the third and the fourth most frequently occurring services, i.e., `lambda` and `ssm`, belong to a different category, as they are configuration and management-oriented. From a developer perspective, using these services is advantageous, as they provide a high degree of flexibility. However, we underscore that they also add workflows, e.g., aiming at the dynamic configuration of cloud resources, that are difficult to inspect statically.

Moreover, we observe that `sqs` and `sns`, a message broker and a notification service with a key role in event-driven architectures, are only the fifth and the sixth in Table 3.3. This shows that the `AWSomePy` applications are less architecturally complex than expected, a trend also highlighted by Eskandani *et al.* in their analysis of the Wonderless dataset [46].

As regards the identified cloud APIs, `put_object` and `put_item` are the most frequent for the `s3` and the `dynamodb` services, respectively (Table 3.4). Our analysis also illustrates that developers often rely on APIs, i.e., `create_bucket` (`s3` service) and `create_table`

(`dynamodb` service), that enable the programmatic creation of infrastructure resources. From a static analysis point of view, the usage of such APIs presents a challenge, as the created resources cannot be inspected at coding time by analysing the infrastructure code.

In addition, we note that the `lambda` service API with the highest number of occurrences is `invoke` (Table 3.4). If its inputs are parametrized, as explained in §3.3.2, the code to be executed through this API can be hard to identify. Similarly to the aforementioned case of configuration and management-oriented services, therefore, its use potentially introduces code that cannot be analysed statically.

RQ2. Our analysis of the available studies on serverless benchmarks highlights that all the previously published works are focused on platform performance or cost-effectiveness (§4.4). Considering documented examples of risks and vulnerabilities, therefore, was essential to design security-oriented microbenchmarks. The previously mentioned CSA report [34] and the scenarios described in the OWASP Serverless Top 10 Project [100] were key references. The former, in fact, lists the 12 most critical risks for serverless applications, whereas the latter extends well-established web application security concepts to the FaaS model (§2.2.1). Other important examples were provided by Daly [71], who elucidates the concept of event injection, Bhargav [1], who shows an example of NoSQL injection, and Podjarny *et al.* [103], who describe several types of serverless vulnerabilities. We note that none of these are academic resources, which is important for two reasons. First, it illustrates the gap that this thesis tries to bridge. Second, it confirms that the FaaS model is enterprise-oriented, despite the availability of open-source platforms.

Relying only on the aforementioned vulnerabilities, however, is not sufficient to develop a high-quality microbenchmark suite. Modern serverless environments offer a wide range of cloud-based services, which can be accessed with numerous APIs. Both contribute to a complex ecosystem, and it is therefore crucial to consider key trends in real-world applications that show how such services and APIs are used in practice. Thus, we also base the design of `CloudBench`, the proposed microbenchmark suite (Chapter 4), on the characterization of `AWSomePy`. `CloudBench`, in fact, supports five services, chosen among the most frequently detected in the dataset (Table 4.2).

With the constant evolution of the serverless offerings though, `CloudBench` will have to be updated in the future to incorporate newly developed services and APIs. Moreover, some configuration and management-oriented services often used in `AWSomePy` are not currently supported. As explained in §4.3, these cause data flows hard to identify statically, and will consequently require improved detection strategies as well as additional microbenchmarks.

RQ3. The serverless-specific static analysis pipeline visible in Fig. 5.1 shows the three stages of the proposed approach as well as the steps that each of them requires. As detailed in §5.2, the key principle underlying our analysis pipeline is that both the infrastructure and the application code need to be processed in order to extract relevant information, e.g., permissions and environment variables. Such information is then used to automatically instrument the application under test, thus obtaining a program representation (§5.2.4) that allows identifying security-sensitive data flows with a general-purpose static analysis tool. It is also worth noting that the illustrated approach relies on documentation-based cloud API call models, along with application and cloud API-specific static analysis models (§5.2.5).

`CloudFlow`, the automated framework that instantiates our analysis pipeline, is evaluated against the `CloudBench` microbenchmarks. As shown in §5.3, `CloudFlow` passes 92.5% of such microbenchmarks, i.e., 37 out of 40, which proves its effectiveness, despite the unavoidable limitations (§5.4).

RQ4. As illustrated in Chapter 6, the analysis of the `AWSomePy` dataset conducted with our framework shows that 11 vulnerabilities are present in five applications (Table 6.2). The manual validation of these, summarized in Table 6.3, identifies seven code injection vulnerabilities, triggered by either CLI commands or HTTP requests, and four cases of exception traceback leakage. The latter is a specialized form of information leakage, as it discloses details about raised exceptions. Although not directly exploitable and useful for testing and debugging, such information can be leveraged by malicious users.

These findings confirm that real-world serverless applications are affected by vulnerabilities that can be detected via static analysis. It is also important to note that code injection vulnerabilities are the most frequent in our investigation. This result validates the assessments by PureSec [108] and the CSA [34] discussed in §2.2.1, as code injection is deemed to be the most critical risk in both cases. Importantly, such risk analyses were published in 2019, whereas the `AWSomePy` dataset was created in 2022. Albeit detected only in five out of 104 applications, code injection is therefore still an issue, and its prevention should be prioritized.

The second type of identified vulnerability, i.e., exception traceback leakage, is also present in the PureSec and the CSA report. The tenth risk in both of them, in fact, is *Improper Exception Handling and Verbose Error Messages*, which includes the case of exception traceback leakage. Given its ranking, the latter is considered significantly less critical than code injection in the reference assessments. The results of this thesis, however, illustrate that improper exception handling should not be underestimated.

To fully answer our vulnerability-related research question, we also observe that 64 security-sensitive data flows are classified as potential. This is due to the lack of certainty

caused by the high number of intermediate function calls or the presence of previously deployed cloud-based resources, which are both difficult to inspect manually. Notably though, it is possible that some of these data flows are actual vulnerabilities. To improve their detection rate, further research into static analysis of serverless applications is necessary, but we hope that this will be underpinned by the ideas and the results presented in this thesis.

1.5 Contributions

Taking into account the key objective of this work (§1.2) and the answers to the initial research questions (§1.4), our contributions can be summarized as follows:

- **Serverless Dataset**. We publicly release `AWSomePy`, a new collection of AWS serverless applications implemented in Python. Importantly, we provide a characterization of this dataset, which we obtain by conducting an architectural and an application code-level analysis. The extracted information, as shown by the `CloudBench` suite and the `CloudFlow` framework, can guide the development of novel models and tools.

- **Microbenchmark Suite**. We present `CloudBench`, a new suite of security-oriented serverless microbenchmarks. `CloudBench`, which we open-source[2], can be extended and used to test future static analysis tools focused on identifying code security properties in serverless environments.

- **Static Analysis Framework**. We propose `CloudFlow`, a novel automated framework for the static detection of security-sensitive data flows in serverless applications. This allows application developers to identify possible security vulnerabilities before their code is deployed to the cloud.

- **Real-world Application Analysis**. We analyse with our framework a total of 104 real-world applications, obtained from the `AWSomePy` dataset. This analysis identifies code injection and information leakage vulnerabilities, thus confirming the suitability of the `CloudFlow` design. To the best of our knowledge, this is the largest security-focused analysis of serverless applications to date.

[2] As further detailed in Chapter 4, only a preliminary version of `CloudBench`, which we published with our paper P2, is publicly available at the time of this writing. We plan to release all the developed microbenchmarks in the near future.

1.6 Publications

The published works P1, P2 and P3 constitute the basis of this thesis. They are the result of the collaboration with my supervisory team. Even though I conducted all the analyses, development, and testing activities, the guidance, feedback, and corrections provided by my supervisors were invaluable.

- **P1:** Raffa, G., Alis, J. B., O'Keeffe, D., and Dash, S. K. (2023). Awsomepy: A dataset and characterization of serverless applications. In *Proceedings of the 1st Workshop on SErverless Systems, Applications and MEthodologies*, SESAME '23, page 50–56 [123]. ***This paper is included in Chapter 3***.

- **P2:** Raffa, G., Blasco, J., O'Keeffe, D., and Dash, S. K. (2024). Towards inter-service data flow analysis of serverless applications. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 654–658 [124]. ***This work provides the initial results of the research presented in Chapter 4 and Chapter 5***.

- **P3:** Raffa, G., Blasco, J., O'Keeffe, D., and Dash, S. K. (2025). CloudFlow: Identifying Security-sensitive data flows in serverless applications. In *34th USENIX Security Symposium (USENIX Security 25)*, Seattle, WA. Accepted for publication [125]. ***This work provides the final results of the research presented in Chapter 4 and Chapter 5. In addition, it includes the analysis of the AWSomePy dataset discussed in Chapter 6.***

Chapter 2

Background

In this chapter, we provide the reader with all the required background information. After an overview of the serverless computing paradigm, where we explain its advantages and disadvantages (§2.1), we discuss the challenges of securing serverless applications (§2.2). We next illustrate the concept of Infrastructure as Code (§2.3), and then describe the code analysis frameworks assessed (§2.4). Considering the importance of Python in our work, relevant language-specific notions and tools are introduced as well (§2.5). The chapter ends with a summary of the concepts presented (§2.6).

2.1 Serverless Computing Paradigm

In recent years, many companies have adopted the *serverless* computing paradigm, also referred to as *FaaS* (Function as a Service), a relatively recent cloud computing model. Software engineers, in fact, have recognized that to successfully develop and maintain complex applications, these need to be implemented as cloud-based systems featuring loosely coupled components, thus abandoning more traditional monolithic architectures. With microservices-oriented design approaches becoming more prominent, Amazon Web Services (AWS) [6] released the Lambda platform [13] in 2014. The latter was the first commercially available FaaS environment, which can be seen as an evolution of the pre-existing Infrastructure as a Service (IaaS) and Platform as a Service (PaaS) models [160]. The serverless computing paradigm has since changed how modern cloud applications are developed.

With regard to the platforms currently available, AWS, which has the largest service catalogue on the market, is the most popular[1], though both Microsoft with Azure [89] and

[1] References with statistics on the popularity of the AWS platform are provided in Chapter 3.

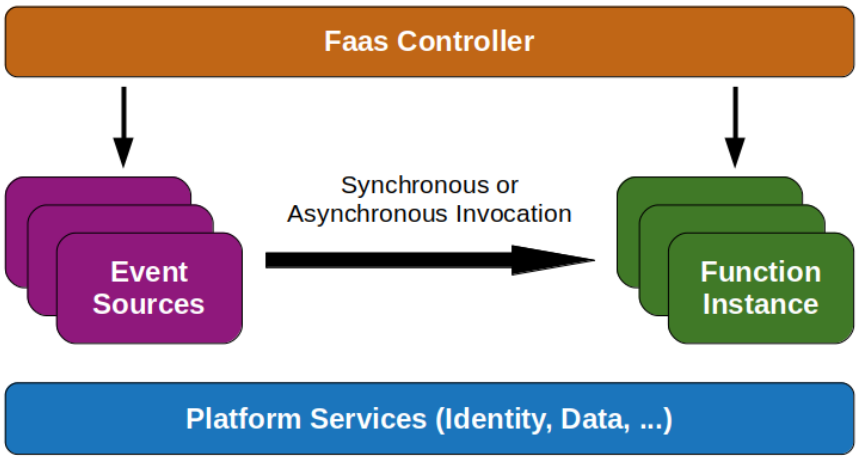

Fig. 2.1 Key elements of a FaaS solution [33].

Google with its Cloud Platform system [61] have competitive offerings. While the serverless model is more enterprise-oriented [96], open-source solutions such as OpenWhisk [99], OpenFaaS [97], OpenLambda [98], and Knative [32, 78] are available as well.

In the remainder of this section, we provide an overview of the serverless computing paradigm (§2.1.1). Next, we introduce and explain its advantages and disadvantages (§2.1.2).

2.1.1 Overview

As shown in Fig. 2.1, the serverless computing model enables the execution of applications that leverage backend services fully managed by a cloud provider. Therefore, serverless is a misnomer, as the execution of the deployed applications still relies on servers, but their provisioning, maintenance, updates, scaling and capacity planning are delegated to another organization [33, 75][2]. As further clarified in §2.1.2, in many cases this has cost-related advantages, as it enables modern enterprises to focus on implementing business logic and delivering features, rather than managing the supporting infrastructure.

Serverless applications are commonly implemented by integrating several event-triggered, short-running, and stateless functions, also known as *handlers*. These provide the application functionality, and constitute the *application code*. Such applications also require the definition of a set of cloud-based resources, e.g., file repositories and databases. These are specified in the *infrastructure code*, along with permissions, events, and handler-related configuration parameters.

If supported by the serverless environment, the infrastructure code can additionally include *event filters*. These allow triggering the execution of code only when additional con-

[2]The citations of [75] included in §2.1 refer to *Introduction to Serverless* (Pages: xxi-xxxiv).

ditions are met, e.g., the update of a specific folder in a cloud-based repository. Furthermore, depending upon the adopted deployment technology, the infrastructure definition can also include *extension plugins*. While simplifying some developer tasks, these affect the definition of cloud resources as well as the application architecture and permissions.

As explained in §2.3, there are both declarative and imperative approaches to the development of infrastructure code, although recent research indicates that declarative approaches are more popular [45, 46]. Regardless of the specific implementation choices, we emphasize that the infrastructure code contains security-related information that is critical for static analysis of serverless applications.

2.1.2 Advantages and Disadvantages

In this section, we illustrate the advantages of serverless platforms from an infrastructure management, cost-effectiveness, and developer productivity point of view. Moreover, we explain their disadvantages in terms of application responsiveness and cloud provider-imposed limits. Note that testing and debugging-related challenges could be elucidated in this section as well. Since they are particularly relevant to the research presented in this thesis, we choose to discuss them in §5.5 and §6.7.

Infrastructure Management. Serverless environments feature on-demand scalability and high reliability [75]. Companies adopting them can therefore prioritize implementation and delivery of application logic without planning for future capacity. However, the limits imposed by the cloud provider, further discussed below, and the interactions with non-serverless components need to be taken into account as well. In addition, it is crucial to observe that delegating scalability and maintenance-related tasks to another organization does not imply that security is outside the scope of enterprises relying on serverless platforms. As illustrated in Fig. 2.2, in fact, the *shared responsibility model* [75, 108], a general cloud computing concept especially relevant to serverless environments, establishes that there are tasks, e.g., installation of OS security updates, for which only the cloud provider is responsible. Enterprises still have to test their code though, which implies considering serverless-specific security challenges (§2.2).

Cost-effectiveness. Another attractive feature of serverless platforms is the *pay-per-usage* billing model [75]. As the cost for running applications depends on how often they are executed, enterprises no longer have to pay for idle time, which in the past was avoided by simply switching off underutilized platforms, for instance during holiday seasons or at weekends. In many cases, serverless platforms therefore guarantee availability in a cost-

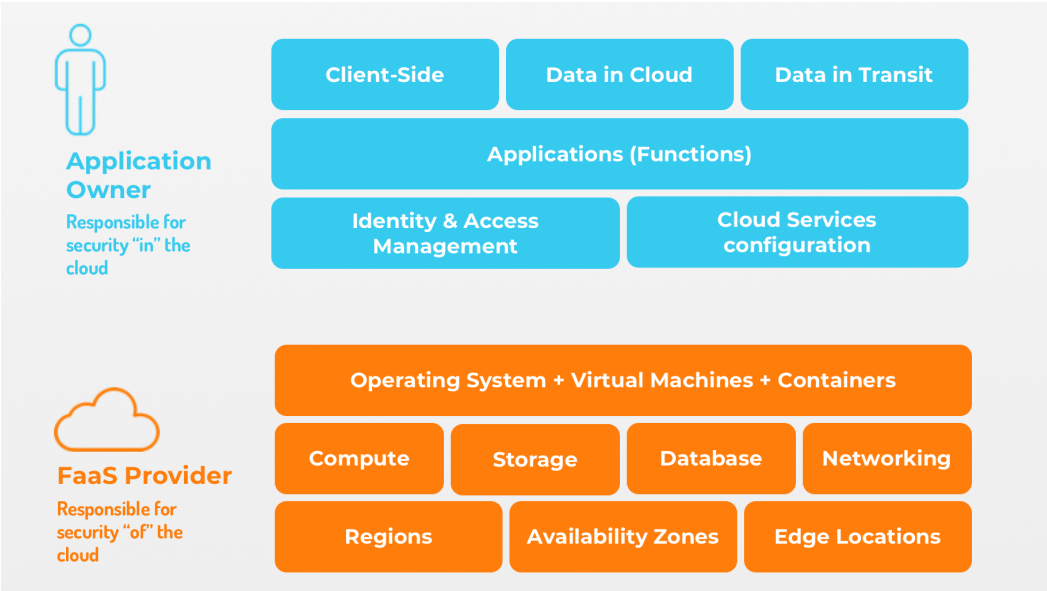

Fig. 2.2 Shared responsibility model [108].

effective manner, as the cloud environment spins up the resources necessary to execute a serverless function if an unexpected request arrives, e.g., to an Internet-facing application. A sudden increase in workload can also be efficiently managed. As the service provider is in charge of scalability, software-focused companies no longer have to bear the costs of a team responsible for capacity planning and related tasks.

Developer Productivity. Serverless platforms also allow increasing developer productivity [75]. Implementing applications suitable for these implies adopting a finely grained architectural model that consists of small functions triggered by events, e.g., an API request or the update of a database. While these architectural features can be an obstacle when migrating legacy software, developing applications by chaining together custom code and platform services with events can simplify the entire process and decrease the development time. Cloud-based services, in fact, provide complex pieces of functionality that developers can leverage to implement their custom business logic. For instance, AWS Cognito [10] allows managing user pools. Among other features, as illustrated by Shields [144, Ch. 11], this service enables the configuration of federated sign-in, and can send new users a verification code to confirm their account. As a result, developers can integrate such features in their applications without any additional implementation effort.

Application Responsiveness. The responsiveness of serverless applications is affected by *cold start* [75]. This occurs when an application invokes a function, but the latter cannot

be executed because there are no readily available resources. This can lead to insufficiently responsive applications, and explains why functions are frequently kept *warm*, i.e., ready to be executed in the cloud, despite the inevitable increase in cost. As pointed out by other authors [75, 157], solutions based on Content Delivery Networks (CDN) are being considered along with *snapshotting*, whereby faster invocation is achieved by employing an image of a fully-booted function stored on disk. A study recently conducted by Ustiugov *et al.* [157] with their open-source framework vHive [161], shows that an improved methodology based on optimized memory management can reduce cold-start delays nearly four times on average.

Cloud Provider-imposed Limits. Obetz *et al.* [96] emphasize that serverless platforms impose strict limits on function execution times. According to Katzer [75] though, it is possible that cloud providers will relax these constraints over time. AWS, in fact, increased the limits on its Lambda platform from five to fifteen minutes in 2018. As a result, implementing complex applications by linking together several small functions in event-driven architectures is the most reliable solution. However, this means handling numerous software components, which, in turn, causes problems during debugging, e.g., while tracing an user action throughout the entire system. Although increasingly sophisticated monitoring services, such as AWS CloudWatch [9], can greatly facilitate these tasks, testing and debugging serverless applications remains a challenge [80].

In summary, the advantages offered by serverless platforms in terms of scalability and cost reduction make this computing paradigm an attractive option for applications consisting of small and independent steps. This is especially true if their usage pattern is infrequent or unpredictable [75]. By contrast, if computationally-intensive or long-running tasks are required, alternative platforms or architectures [42] should be considered. We finally observe that, when responsiveness is an issue, the undesirable effects of a cold start can be mitigated by employing a serverless solution only for background tasks.

2.2 Securing Serverless Applications

Securing serverless applications is a challenging task. Adopting the shared responsibility model (§2.1.2), in fact, does not imply that only the cloud provider is responsible for security. Development teams that intend to leverage serverless platforms still have to take into account generic principles, such as *least privilege* [60, Ch. 10], test their code to identify vulnerabilities, and scrutinize all external software dependencies [75, Ch. 9]. In addition, they have to consider serverless-specific aspects. These include the correct configuration of cloud-

based services, and the absence of traditional countermeasures, e.g., intrusion prevention and detection systems, which, in a FaaS environment, are managed by the provider. Consequently, as emphasized by the OWASP Serverless Top 10 project [101, 100], migrating to a serverless architecture does not mean eliminating security risks, but rather considering different ones.

The analysis published in 2019 by the company PureSec [108], now part of Palo Alto Networks [102], explains that serverless applications are complex systems characterized by a large attack surface, as their functions consume data from a wide range of event sources, such as HTTP APIs, cloud storage facilities, and IoT devices. To secure these systems, infrastructure providers offer services dedicated to authentication, authorization, and cryptographic key management that developers should systematically use [133, Ch. 6]. In addition, as highlighted in the OWASP report [101] and by Katzer [75, Ch. 9], the general advice is that almost each serverless function should have a specific set of permissions. This approach is time-consuming though, and requires expertise not always available in the development team.

From a security testing point of view, which is particularly relevant to this research, the PureSec report also underscores that both static and dynamic automated scanning tools do not fully meet the requirements of serverless environments. Specifically, *static application security testing* (SAST) tools, which typically rely on data and control flow analysis, are rather inaccurate. This, as explained in our challenge C1, is determined by the presence of numerous event triggers and cloud services, which increase the amount of sources and sinks to inspect, and cause interactions difficult to analyse. In addition, *dynamic application security testing* (DAST) tools are still being improved to stimulate the software under test with a variety of suitable inputs (i.e., including non-HTTP sources), and to properly monitor the interactions with the backend services.

To further illustrate the security challenges involved in the development of serverless applications, this section begins by elucidating the most important risks that affect them (§2.2.1). Since the most critical of such risks is code injection, an example of a serverless application with this type of vulnerability is next discussed (§2.2.2). Finally, the section introduces the identity and access management service (§2.2.3), a key tool to secure serverless applications available on all platforms.

2.2.1 Critical Risks for Serverless Applications

Having provided an overview of the most important factors that affect the security of serverless applications, the purpose of this section is to illustrate the associated risks. As part of this research, the list presented in the 2018 OWASP Serverless Top 10 report [101] was compared to the one published by the Cloud Security Alliance (CSA) in 2019 [34], which

Table 2.1 Critical risks for serverless applications according to the CSA [34].

Identifier	Critical Risk
SAS-1	Function Event Data Injection
SAS-2	Broken Authentication
SAS-3	Insecure Serverless Deployment Configuration
SAS-4	Over-Privileged Function Permissions & Roles
SAS-5	Inadequate Function Monitoring and Logging
SAS-6	Insecure Third-Party Dependencies
SAS-7	Insecure Application Secrets Storage
SAS-8	Denial of Service & Financial Resource Exhaustion
SAS-9	Serverless Business Logic Manipulation
SAS-10	Improper Exception Handling and Verbose Error Messages
SAS-11	Obsolete Functions, Cloud Resources and Event Triggers
SAS-12	Cross-Execution Data Persistency

further expands on that proposed by PureSec [108]. Despite the partial overlap, while the first is undoubtedly an important starting point, the second is more comprehensive and better captures the specifics of current serverless platforms. Therefore, the CSA classification, which is arranged in order of criticality in Table 2.1, is a key reference of this work.

Injection attacks pose the most critical risk. Even though they are well-studied, serverless platforms provide a different context. To better illustrate this, we present two code injection examples. The first is discussed in §2.2.2, whilst the second, which includes a higher number of cloud services, is described in §4.2.1. It should be noted that code injection vulnerabilities are among those identified in the `AWSomePy` dataset (§6.3).

Despite the availability of dedicated platform services, we also observe that broken authentication is considered a major risk both by the CSA and the OWASP report. Serverless applications typically comprise several functions and cloud services that require authentication, with the latter often implemented by using different schemes due to the constraints of the integrated components. Since testing serverless applications is, in general, challenging (§2.2), adopting dedicated platform services, e.g., AWS Cognito [10], rather than deploying customized authentication schemes, is crucial to mitigating this risk.

Insecure serverless deployment configuration, which is a generalization of the sensitive data exposure issue highlighted in the OWASP report, is another significant risk. Customizations affecting cloud-based repositories and databases, however, remain particularly critical. As confirmed by the results of the `AWSomePy` dataset characterization (§3.3.2), such resources are an ubiquitous feature of the studied applications, as their functions are *stateless* and need to store information permanently. While vendor-specific best practices exist, the absence of

widely adopted industry standards and tools to secure serverless applications is an important contributing factor.

Unsurprisingly, over-privileged function permissions and roles, which are reported in the OWASP report under the broken access control category, are a major risk as well. As illustrated in §2.2.3, identity and access management services are complex, which implies that inexperienced developers and administrators are likely to make mistakes. Furthermore, as previously mentioned (§2.1.2), microservices-oriented design approaches inevitably lead to a high number of functions. According to well-established best practices, almost each function should have its own permission policy. The characterization of `AWSomePy`, however, shows that development teams tend to manage their functions' permissions by using application-wide policies (§3.3.1).

As regards more serverless-specific risks, the first examples are denial of service and financial resource exhaustion, with the latter also known as Denial of Wallet (DoW). Since serverless providers define per-function and per-account concurrent execution limits, if an attacker succeeds in repeatedly triggering workflows, the application will no longer be available when one of the limits is reached. Similarly, malicious actors can also attempt to execute functions for a prolonged period of time, for instance by exploiting vulnerabilities caused by regular expressions [103]. In both cases, such actors will inflict a financial loss on the target organization.

Finally, it should be observed that attacks aiming at altering the business logic of an application can have a considerable impact. They are an alternative form of the above-described denial of service and DoW attacks. In this case, the objective is to bypass the access control system by replicating an event[3] that triggers the execution of a function. For this reason, designing the software without making any assumptions about the invocation flow, which implies using fine-grained permissions and suitably configured access control mechanisms, is considered the best approach to mitigate this risk.

2.2.2 Example of Vulnerable Serverless Application

The high-level diagram of Fig. 2.3 shows an example application with a code injection vulnerability[4]. Its user-controlled entry point is a HTTP Post request. This triggers the execution of a handler that uploads a file identified through the body of the request to a cloud-based repository (Listing 2.1). Upon detecting the new file, the S3 service processes

[3]For example, an email sent to a specific recipient.

[4]This example is the `CloudBench` microbenchmark `api-put-item-boto3-client`, and is further described in the paper P2. It is also noteworthy that the application is compatible with the AWS platform [6], but this does not affect the generality of the presented observations.

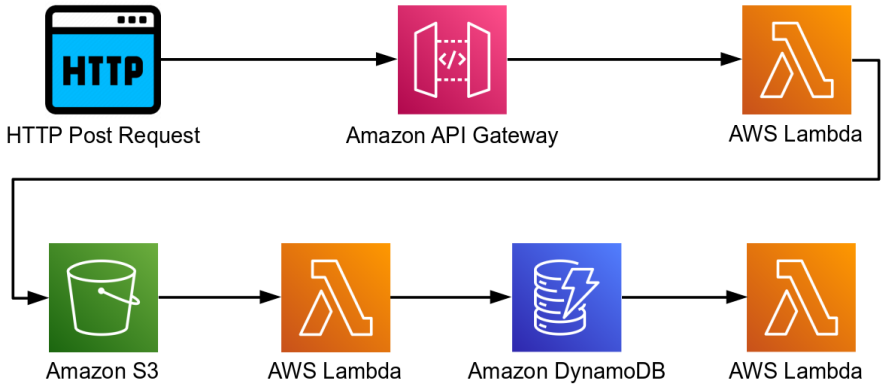

Fig. 2.3 Example application with an event-driven data flow and a code injection vulnerability. The blocks AWS Lambda identify the three event-triggered handlers included in the application.

the event and runs the second handler, which extracts information from the uploaded file and stores it in a DynamoDB database. This triggers a third handler that uses the newly-stored item in the database as part of a system command called via `os.system` (Listing 2.2). The latter is therefore executed with an input controlled by the user, which implies that this is a high-risk scenario.

```
def onHTTPPostEvent(event, context):
    msgBodyDict = event['body']
    uploadFileFolder = 'uploads'
    uploadFileName = msgBodyDict + '.txt'
    s3BucketKey = os.path.join(uploadFileFolder, uploadFileName)
    localFile = os.path.join('/tmp', uploadFileName)
    s3Client = boto3.client('s3')
    s3Client.upload_file(localFile, os.environ['BUCKET_NAME'],
   s3BucketKey)
    return
```

Listing 2.1 Handler triggered by a HTTP request.

From a static analysis point of view, while manual analysis of the application, aided by the architectural diagram, allows identifying a data flow from the user-controlled input to the security-sensitive sink `os.system`, its automatic detection is not possible with general-purpose tools. These, in fact, do not take into account the effect of triggered events, e.g., the fact that the API call `upload_file` in Listing 2.1 triggers the execution of another handler. As explained while introducing challenge C1, this is a cloud interaction that is not part of the synchronous execution of the application. Information about such interactions is only present in the infrastructure code, which must therefore be incorporated into the analysis.

Furthermore, a non specialized tool would not be capable of modelling the `event` inputs expected by the handlers, due to a lack of cloud service-specific information. This explains why `CloudFlow` relies on event object models (§5.2.1), and, more generally, further clarifies the importance of challenge C3.

```
def onDynamoDBStream(event, context):
    authorsInfo = event['Records'][0]['dynamodb']['NewImage']['authors']['S']
    titleInfo = event['Records'][0]['dynamodb']['NewImage']['title']['S']
    eventData = authorsInfo + titleInfo
    os.system('echo %s' % eventData)
    return
```

Listing 2.2 Handler triggered by a DynamoDB database update.

2.2.3 Identity and Access Management

All serverless providers offer an identity and access management (IAM) service focused on user authentication and authorization. This is a crucial security-related component used to accomplish several tasks, such as granting granular permissions, also known as *credential scoping* [133, Ch. 6], deploying multi-factor authentication (MFA) and managing federated identities, with the latter being leveraged to access cloud-based resources with corporate credentials or through external identity providers, e.g., Google.

AWS, in particular, provides a highly complex IAM service, which can be accessed in several ways, for instance via the AWS command-line interface [144, Ch. 2] or HTTPS APIs [12]. In essence, the AWS IAM allows the management of *identities*, which include *users*, *groups*, and *roles*, through the application of one or more sets of rules known as *policies*. This means that every time an *entity*, i.e., an authenticated identity, makes a request to access or modify a resource, e.g., a database, the IAM double-checks the relevant policies prior to authorizing the request. It is important to emphasize that, in this context, a user can be associated to an application, and that policies have to be written to explicitly authorize a request, otherwise the latter will be denied by default.

To clarify the concept of policy, an AWS-specific example is provided in Fig. 2.4 [12]. In most cases, AWS policies are stored as JSON documents, which explains the syntax used in the provided example. Regarding the permissions specified by the policy, the user is allowed to perform all DynamoDB [11] actions (`dynamodb:*`) on the `Books` database table in the account `123456789012` within the region `us-east-2` of the infrastructure. We note that this is an example of *identity-based* policy, because it controls what actions can be performed by

```
{
  "Version": "2012-10-17",
  "Statement": {
    "Effect": "Allow",
    "Action": "dynamodb:*",
    "Resource": "arn:aws:dynamodb:us-east-2:123456789012:table/Books"
  }
}
```

Fig. 2.4 AWS-specific example of IAM policy [12].

the identity to which it is attached. This means, in particular, checking the resources that are affected by such actions.

Identity-based policies can be further categorized in *managed*, which are stand-alone and can be associated to multiple identities, and *inline*, which are directly specified as part of an identity and, therefore, more difficult to reuse. Those belonging to the first category are either *customer-managed* or *AWS-managed*, with the latter being maintained by the provider to meet the most common user requirements. In addition, AWS supports *resource-based* policies, which control what actions a specified entity can perform on a given resource and under what conditions [144, Ch. 2].

It is important to note that the above-described policy mechanism supports the well-established role-based access control (RBAC) model [60, Ch. 9]. Role-specific policies, where a role frequently corresponds to a person's job function, list all the resources that can be accessed as well as the permitted actions. However, differently from identity-based policies, role-based policies remain in place only for a limited period of time, at the end of which all the role permissions are revoked.

Managing policies developed to support RBAC can be time-consuming though, as they have to be updated every time a new resource, e.g., an additional database, is added to the set that a given role should be able to access. For this reason, AWS also supports an alternative authorization strategy, known as attribute-based access control (ABAC), which defines permissions based on attributes. These are specified as *tags*, which are added to IAM identities and resources. When an entity sends a request, the latter is authorized only if the ABAC tags match. The advantage of this access control scheme is that, if new resources are created with the necessary tags, then no policy updates are needed, which is particularly advantageous in large, multi-project enterprise environments [12].

2.3 Infrastructure as Code

To further illustrate the foundations of the serverless computing paradigm, it is necessary to introduce the wider concept of *Infrastructure as Code* (IaC), which, as pointed out by Morris [91, Ch. 1], has recently become a cornerstone of modern software engineering. The advent of cloud-based technologies and infrastructure has created a different and challenging environment for IT architects, system administrators, and software developers. Old approaches to managing deliverables, which nowadays include infrastructure as well, are unsuitable for the *Cloud Age* [91, Ch. 1], as they were developed when static pre-cloud systems were mainstream. By contrast, modern IT architectures are highly dynamic and require constant changes to integrate new applications and features, provide better scalability and, even more importantly in the context of this work, guarantee adequate levels of security.

The concept of IaC, which can be defined as an approach to infrastructure automation based on practices from software development [91, Ch. 1], was conceived to meet these challenges. From a practical perspective, the previous definition has three important implications. The first is that every infrastructure component is defined as code, thus achieving reusability, consistency, and transparency. The second implication is that automation can be used to test and deploy new components and configurations. Finally, adopting IaC implies developing codebases that consist of small and well-documented parts managed via a Version Control System (VCS), thus overcoming the maintenance-related issues caused by large and tightly coupled systems.

All the above-mentioned IaC core practices are important to understand how serverless applications are developed and secured. In particular, the principle that every component is defined as code is the most relevant to this thesis. In essence, this means that infrastructure definitions, which include both the necessary cloud-based resources and their configurations [91, Ch. 4], are managed through text files containing, as further clarified in the remainder of this section, either structured configuration parameters or source code. Therefore, IaC files include several different pieces of information, e.g., names of cloud-based repositories and packages to be installed. Importantly, security implications have to be taken into account while defining such resources. IaC files, for instance, should not contain passwords or cryptographic keys [91, Ch. 7].

Since a significant amount of information is needed to manage a modern IT system, it is important to understand how the concept of defining every infrastructure component as code can be put into practice. There are three different approaches, which rely on *declarative*, *imperative* and *domain-specific* languages. When a declarative approach is used, for instance through YAML [173] (Fig. 2.5), the focus is on what has to be achieved, but not how. The chief implication of this is that declarative code is seen by many practitioners as a

```
virtual_machine:
  name: my_application_server
  source_image: 'base_linux'
  cpu: 2
  ram: 2GB
  network: private_network_segment
  provision:
    provisioner: servermaker
    role: tomcat_server
```

Fig. 2.5 Example of declarative code for IaC [91, Ch. 4].

configuration file, given that it does not include any logic[5]. By contrast, as shown in Fig. 2.6, an imperative IaC file typically contains parameters as well as statements that implement checks, loops or similar programming constructs. Among the frameworks that allow the definition of infrastructure with a general-purpose programming language, Pulumi [107] is one of the most widely adopted. Domain-specific languages (DSLs), which normally rely on a declarative model [91, Ch. 4], are an alternative way of providing developers and system administrators with a tool to write and understand infrastructure code. Several DSL-based frameworks, such as Puppet [48] and Terraform [29], currently exist.

The IT industry has not yet identified a standardized approach to implementing IaC, which implies that infrastructure codebases sometimes rely on a mixture of tools and languages [91, Ch. 4]. Declarative code, in fact, supported or not by a DSL, is more suitable to define the desired state of a system, whereas imperative languages are a better choice to implement logic and abstraction layers. Recent research on serverless applications [45, 46], however, suggests that declarative infrastructure code tools are more widely adopted in this specific domain.

2.4 Overview of Assessed Code Analysis Frameworks

In this section, we present an overview of existing code analysis frameworks that were assessed during the initial stages of our work to identify possible baselines. Considering the vast number of research contributions in the area, this overview is not comprehensive. However, it provides the reader with useful background information that illustrates why

[5]Declarative infrastructure code is particularly important for the presented research. As illustrated by the examples in §2.2.2 and §4.2, the proposed static analysis framework targets applications deployed with this type of infrastructure code.

```
this_country = getArgument("country")
data_centers = CloudApi.find_data_centers(country: this_country)
full_ip_range = 10.2.0.0/16

vlan_number = 0
for $DATA_CENTER in data_centers {
  vlan = CloudApi.vlan.apply(
    name: "public_vlan_${DATA_CENTER.name}"
    data_center: $DATA_CENTER.id
    ip_range: Networking.subrange(
        full_ip_range,
        data_centers.howmany,
        data_centers.howmany++
    )
  )
}
```

Fig. 2.6 Example of imperative code for IaC [91, Ch. 4].

serverless-focused frameworks are needed. Some of the specifics of our approach, further explained in Chapter 5, are introduced as well.

We begin by discussing a selection of frameworks designed to analyse event-driven applications (§2.4.1). We then present a set of tools for data flow analysis proposed in academia (§2.4.2). It is noteworthy that all the described frameworks are language-specific, as, despite recent progress [83], very few of them support multilingual software systems.

2.4.1 Frameworks for Event-driven Applications

Madsen *et al.* [85] extend the traditional notion of call graph by proposing the concept of *event-based* call graph. The authors focus their attention on Node.js JavaScript applications, where event-triggered code plays a crucial role. The work emphasizes that inter-procedural static analysis that does not consider event-triggered data flows is insufficient to debug such applications and detect their security vulnerabilities. This is also a crucial problem in the serverless domain, but the algorithms proposed by Madsen *et al.* cannot be easily adapted to statically analyse serverless applications. The key reason that justifies this is that such algorithms do not extract information from the infrastructure code, an essential step in the methodology proposed in this thesis (§5.2). Specifically, as summarized in Table 2.2, the authors extend existing JavaScript operational semantics [62] with constructs for event listener registration and event emission, the serverless equivalent of which are handled by the

Table 2.2 Summary of assessed frameworks for event-driven applications.

Framework	Reference	Target Platform	Approach
Radar	[85]	Node.js	Operational semantics
Node.fz	[41]	Node.js	Fuzzing
NodeRacer	[44]	Node.js	Code instrumentation
nAdroid	[49]	Android	Program representation

cloud platform via the infrastructure code. Furthermore, Madsen *et al.* rely on the assumption that concurrent executions of multiple event handlers are severely limited, which is generally a mismatch for modern serverless platforms, as they feature on-demand scalability. Similar considerations hold true for Node.fz, proposed by Davis *et al.* [41]. The work presents a novel fuzzing test tool for Node.js event-driven applications, which is specifically designed to detect concurrency bugs, i.e., atomicity and ordering violations. It is noteworthy that such bugs are frequently caused by race conditions on system resources, such as databases. While these issues can also affect, in principle, applications deployed to a serverless platform, they are normally handled by the cloud-based services, thus lessening the burden on the FaaS developer.

More recently, Endo *et al.* [44] have proposed NodeRacer, a tool that implements an event race detection technique targeting Node.js applications. The authors reduce the number of false positives obtained with the aforementioned Node.fz by adopting a code instrumentation-based approach. Specifically, Endo *et al.* instrument the application under test to log event-related information. The latter is subsequently used to identify a happens-before relation, which allows studying selected event handler execution orders. Unlike Node.fz, the technique does not require modifying the platform internals, which greatly facilitates its deployment and, in principle, its reuse in different domains. Although NodeRacer is able to detect a wider set of bugs than Node.fz, its approach is unsuitable for a serverless platform, where, as stated above, events are managed by cloud-based services. We also highlight that the work relies on a different type of code instrumentation compared to ours. While `CloudFlow` makes use of code synthesis and injection (§5.2.1), NodeRacer runs *hook functions*, which log relevant information during the lifecycle of an event, e.g., immediately before a callback is executed. NodeRacer, therefore, cannot be classified as a static analysis tool.

In contrast, nAdroid, proposed by Fu *et al.* [49], statically detects ordering violations in Android applications. The framework is conceived to address the challenges posed by the hybrid concurrency model used by modern mobile applications. These, in fact, are event-driven, but also capable of offloading long-running tasks to dedicated threads in order

Table 2.3 Summary of assessed frameworks for data flow analysis.

Framework	Reference	Target Platform	Approach
Doop	[28]	Java	Static analysis
FlowDroid	[7]	Android (Java)	Static analysis
SootUp	[74]	Java	Static analysis
TaintDroid	[43]	Android (Java)	Dynamic analysis

to enhance their responsiveness. Albeit convenient from a developer standpoint, the co-existence of such features implies that ordering violations affect Android applications in multiple ways. The work by Fu *et al.* is an important research contribution, but tailored to a specific programming model and, as such, not transferable to others. We underscore that nAdroid relies on an intermediate program representation that models the execution of event handlers as threads. As a result, the analysis of the application under test can be conducted with an existing race detector for multi-threaded programs. Similarly, `CloudFlow` leverages a FaaS model-specific program representation (§5.2.4) that facilitates the use of a general-purpose static analysis tool.

2.4.2 Frameworks for Data Flow Analysis

Over the last two decades, a wealth of research has been devoted to static analysis frameworks that can be used to identify data flows. Originally proposed in 2000, Soot [158] is one of the most well-known among them, and it has been used by several researchers. The framework has recently been reimplemented and renamed SootUp [74], which includes a number of architectural changes and an improved underlying intermediate representation aiming at supporting multiple languages. While the latter feature means that SootUp might be leveraged to analyse JavaScript and Python code in the near future, the framework is currently compatible only with Java. As summarized in Table 2.3, the same observation holds true for other works, for instance Doop [28], which enables configuring static analyses declaratively and relies on an intermediate representation based on that used by Soot. Despite supporting only one language, these frameworks are significant research contributions, especially because they are the baseline for other, more security-focused tools.

Among them, one of the most important is FlowDroid, an Android-specific static taint analysis framework proposed by Arzt *et al.* [7]. FlowDroid features context, flow, field and object sensitivity. Its principal aim is the identification of data leaks caused by privacy-sensitive data flows present in insufficiently tested or malicious applications. To improve coarse-grained over and under approximations on which previous research contributions rely,

FlowDroid is built on top of the Soot and IFDS [128] frameworks. These are extended with a model for Android applications' lifecycle and novel algorithms for alias analysis. The authors also open-source their microbenchmark suite, DroidBench [150], which is used to show that their framework outperforms two commercially available tools in terms of recall and precision.

We highlight that there are similarities and differences between the work by Arzt *et al.* and the research presented in this thesis. With regard to the similarities, the detection of information leakage is an important objective in both cases. Moreover, both works propose a novel collection of microbenchmarks. As observed while discussing other frameworks designed for Android (§2.4.1), however, its programming model is significantly different from that adopted in serverless environments. Android applications rely on a set of user-developed components, such as *activities*, *services*, and *content providers*, that cannot be matched one-to-one with those included in serverless applications. While activities resemble serverless handlers, the functionality provided by services and content providers, i.e., background tasks and database-like storage respectively, are typically part of cloud-based services. We also note that Android applications make use of event-driven callback handlers specified either declaratively in manifest files or imperatively through system methods. The first case is similar to the declarative definition of infrastructure code used in serverless applications, but the file formats[6] and contents are inevitably different, thus requiring a serverless-specific approach (§5.2.2).

For completeness, we observe that data flows can be identified with dynamic taint tracking approaches as well. One of the most prominent examples is TaintDroid [43], a system-wide information flow detection tool for Android. Implemented as a firmware-level extension, TaintDroid takes into account the specifics of the target OS by integrating multiple levels of taint propagation granularity. This enables the identification of data flowing through variables, files, and application-to-application messages. The evaluation results show that TaintDroid is able to identify cases where applications send location data, phone numbers, and device IDs to remote servers without notifying the user.

Finally, in addition to the aforementioned differences between the Android and the FaaS programming model, we underscore that all the frameworks discussed in this section are designed to analyse applications implemented in Java (Table 2.3). This language, however, is not widely adopted in the serverless domain [45]. As further justified in §2.5, we therefore instantiate the methodology proposed in this thesis into an analysis framework focused on

[6]The Android configuration files analysed by FlowDroid are in XML format. As illustrated in Chapter 5, the declarative infrastructure code processed by `CloudFlow` is specified in YAML files. We are not aware of declarative serverless deployment tools that rely on XML files.

Python applications, though its extension to other languages is a possible avenue for future research (§5.4).

2.5 Static Analysis of Python Code

According to the characterization of the Wonderless dataset [45], Python is the second most popular language in the serverless domain, with JavaScript being the first. An assessment conducted at the beginning of this research project, however, revealed that the Python ecosystem includes static analysis tools more mature than those available for JavaScript. Recent work focused on the latter [30], in fact, shows that JavaScript static analysis tools are affected by significant limitations due to the highly dynamic features of the language and the complex dependency system. For these reasons, the implementation of the static analysis methodology proposed in this thesis (§5.2.1) is focused on Python[7].

However, analysing Python code statically is not without challenges. As pointed out by Viafore [162, Ch. 2], the language is strictly typed, but also dynamically typed. From a static analysis standpoint, the latter feature is particularly problematic, as it implies that variables can change their type at runtime. Furthermore, Kummita *et al.* [79] have recently demonstrated that this and other dynamic features, such as *duck typing* [162, Ch. 2] and the built-in function `eval` [118], lead to the generation of inaccurate static *callgraphs* (CGs), key data structures for the execution of inter-procedural static analyses[8]. The authors, who use a suite of microbenchmarks and a collection of real-world programs for the evaluation of three CG generation frameworks, show that none of these produce sound CGs. This implies that the obtained CGs are incomplete, as some of the edges to possible runtime call targets are missing. Moreover, Kummita *et al.* report that the number of spurious edges, i.e., not possible at runtime, is higher than expected, with one of the assessed frameworks exhibiting precision and recall close to zero when evaluated against real-world programs.

To overcome at least some of these challenges, the Python community has proposed the adoption of an optional type system, which is described in §2.5.1. We then conclude this section by introducing Pysa (§2.5.2), the static analysis tool with data flow detection capabilities used to implement `CloudFlow`.

[7]Incidentally, Python is widely adopted in numerous domains. According to the Tiobe index, Python was the most popular programming language both in April 2024 and April 2025 [155]. While this does not imply that the language will be increasingly adopted in the future to implement serverless applications, this is not an unlikely scenario.

[8]A static callgraph links *call sites*, e.g., containing function invocations, to *call targets*, i.e., invoked code. In the proposed approach, a general-purpose static analysis tool obtains the CG of the analysed serverless application after the code synthesis and code injection steps shown in Fig. 5.1.

2.5.1 Python Type System

Dynamic language features, such as the aforementioned dynamic and duck typing, are one of the fundamental reasons behind the growing popularity of Python. However, the flexibility that they provide also poses some challenges. As previously mentioned, static analyses are negatively affected by such features, and codebase documentation becomes more complex to produce and maintain [65, Ch. 6]. To address these issues, *type hinting* was first proposed more than a decade ago [121], and has been part of the language since version 3.5, released in 2015.

As illustrated in Listing 2.3, type hints or *type annotations*[9] allow developers to add information to their code, such as the type of a variable (e.g., `float` for the variable `pi`), the type of a function input (e.g., `float` for the input `radius`), and the type of a function output (e.g., `float` for the function `circumference`). Moreover, it is possible to annotate the type of objects included in built-in collections, which include lists, tuples, and dictionaries. The code excerpt in Listing 2.4 shows, for instance, how to specify a list of strings (`List[str]`), a tuple with three integers (`Tuple[int, int, int]`), and a dictionary where the keys are strings and the values are boolean (`Dict[str, bool]`)[10].

```
1 pi: float = 3.142
2
3 def circumference(radius: float) -> float:
4     return 2 * pi * radius
```

Listing 2.3 Variable initialization and function definition with type annotations.

Type annotations offer two key advantages. First, they improve the quality of the source code documentation. Hints, in fact, enable the developer to explicitly state their intentions, e.g., what the code expects as an input and returns as an output. The second advantage is that type annotations support the functionality provided by some development tools. Modern code editors, for instance, offer better code completion when hints are present [162, Ch. 3]. These also provide static type checkers, e.g., Pyre [111], and data flow detection tools, e.g., Pysa (§2.5.2), with additional information that increases the accuracy of their analyses.

[9] For readers interested in the history and evolution of Python, it is noteworthy that the term *annotation* was introduced by core language developers several years before type hinting. Annotations were initially conceived as a mechanism to add arbitrary expressions, i.e., without specific semantics, to function definitions [119]. Nowadays, the term annotation is mainly used to refer to type-related information added to the source code.

[10] Python type hints can be expressed by using a variety of constructs, such as aliases [51], constraining types [162, Ch. 4], and complex collection types [162, Ch. 5], not included in the presented examples. In addition, annotations can be specified in *stub files*, which are used when the original source code cannot be changed. Readers interested in learning more about the topic can find additional implementation-level details in the references provided in this section and the collection of examples by Gábor [20].

```python
from typing import Dict, List, Tuple

names: List[str] = ["String_A", "String_B", "String_C"]
version: Tuple[int, int, int] = (3, 7, 1)
options: Dict[str, bool] = {"centered": False, "capitalize": True}
```

Listing 2.4 Data structures with annotations specifying the type of the contained objects.

However, as highlighted by Viafore [162, Ch. 7] and Hjelle [51], adding type annotations to source code is time-consuming and, therefore, expensive. While this remains a significant issue, it is important to emphasize that Python supports the concept of *gradual typing* [120]. This implies that hints can be included gradually in the codebase after identifying the areas where they are more beneficial, e.g., newly added code. It is also worth noting that some static type checkers feature *automated type inference* [112], which is part of the implementation of `CloudFlow` (§5.2.3).

Finally, regarding possible future scenarios, we underscore that core Python developers have stated that the language will remain dynamically typed and will feature only an optional type system [121, 51]. In practical terms, this means that the Python interpreter will not be modified to enforce types[11]. Consequently, the identification of effective approaches to statically analysing Python code will likely be an active area of research for the foreseeable future[12].

2.5.2 Pysa Static Analyser

Developed by Facebook on top of the static type checker Pyre [111], Pysa, the tool chosen for the implementation of `CloudFlow`, is a Python-specific open-source tool capable of running inter-procedural analyses [113]. Its main feature is that it has been specifically designed to detect security-sensitive data flows in large code repositories. As illustrated in Fig. 2.7, the tool workflow starts with the identification of *sources* and *sinks*, which are linked via *rules* defining the targeted data flows. In addition, sources and sinks are specified in Pysa *models*, which can be described as *annotated* code, i.e., code containing additional textual information on sources and sinks. To minimize the number of false positives, the models can

[11] Tools that provide runtime type checking, e.g., Pydantic [109], exist though. They have interesting features and use cases [162, Ch. 14], but it is very unlikely that they will be added to the language standard library in the future.

[12] Attentive readers will have noted that this succinct introduction to the Python type system does not include any notion of formal type theory. We reckon that this is outside the scope of this research, which aims to identify a language-independent approach to statically analysing serverless applications. Readers interested in formal type theory concepts relevant to the Python type system can consult the Python Enhancement Proposal 483 [120] and the Python type checking guide by Hjelle [51].

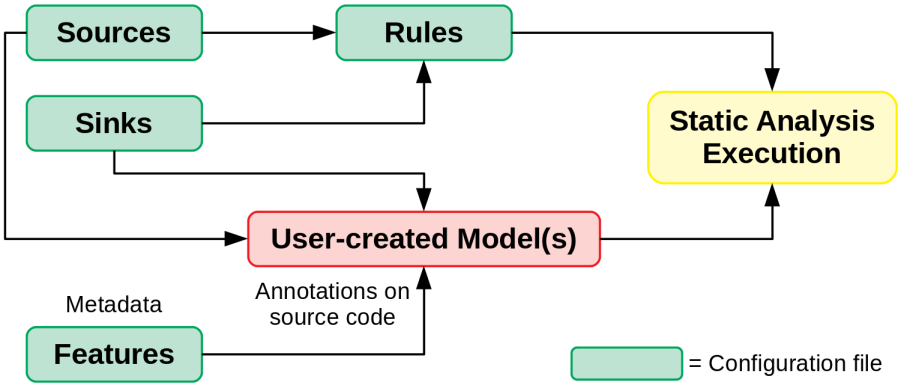

Fig. 2.7 Pysa workflow.

be enhanced by using metadata, also known as *features*, which allow, for instance, to specify whether a data flow has to go through a particular variable or function [117].

```
{
 "code": 6065,
 "message_format": "Data from [{$sources}] source(s) may reach [{
   $sinks}] sink(s)",
 "name": "Commandline arguments injection may result in RCE",
 "sinks": [
   "ExecArgSink"
 ],
 "sources": [
   "UserControlled"
 ]
}
```

Listing 2.5 Pysa rule example. The rule specifies that data flows between `ExecArgSink` sinks and `UserControlled` sources need to be identified to detect code injection vulnerabilities.

To exemplify the described workflow, Listing 2.5 provides an example of rule for the detection of flows between sinks categorized as `ExecArgSink` and user-controlled sources [115]. Listing 2.6, instead, shows a model for a Python function (`handler`). Specifically, the `event` input includes the annotation `TaintSource[Test]`, used by Pysa to identify data flows starting from that input. Similarly, Listing 2.7 illustrates a model for the cloud API `put_item`, where the `Item` input is annotated as a sink. Unlike the previous one though, the latter model uses a stub (§2.5.1) included in the module `mypy_boto3_dynamodb` [27, 26][13]. This approach is commonly adopted to develop models for cloud APIs. As shown by our

[13]This is necessary because the `boto3` library, the Python Software Development Kit for AWS that will be introduced in §3.3.2, does not include type annotations. Thus, Pysa needs to rely on stubs implemented for the widely adopted static type checker mypy [92].

discussions with the Pysa community [55, 56], however, such models are generally complex, which confirms the importance of the previously introduced challenge C3.

```
1
2 def publish_request_handler.handler(event: TaintSource[Test]): ...
```

Listing 2.6 Pysa model example. The `event` function input is annotated as a source.

```
1
2 def mypy_boto3_dynamodb.service_resource.Table.put_item(Item:
    TaintSink[Test]): ...
```

Listing 2.7 Pysa model example. The `Item` input of the cloud API `put_item` is annotated as a sink. The model relies on a stub included in the module `mypy_boto3_dynamodb`.

Finally, we observe that Pysa is not the only general-purpose tool capable of identifying data flows in Python code. As part of a toolchain-focused investigation conducted at the beginning of this project, Semgrep, a well-known static analysis framework [137], was considered as well. We evaluated, in particular, the community edition of the product, i.e., Semgrep CE. The analysis of the documentation available in February 2022, however, revealed that this version of the tool was able to perform exclusively single-file analyses [135]. Furthermore, the data flow analysis engine [136] was still in beta version, and it only had intra-procedural capabilities. Both limitations made Semgrep unsuitable for our research. Serverless applications, in fact, typically consist of code stored in multiple files, and inter-procedural analyses are essential to detecting data flows between event-triggered handlers. On reviewing the tool documentation in April 2025, we note that the licence-protected version of Semgrep is now capable of performing cross-file and inter-procedural analyses [134]. Future implementations of the methodology proposed in this thesis might therefore rely on Semgrep, if the use of open-source tools is not a project constraint.

2.6 Chapter Summary

To provide the reader with the necessary background, this chapter begins with an overview of the serverless computing model. After elucidating foundational concepts, e.g., the difference between application and infrastructure code, and introducing essential terminology, e.g., event filters and extension plugins, §2.1 describes advantages and disadvantages of the paradigm. To complete this overview with security-focused information, §2.2 illustrates the most critical risks for serverless applications, recently identified by industry experts, and IAM-related concepts. As emphasized in §1.1, the complexity of the latter is one of the main challenges of this work (C2).

The chapter also presents a selection of existing code analysis frameworks focused on event-driven architectures and detection of data flows (§2.4). Importantly, such frameworks target different programming models, e.g., the Android OS, and therefore fail to capture the characteristics of the FaaS paradigm. Specifically, none of them is designed to process infrastructure code (§2.3), one of the key features of `CloudFlow` (§5.2.2).

Finally, since the implementation of the framework proposed in this thesis is focused on Python, the chapter ends with a section dedicated to the language type system and Pysa, the Python-specific static analyser leveraged by `CloudFlow` (§2.5). The first part, which illustrates the concept of type annotation, is important to understand the language-specific aspects of the program representation explained in §5.2.4. By contrast, the second part, which includes the Pysa workflow, clarifies how `CloudFlow` makes use of static analysis models to identify security-sensitive data flows (§5.2.5).

Chapter 3

Characterization of the AWSomePy Dataset

In this chapter, we present the characterization of a collection of serverless applications developed in Python for the AWS platform. The objective of this study is to identify key trends that can support our static analysis-focused research.

3.1 Introduction

The serverless computing paradigm aims at reducing the overhead associated with deployment and monitoring of traditional servers by leveraging a stateless and event-driven model, which relies on platform services offered by a cloud provider [17]. By abstracting away almost all operational concerns, including infrastructure scalability and IT hardware maintenance, enterprises can cut their costs and focus on developing their software products. However, relying on serverless environments, such as AWS [6], Azure [89] and GCP [61], presents new challenges, especially in the areas of performance [157], traceability [39] and security [82, 71, 34, 108, 100].

Both static and dynamic analysis of serverless applications are problematic. Unlike traditional applications, serverless applications routinely receive their inputs from a variety of sources and code execution can be triggered by different kinds of events, such as a database update or a file upload [17]. Recent academic studies have therefore introduced new frameworks for information flow analysis [4, 38, 145, 131, 69] and tracing of potentially malicious events [39]. While demonstrating the correctness in principle of the proposed approaches, the experimental evaluations of such frameworks rely on a very limited set of

applications. Thus, they are not optimized to consider architectures and cloud services most frequently used in real-world applications.

Static analysis, in particular, is very challenging in the context of serverless computing. In addition to the high number of events, analysing the code that implements platform services is not possible. Obetz *et al.* [96], who extended the concept of call graphs to serverless applications, show that static analysis is undoubtedly useful, but inevitably has to rely on models and approximations. Similarly to the case of information flow and traceability frameworks discussed above, such models and approximations should ideally be based on key trends and features extracted from a large collection of applications. Unfortunately, existing general-purpose datasets, such as PyTraceBugs [3] and BugsInPy [167], do not take into account the specific characteristics of the serverless paradigm, as they were primarily conceived to support static source code analysis and unit testing for traditional Python applications.

In this chapter, to guide the development of models and tools for serverless computing, we analyse a dataset of 145 applications obtained from GitHub by adopting and customizing the Wonderless dataset methodology [45]. Given the growing popularity of Python in the serverless domain, we focus our attention on applications implemented in Python for AWS, which is the most widely used development platform [37]. Our results show that permissions are rarely configured on a per-handler basis, despite this being considered a good practice to secure serverless applications. Furthermore, while data storage and NoSQL services are, as expected, the most frequently used, developers rely on configuration and management-oriented services as well. These provide high flexibility, but they also trigger workflows that are difficult to inspect prior to deployment.

In summary, we make the following contributions:

- We publicly release `AWSomePy`[1], a new dataset of AWS serverless applications implemented in Python and compatible with the Serverless Framework deployment tool.
- We provide a characterization of our dataset, which we obtain by conducting an architectural and an application code-level analysis.

This chapter is structured as follows. We begin by illustrating how the dataset was generated (§3.2). We next present the methodology and the results of our analysis (§3.3). These are further discussed (§3.4) before comparing our work with previous research (§3.5). We finally provide a summary of this chapter (§3.6).

[1] https://doi.org/10.5281/zenodo.7838076

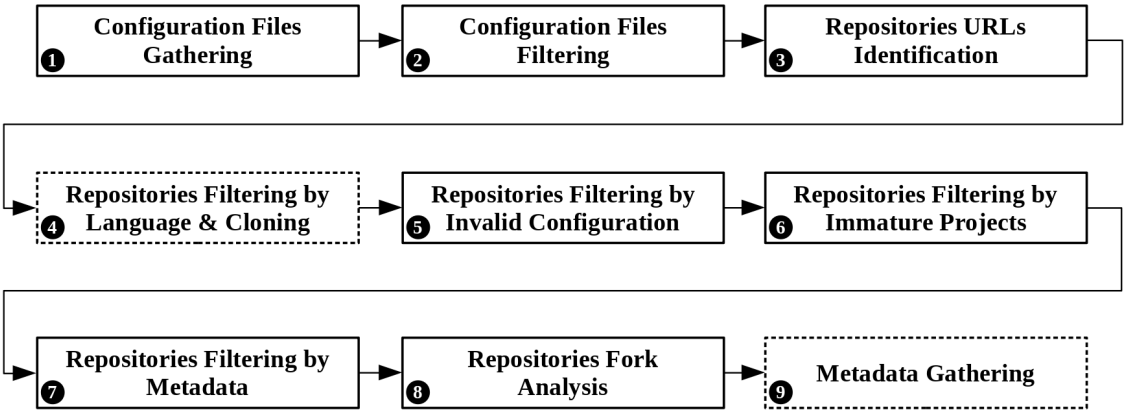

Fig. 3.1 Dataset generation process. The dashed lines indicate the customized steps compared to the Wonderless dataset.

3.2 Dataset Generation

This section describes how the dataset used for our analysis (§3.3) was generated. Starting from the consolidated methodology of the Wonderless dataset [45], which collected GitHub applications compatible with the Serverless Framework deployment tool [141], we generated a new, Python-only version of the dataset in August 2022 by customizing the code provided by its authors [93].

While including a large collection of mature and well-maintained applications, the Wonderless methodology suffers from the limitation that it does not collect repository-specific or application-level metadata. This information can help researchers to identify suitable benchmarks and to assess the complexity of the applications in the dataset, as we show in this study (§3.3.1). For this reason, our methodology includes a further step that gathers a set of metadata, which include, among others, the number of stars, events and handlers. Moreover, Wonderless was created in July 2020. Considering the constant evolution of serverless offerings and the latest features of the widely adopted Serverless Framework[2], which facilitates the deployment of serverless applications through declarative infrastructure code, we decided to base our investigation on the most up-to-date dataset possible.

The remainder of this section is dedicated to detailing the nine steps of the dataset generation process, which is outlined in Fig. 3.1.

Identification of configuration files. The processing pipeline developed by Eskandani *et al.* [45] starts by querying GitHub to identify all the repositories containing at least

[2]According to Datadog, in 2021 the Serverless Framework was adopted by 90% of the surveyed organizations using AWS [36].

one `serverless.yml` file, which is used to configure applications to be deployed with the selected framework. To understand how the analysis presented in §3.3 was conducted, it is important to note that this file contains several parameters. As illustrated in Listing 3.1, these include extension plugins, e.g., `serverless-python-requirements` (line 4), names of cloud resources, e.g., `aws-python-project-test-bucket` (line 8), and permissions, e.g., `s3:ListBucket` (line 18). Furthermore, the tag `functions` of the YAML file identifies a section dedicated to the event-triggered handlers of the application, e.g., `onHTTPPostEvent` (line 24).

```
1  service: aws-python-project
2
3  plugins:
4    - serverless-python-requirements
5
6  custom:
7    settings:
8      BUCKET_NAME: aws-python-project-test-bucket
9
10 provider:
11   name: aws
12   profile: username
13   region: eu-central-1
14   environment: ${self:custom.settings}
15   iamRoleStatements:
16     - Effect: "Allow"
17       Action:
18         - "s3:ListBucket"
19         - "s3:GetObject"
20         - "s3:PutObject"
21       Resource: "arn:aws:s3:::${self:custom.settings.BUCKET_NAME}/*"
22
23 functions:
24   onHTTPPostEvent:
25     handler: handlers/httphandler.onHTTPPostEvent
26     events:
27       - httpApi:
28           method: post
29           path: /produce
30     environment:
31       BUCKET_NAME: ${self:custom.settings.BUCKET_NAME}
```

Listing 3.1 Example of `serverless.yml` file required by the Serverless Framework.

Table 3.1 Summary of the dataset generation process.

Step	YAML Files	Repositories	Dataset Size
1	9,096	✗	✗
2	7,912	✗	✗
3	✗	7,074	✗
4	✗	811	8.7 GB
5	✗	783	8.5 GB
6	✗	159	1.6 GB
7	✗	147	1.6 GB
8	✗	147	1.6 GB
9	✗	145	1.6 GB

As shown in Table 3.1, which summarizes the output of each processing step in terms of identified files or repositories and dataset size, 9,096 configuration files are detected in step ❶. These are filtered in step ❷ to remove files developed by the Serverless Framework community as well as those included in folders, e.g., *demo* and *test*, indicating that the application is either a template or a toy example. The final number of identified configuration files is 7,912. We emphasize that no repository was cloned during the execution of the first two processing steps.

Identification of repositories. Even though it is reasonable to assume that a vast majority of serverless applications are deployed with only one YAML file, the framework allows for the usage of multiple configuration files. This implies that it is not possible to uniquely identify the repositories URLs by exclusively relying on the gathered `serverless.yml` files. The goal of step ❸ is therefore to obtain such URLs by removing duplicate entries, which enables us to shortlist 7,074 repositories. However, since the focus of this research is the analysis of applications implemented in Python, in step ❹ we filter the repositories by primary language prior to cloning. This was achieved by using a GitHub API that returns several pieces of information about a target repository [53]. The outcome of the latter step consists of 811 repositories with a total dataset size of 8.7 GB.

The removal of invalid YAML files is the objective of step ❺. As observed by Eskandani *et al.*, these include *syntactically* and *semantically* invalid files, with the latter category comprising files that do not specify either the cloud provider or the application functions. This round of filtering causes only a modest reduction in the number of repositories and in the size of the dataset, which decrease to 783 and 8.5 GB, respectively. Congruently with the Wonderless dataset, such quantities are much more significantly reduced in step ❻, which aims at filtering out immature projects, i.e., active for less than one year, and identifies 159

repositories with a dataset size of 1.6 GB.

In order to remove non-real-world applications, the inherited pipeline includes an additional round of filtering, namely step ❼, which is focused on the analysis of repository metadata, such as labels, topics and descriptions. They are compared with a set of keywords, e.g., *demo*, *test* and *example*. In our case, this processing step filters out only 12 repositories, thus bringing the total to 147 without any substantial variation of the dataset size. As for step ❽, Eskandani *et al.* explain that they analysed a list of potential forks, i.e., repositories that feature the same name, but different developers, to identify those to be filtered out. The rationale behind this is that, in all probability, an application developed by forking another shares large portions of the original source code. It is noteworthy that the Wonderless dataset code only generates a list of *candidate* forks, which have to be manually analysed. However, in our case, no such candidates are shortlisted, which implies that the total number of repositories remains unchanged.

Metadata gathering. To support the characterization of the obtained applications, in step ❾ we further customize the dataset generation process with code that extracts repository-specific metadata, such as number of stars, watchers and forks, and processes both the YAML files and the source code to gather information about cloud provider, Serverless Framework version, lines of code, number of events and number of handlers. This analysis step enables us to verify that only 2 out of 147 are non-AWS applications, which we remove. As a result, 145 repositories are included in `AWSomePy`, which we release along with the associated metadata.

3.3 Dataset Analysis

In this section, we detail the methodology of our analysis and present the obtained results. We first consider configuration and application architectural parameters (§3.3.1), such as deployment tool plugins used and the number of handlers and events. Second, we analyse the application code (§3.3.2) to identify the most common cloud platform services and APIs.

3.3.1 Configuration and Architectural Analysis

Plugin analysis. The Serverless Framework supports a large number of plugins that facilitate the integration of complex features. To gain insight into the plugins used when deploying `AWSomePy` applications, we parse the `serverless.yml` files and extract the relevant information from a dedicated tag. For simplicity, when a repository contains more than one YAML

Table 3.2 Top eight plugins in AWSomePy.

Plugins	Occurrences
serverless-python-requirements	95
serverless-pseudo-parameters	25
serverless-domain-manager	15
serverless-step-functions	14
serverless-offline	9
serverless-dotenv-plugin	8
serverless-prune-plugin	8
serverless-iam-roles-per-function	7

configuration file, we consider only one of them, as it is plausible that all have similar features and complexity. The processed file is always the first identified by recursively visiting the repository with the Python standard library function `os.walk`.

A total of 44 plugins were identified, and the top eight most frequently occurring are shown in Table 3.2. The most frequent plugin by far is `serverless-python-requirements`, which is designed to assist with dependency management. The second most frequently used plugin, i.e., `serverless-pseudo-parameters`, is also configuration-oriented, as it allows using AWS CloudFormation syntax to specify configuration parameters. Interestingly though, the plugin in question is now deprecated [52], as its functionality is natively supported by the most recent releases of the Serverless Framework. This result is consistent with the approach adopted to generate the dataset (§3.2), which explicitly prioritises mature applications.

Unlike the first and the second, the third and the fourth most frequently occurring plugins, i.e., `serverless-domain-manager` and `serverless-step-functions` are functionality-oriented. The former enables creating custom domain names by leveraging specialized AWS services, whereas the latter facilitates the deployment of step functions.

As for the remaining four plugins, `serverless-offline`, which supports local testing of serverless applications, is present in only 9 YAML files. Finally, with 7 occurrences, the least frequently detected plugin in Table 3.2 is `serverless-iam-roles-per-function`. Crucially, this implies that developers are not adopting the best practice of configuring permissions in a granular fashion. Without this plugin, which defines *per-function* IAM roles, every function in the application is deployed with the same *global* IAM role, thus increasing the chances of it being over-privileged.

Complexity analysis. In order to understand architectural characteristics of the applications in our dataset, we assess their complexity by considering the number of lines of code (LOC)[3], events and handlers. The average LOC is 4,468, while the minimum and the maximum are 26 and 132,658, respectively. Furthermore, the cumulative distribution of the LOC in Fig. 3.2 indicates that 55% of the `AWSomePy` repositories have less than 1 kLOC, though those with under 100 LOC constitute less than 10% of the dataset.

An assessment based on LOC alone, despite its importance, does not capture key complexity indicators of serverless applications. For this reason, we process each `serverless.yml` file to obtain information about the application's events and handlers. Even though both types of information can be specified in a YAML file in multiple ways, to facilitate the implementation of an automated parser, we limit our analysis to the tag `functions`. Considering the plugins present in `AWSomePy` (Table 3.2) and the application code analysis detailed in §3.3.2, we believe that this does not significantly affect the accuracy of our results. As elucidated later (§3.3.2), the AWS services, e.g., `stepfunctions`, that would require specifying events and handlers in other YAML tags are not widely adopted in `AWSomePy`.

The correlation between number of handlers and LOC is visible in Fig. 3.3, where the darkest areas indicate a higher point density. The figure shows that the `AWSomePy` applications typically include at most four handlers. We note that as a consequence of the aforementioned limitation of our parser a few points in Fig. 3.3 have no handlers.

Finally, the number of events versus number of handlers diagram in Fig. 3.4 reveals that the majority of the `AWSomePy` applications comprise less than five events. However, similarly to Fig. 3.3, due to our simplified parsing approach, some applications are classified as having no events, which implies that their architecture should be further inspected.

3.3.2 Cloud Service and API Usage

Cloud services. To identify the provider-managed services most frequently used in our dataset, we look for lines of code that instantiate a client or resource object with the open-source library `boto3` [25]. The latter is an essential component of the analysed applications as it provides a rich interface to a large number of AWS services. The targeted lines of code, e.g., `boto3.client('s3')` and `boto3.resource('s3')`, can be processed via regular

[3]We compute the LOC with the command-line tool Pygount [110].

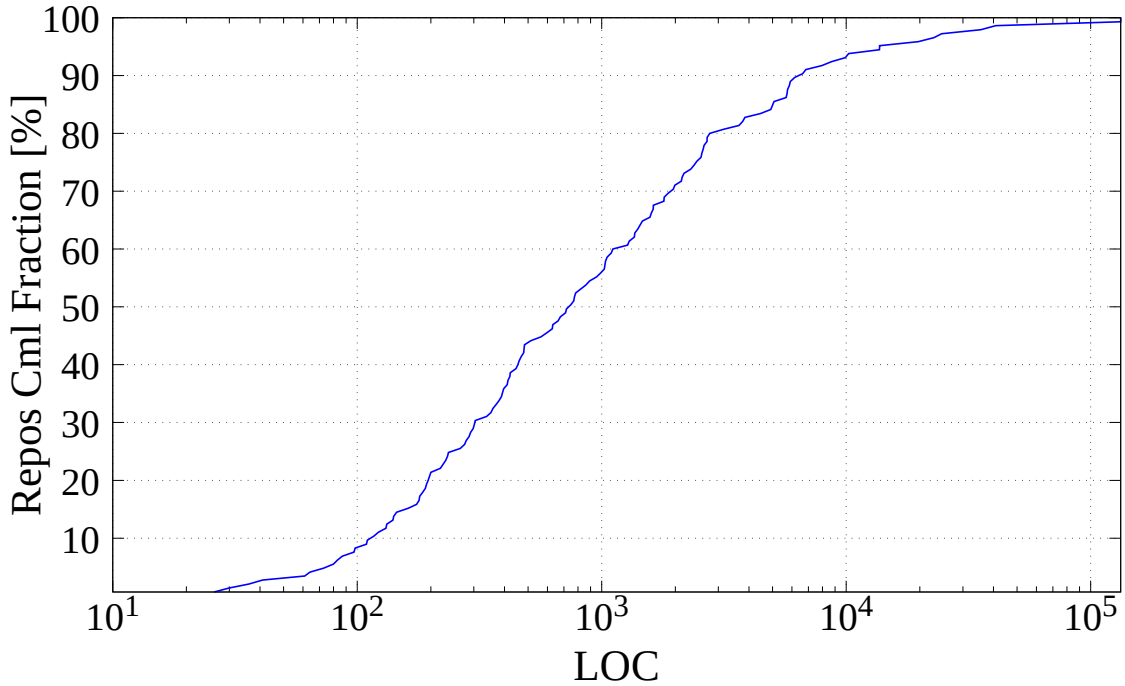

Fig. 3.2 Cumulative distribution of the lines of code in `AWSomePy` (cumulative fraction of repositories on the y axis).

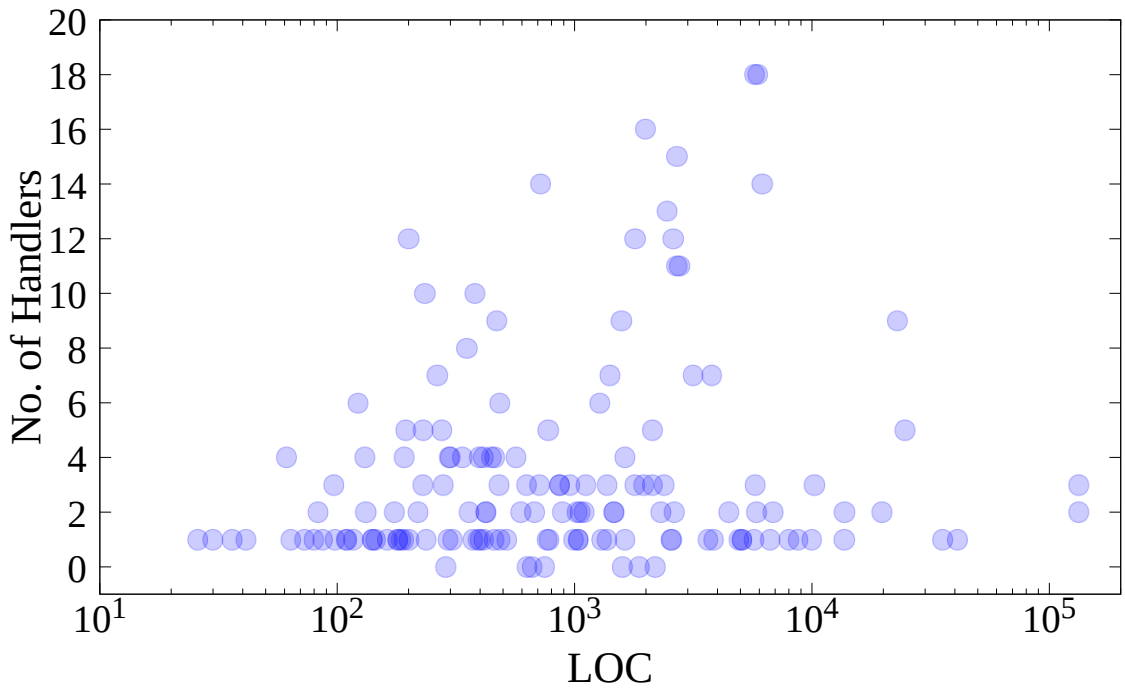

Fig. 3.3 Number of handlers vs lines of code in `AWSomePy`.

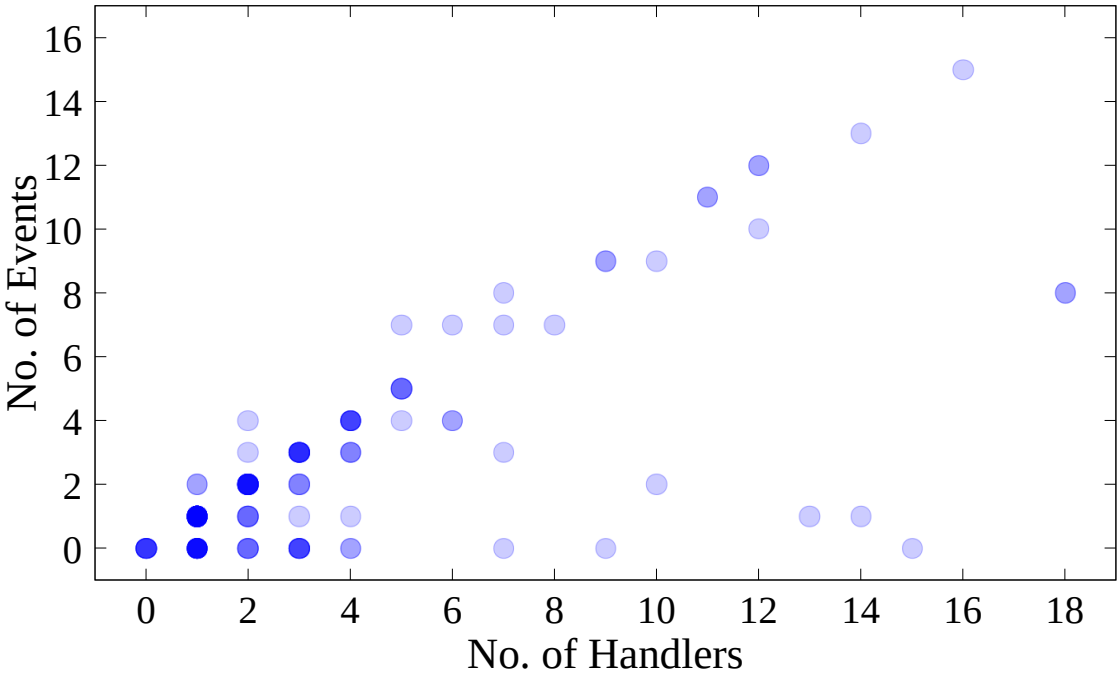

Fig. 3.4 Number of events vs number of handlers in `AWSomePy`.

expressions because, despite the differences between client and resource objects[4], the class constructors always require specifying the cloud service as a string.

Our analysis identifies 46 services in total, with those with the highest number of occurrences, computed in terms of `boto3` client and resource objects instantiations, shown in Table 3.3. Data storage and NoSQL services, i.e., `s3` and `dynamodb`, are significantly more common than the others, with 217 and 201 instantiations detected, respectively. In the case of `s3`, these were found in 59 repositories, whereas they were present in 47 repositories in the case of `dynamodb`.

The third most frequently used service is `lambda`, which allows configuring the serverless platform itself. The fourth is `ssm`, which is used to perform a variety of management tasks. This indicates that despite the wealth of configuration features and the rich plugin ecosystem offered by the Serverless Framework, developers access management-oriented services via their application code.

Cloud APIs. We next provide additional details on how the top five services in our dataset are used by conducting a cloud API-focused analysis. We parse the `boto3` library documen-

[4]While it is possible to instantiate a client object for all the AWS services supported by the `boto3` library, only some of them can be accessed through a resource object. Additional differences are mentioned in the remainder of this section.

Table 3.3 Top eleven AWS services in `AWSomePy`. The column Occurrences reports the number of `boto3` client and resource objects instantiations within the relevant repositories.

Services	No. of Repositories	Occurrences
`s3`	59	217
`dynamodb`	47	201
`lambda`	24	47
`ssm`	14	46
`sqs`	21	41
`sns`	11	30
`ec2`	12	29
`sts`	9	26
`rekognition`	8	15
`cloudformation`	7	14
`stepfunctions`	9	14

tation to extract in a semi-automated manner the APIs exposed by the clients of the AWS services present in our dataset[5]. Our preliminary analysis of the documentation reveals that the sections dedicated to the supported services share the same structure, but there exist some differences. Consequently, implementing a parser capable of dealing with all the services would be time-consuming. We therefore use a simplified documentation parser[6] and subsequently manually validate the obtained results. Unlike an *on-the-fly* processing, i.e., conducted at the same time as the actual analysis of the application code, our pre-analysis allows storing the parser results, which facilitates their manual validation and management in a version control system.

The API-related information is then used to parse the dataset application code in order to identify relevant lines. For simplicity, we focus our attention on the top five services (Table 3.3) and, after filtering out the lines of code including the widely adopted API names `close` and `copy`, we manually check the remaining lines to ascertain that they are legitimate `boto3` API calls for one of the services of interest. We recognize that our approach requires manual validation of the API names extracted from the `boto3` documentation and of the lines of code where such APIs are detected, but we believe that this constitutes a good compromise between accuracy and ease of implementation. We leave to future work the development of a more automated framework.

[5]Note that we extract only the APIs supported by the `boto3` client objects. These implement low-level interfaces, which implies that they offer a richer set of APIs in comparison with the resource objects [126].

[6]Our `boto3` documentation parser is based on the Beautiful Soup library [81].

Our results are shown in Table 3.4, which reports the occurrences of the detected `s3`, `dynamodb` and `lambda` APIs, and in Table 3.5, which includes the same information for the services `ssm` and `sqs`. Interestingly, we observe a similarity between `s3` and `dynamodb`, because their most frequently used APIs, i.e., `put_object` and `put_item`, allow storing information in cloud-based resources. This confirms the importance that these have, given the stateless and ephemeral nature of serverless functions. Moreover, we note that developers rather often create `s3` buckets and `dynamodb` tables *programmatically* via the `create_bucket` and `create_table` APIs. We believe that this trend poses a security challenge, because the configuration of these resources cannot be straightforwardly inspected by analysing the `serverless.yml` file of the application.

As for the `lambda` service, our results highlight that the API with the highest number of occurrences is `invoke`, which supports both asynchronous and synchronous execution of serverless functions. Contrary to our expectations, this implies that some applications do not rely on the facilities provided by the AWS platform for automatic execution of their handlers. While understanding the reasons behind this design choice would require a more in-depth analysis, the manual inspection of ten `invoke` API calls in six applications indicates that, in eight cases, the API is used to facilitate the parameterization of a handler name, which is helpful when there are multiple versions of an application, e.g., production and development. By contrast, the remaining two cases show test scripts that rely on the `invoke` API to artificially trigger the execution of a handler passed as a parameter. Even though a specific permission is required to execute the API in question [24], we emphasize that its usage, unless necessary for testing purposes, affects the application workflows in a way not easily detectable via static analysis, thus potentially leading to security vulnerabilities.

Finally, it is noteworthy that developers rarely use the `lambda` API `add_permission`, which we detect only seven times. In addition, as far as the `ssm` service is concerned, we observe that three out of the top four APIs[7] allow retrieving information from a provider-managed parameter store. Both these trends are example of security-oriented design patterns present in our dataset.

3.4 Discussion

The purpose of this section is to further discuss the obtained results and the limitations of our approach.

[7]Namely, `get_parameter`, `get_parameters` and `get_parameters_by_path`.

Table 3.4 Occurrences of the six most widely used APIs for the top three AWS services in AWSomePy. The occurrences of all the other detected APIs are aggregated in the entry *other*.

s3 API	#	**dynamodb** API	#	**lambda** API	#
put_object	61	put_item	143	invoke	55
get_object	52	scan	64	add_permission	7
create_bucket	50	query	62	list_functions	3
upload_file	48	get_item	58	get_policy	3
download_file	24	update_item	57	get_function	2
list_objects_v2	22	create_table	41	list_tags	2
other	111	*other*	93	*other*	4

Table 3.5 Occurrences of the six most widely used APIs for fourth (ssm) and the fifth (sqs) most frequently used AWS services in AWSomePy. The occurrences of all the other detected APIs are aggregated in the entry *other*. The ssm APIs get_parameters_by_path and describe_instance_information are abbreviated as get_parameters_by_p and describe_instance_i, respectively.

ssm API	#	**sqs** API	#
get_parameter	79	send_message	27
put_parameter	18	get_queue_url	16
get_parameters	7	delete_message	15
get_parameters_by_p	3	create_queue	14
list_commands	2	receive_message	13
describe_instance_i	1	send_message_batch	2
other	6	*other*	1

Application code analysis. We observe that the boto3 client and resource object instantiations reported in Table 3.3 are always higher than the respective number of AWSomePy projects. Thus, on average, the analysed applications include multiple instantiations of these objects for the same service. This affects the overall amount of data flows, and we therefore recommend considering the number of such instantiations for the development of serverless-specific complexity metrics.

Security implications. Our architectural analysis shows that the security-focused plugin serverless-iam-roles-per-function is used in only 7 applications (Table 3.2). Considering the functionality that it offers (§3.3.1), we believe that this is one of the most interesting results of this study. Although over-privileged function permissions and roles are among

the most critical risks for serverless applications recently identified by PureSec [108] and the Cloud Security Alliance [34], the vast majority of the `AWSomePy` applications rely on application-wide IAM roles and do not follow the principle of *least privilege*. While this does not imply that all these applications are vulnerable, since the most common services in our dataset are `s3` and `dynamodb`, it could lead, similarly to traditional SQL injection attacks, to loss or unauthorised disclosure of information. In addition, given that serverless applications frequently use such services to store data between different executions, this mechanism could be compromised as well.

Limitations. As mentioned in §3.1, static analysis of serverless applications is challenging due to the variety of events that can trigger the execution of their handlers. It might therefore appear contradictory that the majority of the applications in our dataset comprise less than five events (Fig. 3.4). To better understand this result, it should be observed that we extract events-related information only from the YAML tag `functions`. This approach has two implications. First, we do not consider that some plugins require specifying events in other parts of the `serverless.yml` file. While one of these plugins, i.e., `serverless-step-functions`, is among the most frequent in `AWSomePy`, it is used in only 14 out of 145 repositories (Table 3.2). The second implication is that we do not count events *implicitly* defined by some services, e.g., `dynamodb`, as we focus our attention exclusively on events *explicitly* listed by the developer. However, implicit events do not necessarily trigger the execution of a handler, as this depends on how the latter is configured. Although in the two discussed cases our analysis yields an underestimation of the total number of events, we believe that our results provide a valid starting point for future research.

3.5 Related Work

Serverless Datasets. Eskandani *et al.* extend their original work on the Wonderless dataset with an in-depth analysis of its applications [46]. The authors highlight that, despite the advantages of the serverless model, its adoption by the open-source community is affected by a number of challenges. As reported in previous studies [80], the limitations of the tools available for testing and debugging, which are particularly relevant to this thesis, play an important role. Eskandani *et al.* state that such limitations are a possible explanation for the relatively simple architectures of the analysed applications. Less than 3% of them, in fact, rely on AWS SNS and AWS SQS, two key cloud services for the implementation of event-driven architectures. A similar trend, as shown in Table 3.3, can be observed in `AWSomePy`. In addition, the authors of the Wonderless dataset observe that less than 5% of

its repositories feature more than 100 GitHub stars, and nearly half of them do not have any star. As pointed out by Borges *et al.* [23], developers consider the number of such stars when deciding whether to use a GitHub repository as a reference. Consequently, these statistics suggest that reusing code in multiple serverless applications is still challenging. Despite their insights into the open-source serverless ecosystem, Eskandani *et al.* do not characterize the Wonderless dataset applications in terms of used APIs, a key part of the work presented in this chapter.

The OS3 search engine by Bhatnagar *et al.* [21] provides a novel approach to identifying serverless functions in open-source repositories. Its authors observe that reusable FaaS components can be found only if descriptive keywords are combined with information about the imported libraries and the software licence. The latter aspect is the element of novelty of OS3, as identifying serverless functions that cannot be deployed due to licence-related issues makes the search process inefficient. Importantly, the authors rely on a publicly available dataset[8] that comprises 5,981 serverless applications, of which 5,220 make use of the Serverless Framework. As a result, the OS3 dataset is significantly larger than `AWSomePy`. There are two reasons that justify this difference. First, the OS3 dataset includes applications compatible with four serverless platforms, whilst `AWSomePy` is AWS-focused. Second, the criteria used to generate the OS3 dataset are different from those adopted to create `AWSomePy`. Bhatnagar *et al.*, in fact, state that the Wonderless dataset methodology, on which we based the filters used for `AWSomePy` (§3.2), is excessively strict, as it leads to filtering out potentially useful repositories. We underscore that the aim of the filters implemented to create `AWSomePy` is to maximize the number of real-world applications. By contrast, the objective of the OS3 authors is to obtain a wider set of relevant functions that developers can promptly include in their applications. In summary, the two datasets were created to meet different requirements, hence the mismatch between their sizes.

Python Datasets for Research. Published in 2020, BugsInPy [167] is a database of 493 Python bugs identified in 17 popular projects available on GitHub. Motivated by the need to advance research on real-world Python applications, the authors propose a framework that reproduces bugs identified via the version control system history of the selected repositories. It is noteworthy that BugsInPy users do not have to learn the specifics of the Python projects, e.g., their build systems and the adopted testing tools. In addition, the framework allows retrieving automatically the relevant software version when reproducing the bugs. The BugsInPy database, however, does not contain any serverless application. Therefore, it cannot be considered an alternative to `AWSomePy`.

[8] https://github.com/edgeumd/serverless_dataset

Akimova *et al.* partially reuse the BugsInPy methodology to create PyTraceBugs [3]. The primary objective of the authors is to obtain a large dataset suitable for training and evaluation of machine learning models used for static analysis. Differently from BugsInPy, PyTraceBugs consists of snippets, i.e., implementations of functions or methods, and all the included bugs manifest themselves by raising an exception. Thus, PyTraceBugs is an important reference for low-level debugging systems, but unsuitable to conduct research on serverless applications. To further confirm this conclusion, we observe that the dataset includes a number of correct code snippets retrieved from repositories[9] dedicated to tools for cloud computing. These, however, are not serverless applications, and, therefore, not relevant to the research presented in this thesis.

3.6 Chapter Summary

In this chapter, we present `AWSomePy`, a novel dataset of 145 AWS serverless applications developed in Python and compatible with the Serverless Framework. We analyse their architecture by considering serverless-specific parameters, such as plugins, events and handlers, along with the LOC. We also conduct an application code-level analysis that aims to identify the most frequently used cloud services and APIs.

Our results highlight that developers tend to use plugins to facilitate the configuration of their applications and the deployment of complex pieces of functionality. Crucially, the security plugin `serverless-iam-roles-per-function` can be found only in 7 `AWSomePy` applications, in spite of the fact that 55% of them feature a number of LOC between 26 and 1,000. As for our application code-level analysis, it confirms that data storage and NoSQL services are by far the most commonly used, followed by, interestingly, configuration and management-oriented services. While these provide a high degree of flexibility, they also add workflows that are difficult to inspect prior to deployment. Moreover, the identified APIs indicate that developers make use of programmatic creation of data stores and database-like resources, which has similar security-related implications.

In conclusion, the analysis of the `AWSomePy` dataset shows that the granular configuration of handler permissions is not widely adopted, and that analysing serverless applications statically to detect misconfigurations and security-sensitive data flows is challenging for real-world serverless applications.

[9] `Azure/azure-cli-extensions`, `Azure/azure-sdk-for-python`, and `oracle/oci-python-sdk`

Chapter 4

CloudBench: A Novel Suite of Security-oriented Microbenchmarks

In this chapter, we introduce a novel suite of security-oriented microbenchmarks. Their design is based on documented serverless-specific vulnerabilities as well as the previously provided characterization of our dataset.

4.1 Introduction

The recent advent of serverless applications has created a need for static analysis tools to analyse them. To address the challenges previously introduced (§1.1), such tools are inevitably complex and therefore require suitable evaluation approaches, such as benchmark-based evaluations. As explained while introducing RQ2, benchmarks constitute standardized references, which allow assessing the effectiveness of novel frameworks and comparing them with others. Existing serverless benchmarks, however, are focused on performance evaluation [164, 87] and cost effectiveness [16], and cannot therefore be used to evaluate security tools, such as `CloudFlow` (Chapter 5). Its evaluation, in fact, necessitates a collection of vulnerable serverless applications, an example of which is provided in §2.2.2.

Furthermore, previous research [7, 146] shows that security tools are effectively evaluated with *microbenchmarks*. Differently from benchmarks, a term widely adopted in numerous areas of research, microbenchmarks are applications *limited* in size and functionality that contain reference test cases. While conceptually similar to traditional unit tests, microbenchmarks provide reusable, i.e., non project-specific, tests that enable validation and comparison of security tools.

To address the above issues, we propose `CloudBench`, a novel microbenchmark suite that contains a set of tests for security tools and is, as such, *security-oriented*. Specifically, the objective of each test is to verify that a given security-sensitive data flow can be detected by the framework being evaluated. It is important to highlight that the `CloudBench` microbenchmarks are based on documented cases of code [100, 71] and database injection [1]. The former enables the execution of system commands in the infrastructure hosting the application, whilst the latter leads to unintended disclosure of information from a cloud-based database. Although these are not serverless-specific vulnerabilities, they are among the most critical risks for serverless applications recently identified by the Cloud Security Alliance [34]. The analysis of the aforementioned cases, therefore, was crucial to understanding possible real-world scenarios relevant to such applications.

To better model the ways in which these are (mis-)programmed, we also rely on the characterization of `AWSomePy` presented in Chapter 3. This allows us to focus on the cloud services and APIs with the highest number of occurrences (§3.3.2), which are then included in our microbenchmarks.

This chapter is structured as follows[1]. To illustrate the serverless-specific challenges of static analysis for security, we begin by presenting two motivating examples (§4.2), which are also two of our microbenchmarks and further expand some of the concepts first introduced in §2.2.2. We next present a characterization of our suite focused on data flow types, included vulnerabilities, LOC, and supported cloud services (§4.3). After comparing our work with existing research contributions (§4.4), we finally provide a summary of the chapter (§4.5).

4.2 Motivating Examples

In this section, we present two applications that constitute our motivating examples. They illustrate in more practical terms the challenges of analysing serverless application statically and, consequently, of validating security tools for serverless. Furthermore, they provide concrete examples of developed microbenchmarks. Specifically, the first application, described in §4.2.1, contains a security-sensitive data flow leading to a code injection vulnerability. In contrast, the second, detailed in §4.2.2, includes an example of inter-procedural data flow that can lead to information leakage.

[1]We remind the reader that additional information on the characterization of the `CloudBench` suite is available in Appendix A.

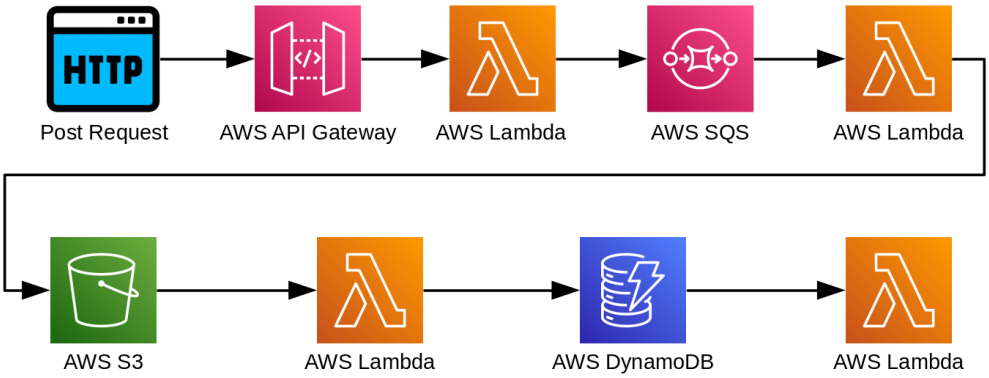

Fig. 4.1 Example application with an event-driven data flow. The four event-triggered handlers are identified by the blocks AWS Lambda.

4.2.1 Code Injection Example

As shown in the architectural diagram of Fig. 4.1, the functionality offered by the first example relies on multiple cloud services, along with four event-triggered handlers[2]. A YAML file specifies the required permissions, such as those visible in Listing 4.1, as well as events and handler-related configuration parameters.

Upon receiving a HTTP Post request from a user, a handler that sends a message to a queue is triggered (Listing 4.2). As a consequence, the SQS service executes the consumer handler `onSQSMessage` (Listing 4.3). The handler processes the last received message, which contains user-specified data, and uploads a local file to a cloud-based repository by calling the `upload_file` API. The upload, in turn, triggers the execution of the `onS3Upload` handler (Listing 4.4), which reads the newly uploaded file and then updates a database by calling the auxiliary function `updateDynamoDBTable`. On detecting this, the DynamoDB service triggers the `onDynamoDBStream` handler (Listing 4.5), which passes a string initialized from the newly added database item to `os.system`. As a result, the application executes a system command with user-controlled input.

```
iamRoleStatements:
    - Effect: "Allow"
      Action:
        - "s3:ListBucket"
        - "s3:GetObject"
        - "s3:PutObject"
```

Listing 4.1 Application permissions specified in the configuration file.

[2]The widely adopted Serverless Framework [141] is the deployment tool of choice for the application. However, the concepts discussed are generic, as they are not platform or deployment technology-specific.

```
def onHTTPPostEvent(event, context):
    # After initialization statements
    try:
        sqs = boto3.resource('sqs')
        queue = sqs.Queue(os.getenv('QUEUE_URL'))
        queue.send_message(MessageBody=event['body'])
        message = 'Message accepted!'
    except Exception as e:
        message = str(e)
        statusCode = 500
    return {'statusCode': statusCode, 'body': json.dumps({'message':
    message})}
```

Listing 4.2 Handler triggered by a HTTP request.

```
def onSQSMessage(event, context):
    # After initialization statements
    uploadFileFolder = 'uploads'
    uploadFileName = msgBodyDict + '.txt'
    s3BucketKey = os.path.join(uploadFileFolder, uploadFileName)
    localFile = os.path.join('/tmp', uploadFileName)
    s3_client = boto3.client('s3')
    s3_client.upload_file(localFile, os.environ['BUCKET_NAME'],
   s3BucketKey)
    return
```

Listing 4.3 Handler triggered by a SQS message.

```
def onS3Upload(event, context):
    # After initialization statements
    s3 = boto3.resource('s3')
    downloadPath = os.path.join('/tmp', os.path.split(fileName)[-1])
    s3.Object(bucketName, fileName).download_file(downloadPath)
    with open(downloadPath, 'r') as inputFileObj:
        inputFileContentsList = inputFileObj.readlines()
    updateDynamoDBTable(inputFileContentsList[0],
   inputFileContentsList[1])
    return
```

Listing 4.4 Handler triggered by the S3 API `upload_file`.

```
def onDynamoDBStream(event, context):
    authorsInfo = event['Records'][0]['dynamodb']['NewImage']['authors']['S']
    titleInfo = event['Records'][0]['dynamodb']['NewImage']['title']['S']
    eventData = authorsInfo + titleInfo
    os.system('echo %s' % eventData)
    return
```

Listing 4.5 Handler triggered by a DynamoDB database update.

This example, which is included in our paper P3[3] and is one of the `CloudBench` microbenchmarks[4], allows us to further explain the three key challenges for static detection of security-sensitive data flows, first introduced in §1.1, as follows:

- C1: Considering the effect of events raised by cloud API calls is crucial. However, general-purpose tools do not support this since they analyse only the source code, which does not contain any event-related information, e.g., the fact that the API call `send_message` in Listing 4.2 triggers the execution of the handler `onSQSMessage`. Such information is present exclusively in the infrastructure file, which must therefore be suitably processed prior to the execution of the analysis. Furthermore, events are propagated via objects, i.e., the handlers' `event` inputs, that encapsulate complex, service-dependent data structures. To incorporate these into the analysis, specific models are required. Their absence prevents a general-purpose tool from accomplishing this task.

- C2: The second challenge is the multitude of mechanisms available to define the application permissions. For example, the AWS environment features identity and resource policies, along with time-constrained roles and attribute-based access control [144, Ch. 2]. Supporting such mechanisms is crucial for providing accurate results.

- C3: The third key challenge is that the developer cannot include in their analysis the proprietary code that implements the cloud-based services. Since these are instrumental in the functionality provided by a serverless application, models are necessary in this case as well.

[3]Differently from the example illustrated in §2.2.2, included in our paper P2, the application described in this section features four cloud services instead of three, and four event-triggered handlers instead of three. However, the illustrated concept is identical, as a system command containing user-controlled input is executed in both cases.

[4]The name of this microbenchmark is `api-send-message-queue-assign`. Note that all the microbenchmarks' names include the `boto3` API [25] that is part of the expected data flow.

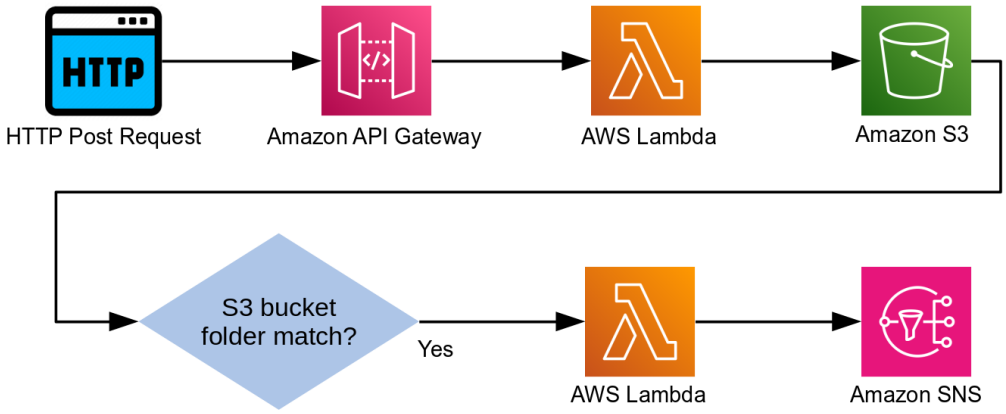

Fig. 4.2 Microbenchmark `api-publish-wrong-bucket-key`.

In summary, the limitations of general-purpose static analysis tools for event-driven serverless applications motivate the need for serverless-specific approaches to identify security-sensitive data flows. As further detailed in §2.4.1, previous research in other ecosystems has shown that some event-driven architectures can be analysed with *augmented* call graphs [85] and specialized testing techniques [41, 49]. However, these existing solutions cannot be applied to serverless platforms, as their design is based on the Node.js and Android programming models. As such, they do not process the complex infrastructure-related information essential to the analysis of serverless applications.

4.2.2 Information Leakage Example

An example of inter-procedural data flow that can lead to information leakage is provided by our microbenchmark `api-publish-wrong-bucket-key`. The architectural diagram in Fig. 4.2 shows that a HTTP Post request is again the user-controlled entry point of the application. The request is processed by the API Gateway, and triggers the execution of a handler. The latter inspects the request body and identifies a file that is uploaded to a S3 bucket. If the upload operation stores the file in a specific folder, identified by the `prefix` tag of the configuration file[5], then another handler is executed. This processes the user-provided information, which is included in an email notification sent via the AWS SNS service.

However, the current microbenchmark implementation forces the first handler to access a different folder, thus preventing the execution of the second. We make this choice to enhance the quality of our test suite, as this is a case that might lead to a false positive result. Without sufficiently accurate mechanisms to process infrastructure and application code, in fact, the

[5]This is an example of event filtering, a concept introduced in §2.1.1.

Table 4.1 Summary of the `CloudBench` microbenchmark suite. This table specifies the categorization of the microbenchmarks, the included vulnerabilities, and the LOC range (CI=Code Injection, II=Information Integrity, IL=Information Leakage, LOC=Lines Of Code, NN=None).

Category	Amount	CI	II	IL	NN	Min LOC	Max LOC
Inter-procedural	30	22	0	2	6	20	70
Intra-procedural	9	4	1	2	2	5	21
Simple Applications	1	1	0	0	0	144	144

Table 4.2 Summary of the `CloudBench` microbenchmark suite. This table specifies the categorization of the microbenchmarks and the supported cloud services.

Category	Amount	S3	DynamoDB	SQS	SNS	Step Functions
Inter-procedural	30	30	14	3	3	0
Intra-procedural	9	6	3	0	0	0
Simple Applications	1	0	1	0	0	1

analysis tool might not be able to discriminate data flows going through different folders within the S3 bucket.

4.3 CloudBench

`CloudBench` consists of 40 AWS and Serverless Framework-compatible Python applications, which are categorized in Table 4.1 and Table 4.2. We observe that 30 of them, which represent 75% of the total number, are inter-procedural. This means that the expected source and sink are in different handlers, and the security-sensitive data flow between them *always* includes event-triggered code. As explained in our code injection-focused motivating example (§4.2.1), which is one of the microbenchmarks included in this category, inter-procedural data flows are harder to detect. We therefore choose by design to focus on them in order to enhance the quality of our suite. We note that Table 4.1 also provides a measure of the microbenchmarks' complexity in terms of lines of code (LOC). In the inter-procedural case, these range from 20 to 70.

In addition, `CloudBench` contains nine intra-procedural microbenchmarks. Albeit structurally simpler, as reflected in the lower number of LOC, they include important cases, such as NoSQL injection [1] and variants of an insecure code example described in the OWASP Serverless project documentation [100]. To improve the overall coverage, we also add a

simple application with 144 LOC, obtained by modifying a baseline [138] that includes an AWS step function.

Regarding the vulnerabilities, also reported in Table 4.1, twenty-seven `CloudBench` microbenchmarks contain a code injection vulnerability, while five focus on information leakage and integrity. The remaining eight do not contain any vulnerability and are designed to capture cases that can lead to false positive results. We emphasize that the vulnerabilities considered, although not serverless-specific, are listed among the most critical risks for serverless applications by the Cloud Security Alliance [34] and Podjarny *et al.* [103].

Table 4.2 shows that five AWS services are supported in total[6]. Four of them, i.e., S3, DynamoDB, SQS and SNS, are among the most frequently occurring in the `AWSomePy` dataset (Table 3.3). Additionally, the aforementioned simple application includes the Serverless Framework plugin dedicated to the AWS step function service [140]. We make this choice because 14 `AWSomePy` applications rely on this plugin (Table 3.2), among the highest number of occurrences in the reference dataset. It should be observed that our microbenchmarks do not yet include the third and fourth most frequent `AWSomePy` services, i.e., Lambda and SSM, respectively. From a security perspective, such configuration and management-oriented services need to be taken into consideration. However, as explained in Chapter 3, they introduce data flows that are difficult to detect statically, and we therefore leave to future work the implementation of suitable strategies.

To comprehensively document the coverage achieved by the presented suite, the characterization in terms of vulnerability and sink of each inter-procedural security-sensitive data flow is reported in the Tables 4.3, 4.4, 4.5, and 4.6, which are organized by target cloud service, i.e., the service that provides the targeted API[7]. Similarly, Table 4.7 details target cloud service, vulnerability, and sink of the intra-procedural data flows. It should be noted that the source of all the inter-procedural and intra-procedural data flows specified in such tables is the `event` input of a handler triggered by a HTTP request, the only exception being the intra-procedural `owasp-serverless-injection` and its variants[8]. In the latter cases, the source of the security-sensitive data flows is the S3 service API `list_objects`.

To further increase the overall coverage, the `CloudBench` microbenchmarks include both types of `boto3` interface object, i.e., `client` and `resource`, as they require different processing of the code under analysis. More specifically, since a `resource` provides a

[6]Several microbenchmarks include multiple cloud services. Each of them has a target cloud service though, which is determined by the specific `boto3` API [25] that the microbenchmark is supposed to test. This organization is reflected in the structure of the `CloudBench` repository, which, as mentioned in §1.5, we plan to open-source in the near future.

[7]The detailed characterization of the inter-procedural microbenchmarks in terms of target cloud service and other services used can be found in §A.2.

[8]The names of such variants begin all with the string `owasp-serverless-injection`.

Table 4.3 Data flow characterization. `CloudBench` inter-procedural microbenchmarks with S3 as target service (CI=Code Injection, II=Information Integrity, IL=Information Leakage, NN=None, VUL=Vulnerability).

Microbenchmark	VUL	Sink
`api-put-object-boto3-client`	CI	`os.system`
`api-put-object-bucket-assign`	CI	`subprocess.call`
`api-put-object-iam-plugin`	CI	`os.system`
`api-put-object-partial-import`	CI	`os.system`
`api-put-object-two-handlers`	CI	`subprocess.call`
`api-upload-file-boto3-client`	CI	`os.system`
`api-upload-file-bucket-assign`	CI	`os.system`
`api-upload-file-iam-plugin-custom`	CI	`os.system`
`api-upload-file-iam-plugin-inherit`	CI	`os.system`
`api-upload-file-iam-plugin`	NN	NN
`api-upload-file-kw-args`	CI	`os.system`
`api-upload-file-object-assign`	CI	`os.system`
`api-upload-file-partial-import`	CI	`os.system`
`api-upload-file-two-handlers`	CI	`os.system`

Table 4.4 Data flow characterization. `CloudBench` inter-procedural microbenchmarks with DynamoDB as target service (CI=Code Injection, II=Information Integrity, IL=Information Leakage, NN=None, VUL=Vulnerability).

Microbenchmark	VUL	Sink
`api-put-item-boto3-client`	CI	`os.system`
`api-put-item-dict-var`	CI	`os.system`
`api-put-item-kw-args`	CI	`subprocess.getoutput`
`api-put-item-misspelt-service`	CI	`os.system`
`api-put-item-table-assign`	CI	`subprocess.check_output`
`api-put-item-via-file`	CI	`subprocess.check_output`
`api-put-item-wrong-table-os-environ`	NN	NN
`api-put-item-wrong-table-os-getenv`	NN	NN
`api-put-item-wrong-table-str`	NN	NN
`api-put-item-wrong-table`	NN	NN

high-level object-oriented interface to a cloud service [126], this is normally followed by the initialization of an *intermediate* object, e.g., `Object` in Listing 4.4 (line 5), on which the targeted API is called.

Table 4.5 Data flow characterization. `CloudBench` inter-procedural microbenchmarks with SQS as target service (CI=Code Injection, II=Information Integrity, IL=Information Leakage, NN=None, VUL=Vulnerability).

Microbenchmark	VUL	Sink
`api-send-message-boto3-client`	CI	`os.system`
`api-send-message-kw-arg`	CI	`os.system`
`api-send-message-queue-assign`	CI	`os.system`

Table 4.6 Data flow characterization. `CloudBench` inter-procedural microbenchmarks with SNS as target service (CI=Code Injection, II=Information Integrity, IL=Information Leakage, NN=None, VUL=Vulnerability).

Microbenchmark	VUL	Sink
`api-publish-boto3-client`	IL	`sns.publish`
`api-publish-no-file-system`	IL	`sns.publish`
`api-publish-wrong-bucket-key`	NN	NN

Table 4.7 Characterization of intra-procedural microbenchmarks (CI=Code Injection, II=Information Integrity, IL=Information Leakage, NN=None, TS=Target Service, VUL=Vulnerability).

Microbenchmark	TS	VUL	Sink
`delete-all-bucket-objects`	S3	II	`all.delete`
`owasp-serverless-injection-eval`	S3	CI	`eval`
`owasp-serverless-injection-exec`	S3	CI	`exec`
`owasp-serverless-injection-iam-plugin`	S3	CI	`subprocess.call`
`owasp-serverless-injection-no-perm`	S3	NN	NN
`owasp-serverless-injection`	S3	CI	`subprocess.call`
`api-scan-boto3-client`	DDB	IL	`scan`
`api-scan-no-permission`	DDB	NN	NN
`api-scan-table-assign`	DDB	IL	`scan`

Finally, we highlight that `CloudBench` includes in total 30 new microbenchmarks in comparison with the suite published with our paper P2[9]. We expand both the inter-procedural and intra-procedural category, add a simple application, and provide support for features not included in our preliminary work, such as permission-relevant deployment tool plugins, event filters, and environment variable-based configuration.

[9] https://github.com/giusepperaffa/serverless-security-microbenchmarks

4.4 Related Work

The measurement study by Wang *et al.* [164] aims at comparing architecture and resource management strategies in three major serverless platforms. While identifying security risks and vulnerabilities, the work relies on measurement functions rather than security-oriented benchmarks. Back *et al.* [16] design a microbenchmark specifically to evaluate the cost model adopted by four serverless providers. The microbenchmark is, therefore, designed to study exclusively the effect of number of function invocations and memory usage.

In contrast, FunctionBench [76] is a suite of workloads that includes both microbenchmarks and fully-fledged data-oriented applications. The latter are focused on image and video processing as well as machine learning tasks. The workloads are classified in terms of CPU, memory, disk I/O and network resources utilization, which confirms that they are designed to evaluate platform performance. Following its publication, FunctionBench was then expanded by adding one security-related application containing an implementation of a private key-based encryption and decryption algorithm [77]. Thus, the objective of this benchmark is to assess the performance of the serverless environment rather than providing a reference example of security vulnerability.

More recently, the SPEC Research Group (RG) for cloud computing has highlighted the need for more comprehensive serverless benchmarks [159]. To achieve this, the authors stress that new benchmarks should have a broader scope, e.g., by supporting open source platforms and the analysis of event propagation, and present a prototype benchmarking framework. Even though some of the proposed improvements have security implications, e.g., event analysis, the SPEC RG approaches the problem exclusively from a system and performance point of view. This is a major difference in comparison with the work presented in this thesis.

Recognizing the need for automated benchmarking of serverless platforms, Somu *et al.* propose PanOpticon [147]. Their tool facilitates the collection of useful performance-related metrics, such as execution time, memory usage and latency, and relies on Apache JMeter [72] to generate realistic workload scenarios. Albeit more advanced and configurable than the SPEC RG framework, PanOpticon, unlike FunctionBench, does not include a set of ready-to-use applications. FaaSdom [87] bridges this gap and provides a higher level of automation in order to effectively compare serverless platforms in terms of cost and system performance. However, while incorporating a range of workloads and supporting multiple languages, FaaSdom does not include security-focused tests.

4.5 Chapter Summary

In this chapter, we present `CloudBench`, a novel collection of 40 security-oriented microbenchmarks for serverless computing. `CloudBench` bridges a gap in the literature, which currently includes only serverless benchmarks designed for performance [164, 87] and cost-related evaluations [16]. To further explain the challenges of analysing serverless applications statically and show some of the developed microbenchmarks, the chapter begins with two motivating examples (§4.2). The first illustrates an application with an event-triggered data flow leading to code injection, whilst the second describes a case of information leakage. The principles behind such examples, derived from documented serverless-specific vulnerabilities [100, 71, 1], and the characterization of `AWSomePy` (Chapter 3) inform the design of the proposed suite.

As detailed in Table 4.1, 75% of the `CloudBench` microbenchmarks are inter-procedural, being these harder to detect. In addition, the suite features one simple, but fully-fledged, application that we obtain by modifying an existing baseline. With regard to the included vulnerabilities, 27 out 40 microbenchmarks are focused on code injection. To achieve better coverage, we also develop four cases of information leakage and one dedicated to information integrity. We underscore that `CloudBench`, a preliminary version of which is publicly available, can be extended to increase the number of supported cloud services and APIs. Thus, we hope that it will be an important reference for future research on static analysis of serverless applications.

Chapter 5

CloudFlow: A Framework for the Detection of Security-sensitive Data Flows

In this chapter, we present a novel framework to statically detect security-sensitive data flows in serverless applications. The framework relies on a serverless-specific analysis pipeline, and is evaluated against the previously introduced suite of microbenchmarks.

5.1 Introduction

To address the challenges elucidated in §1.1, we present `CloudFlow`, a fully automated framework to statically detect security-sensitive data flows in serverless applications. `CloudFlow` begins by processing the infrastructure code, which specifies the required cloud-based resources along with a host of configuration parameters. It then analyses the application code, which defines the business logic of the application. The information gathered enables the framework to automatically instrument the code under test, thus transforming the underlying asynchronous system into a synchronous equivalent. The chief advantage of the proposed technique is that the instrumented code can be inspected with a general-purpose static analysis tool capable of detecting security-sensitive data flows.

This chapter is structured as follows[1]. After introducing `CloudFlow` (§5.2), we illustrate the results of its evaluation (§5.3) against the microbenchmark suite `CloudBench`. We next discuss some design-related aspects of the proposed framework (§5.4) as well as the relevant literature, with the latter encompassing contributions in several areas, e.g., dynamic information flow frameworks and serverless infrastructure security (§5.5). The last section summarizes the key results of this chapter (§5.6).

[1]We remind the reader that additional information on the architecture of `CloudFlow` and its evaluation is available in Appendix B.

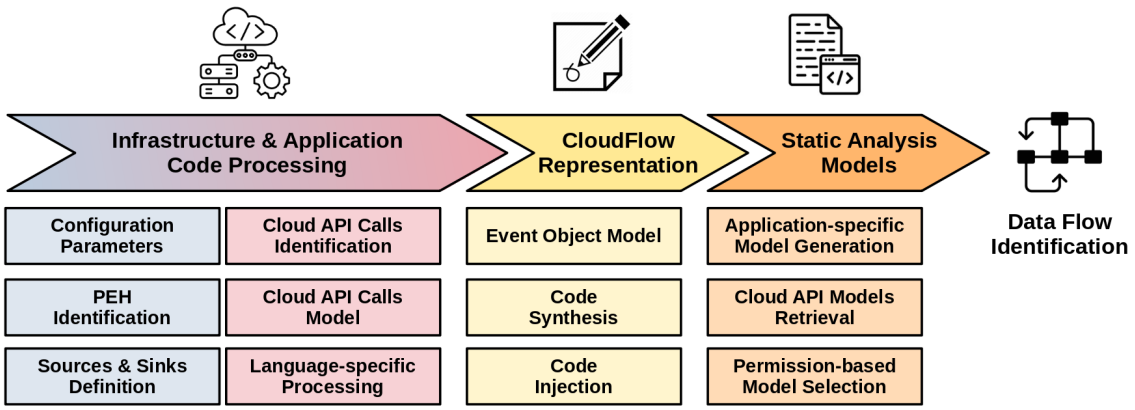

Fig. 5.1 Proposed static analysis pipeline for serverless applications (PEH=Permissions, Events and Handlers).

5.2 CloudFlow

This section begins with an overview of `CloudFlow` (§5.2.1). We then describe in detail how the framework processes infrastructure code (§5.2.2) and application code (§5.2.3). Finally, we explain how the aforementioned synchronous representation (§5.2.4) is obtained, and how the static analysis models required to identify security-sensitive data flows are prepared (§5.2.5).

5.2.1 Overview

The high-level diagram in Fig. 5.1 shows the three main stages of `CloudFlow`'s execution, each of which consists of a number of steps. The first stage of `CloudFlow` analyses the infrastructure and unmodified application code. It begins by extracting configuration parameters, e.g., environment variables, from the infrastructure code (*Configuration Parameters*). `CloudFlow` then extracts permissions, events, and handler-related information (*PEH Identification*). Upon completion of these steps, it is possible to identify the allowed cloud API calls, the events processed by the application, and the handlers executed as a consequence of these events. Some of this information is used in our event object models, which are configured later as part of the `CloudFlow` synchronous representation stage (§5.2.4). The infrastructure code also specifies user-controlled entry points of the application, i.e., *sources*, along with security-sensitive *sinks*, e.g., cloud-hosted repositories and databases, which `CloudFlow` next extracts (*Sources & Sinks Definition*). We note that processing the infrastructure code is the initial step towards addressing challenge C1. Moreover, extracting permission-related information, such as that shown in Listing 4.1, helps address challenge C2.

To complement the infrastructure code processing, we also analyse the application code. This enables us to identify cloud API calls (*Cloud API Calls Identification*) and to extract the information included in their inputs (*Cloud API Calls Models*). These are processed via models that we develop starting from the API documentation. Specifically, we rely on the API documentation to identify relevant API call inputs, such as names of cloud-based resources. We highlight that the analysis of these inputs is required to complete the configuration of our event object models, and is, therefore, another step towards addressing challenge C1. The API call models, instead, are our solution to challenge C3, as they allow incorporating an approximation of the cloud service functionality into our approach. Finally, we observe that language-specific steps might also be needed to complete the processing of the application code (*Language-specific processing*). This can provide the underlying static analysis tool with additional information that improves the accuracy of the analysis. For example, dynamic languages (e.g., Python) can benefit from an additional automated type annotation step (§5.2.3).

The second stage of `CloudFlow` focuses on creating a synchronous representation of the analysed application. This enables general-purpose static analysis tools to analyse the application. Starting from the event object model (*Event Object Model*), the framework synthesizes (*Code Synthesis*) and then injects the instrumentation code (*Code Injection*). Specifically, for each cloud API that triggers the execution of a handler, we inject two distinct code statements. The first initialises the event object, whilst the second is an *explicit* handler call where the event object is one of the inputs. This step helps us transform an event-driven asynchronous application into a synchronous one for the purpose of static analysis. Considering the importance of this representation, we provide a detailed example in §5.2.4.

The last stage of `CloudFlow` creates the static analysis models necessary for detection of security-sensitive data flows. These models are required by the underlying analysis tool to identify sources and sinks. `CloudFlow` is capable of generating application-specific models (*Application-specific Model Generation*). They target, for instance, the public API endpoints of the application under test. In addition, we develop cloud API-specific models, which are, in contrast, application-independent (*Cloud API Model Retrieval*). To minimize the number of false positives, the latter category of models is also processed to select only those relevant to allowed cloud APIs (*Permission-based Model Selection*). Once all the required static analysis models are available, the instrumented application can be analysed by running a general-purpose tool with data flow detection capabilities (*Data Flow Identification*).

`CloudFlow`'s high-level architecture is not specific to a particular cloud platform. However, our current implementation targets AWS Lambda [13] applications deployed using the Serverless Framework deployment tool [141]. Similarly, `CloudFlow` can in principle be

used with any general-purpose static analysis tool, but we focus for the remainder of this chapter on the analysis of Python applications using Pysa, an interprocedural static analysis framework [113]. We next describe in more detail each stage of `CloudFlow`'s execution[2].

5.2.2 Infrastructure Code Processing

As illustrated in §4.2, serverless applications require the definition of cloud resources that will be deployed for the application along with their configuration. `CloudFlow` processes these definitions automatically to extract relevant information that will be later used to build the representation necessary to perform data flow analysis. Serverless applications normally define infrastructure code within YAML files. We begin their processing with a custom YAML parser that first stores the infrastructure code in a tree-like data structure. This is then recursively traversed with the strategy illustrated by Zaczyński [18][3]. During the traversal, we use regular expressions to detect configuration parameters that need to be resolved, e.g., cloud-based repositories' names that depend on the application development stage. To resolve such parameters, we retrieve the necessary values from the YAML file or by accessing, if needed, external files referenced within the infrastructure definition. This step is important to the precision of our analysis, as the Serverless Framework allows specifying resource-related configuration data through parameters included in the infrastructure code. The latter is finally mapped into a new data structure from which we extract security-relevant information. This includes, for example, associations between events and their triggered handlers, and permissions with cloud-based resources. To obtain this information, `CloudFlow` leverages an abstraction layer that embeds the YAML tags and structures used for permissions, events, handlers, plugins, and their configurations.

To better analyse the execution context of the application under test and increase the precision of the analysis, `CloudFlow` also extracts information about environment variables and event filters. Environment variables are frequently used to specify cloud API inputs. For example, the API `upload_file` in Listing 4.3 is called with the bucket name input set to `os.environ['BUCKET_NAME']`. In contrast, event filters are analysed because they configure the AWS infrastructure to raise events only under specific conditions (§2.1.1). In the case of the S3 service, for instance, a filter defined as `prefix: uploads/` implies that a given handler is executed only if the event originates from the folder `uploads`.

[2]A more implementation-oriented description of the `CloudFlow` architecture can be found in §B.1.

[3]Specifically, the traversal strategy employed by `CloudFlow` is described by Zaczyński in the section *Build a Tree of Nodes* of his tutorial. The strategy is included among those that allow YAML documents to be parsed at a low level, and relies on the widely adopted PyYAML library [122].

The functionality provided by the deployment tool can be extended with plugins, which can also be modeled and incorporated in `CloudFlow`. As previously mentioned (§2.1.1), such plugins can affect both the application architecture and its permissions. For this reason, plugin modeling is an important practical contribution of our framework. Our plugin models combine the logic detailed in their documentation with information obtained from the YAML file. As a result, they encapsulate data structures that can be queried by other components to refine their analyses. `CloudFlow` features full support for the security-oriented plugin that enables handler-level permissions [139]. In addition, we provide basic support for the plugin dedicated to AWS step functions [140], which is among the most frequently used in real-world applications (Table 3.2).

5.2.3 Application Code Processing

Serverless applications access the cloud-based platform through language-specific Software Development Kits (SDKs), such as the Python library `boto3` [25]. These provide interface objects, e.g., `sqs` in Listing 4.2, on which service-specific APIs can be called[4]. Therefore, `CloudFlow` processes the Abstract Syntax Tree (AST) of the application code to identify such objects and, subsequently, the cloud API calls[5]. Their security-relevant inputs are then extracted with documentation-based models. Importantly, to increase the precision of our analysis, we also attempt to statically *resolve* the values of the cloud API inputs by using the configuration parameters obtained from the infrastructure code (§5.2.2).

Finally, since our current implementation focuses on Python, our framework also adds type annotations as necessary, as the language includes only an optional type system. The absence of type-related information, in fact, prevents static analysers from identifying issues, thus leading to false negatives. `CloudFlow` relies on the automated type inference feature of Pyre [111], the static type checker on which Pysa is built, to add type annotations. However, Pyre supports only the core language types, such as lists and dictionaries, and cannot be used to insert the necessary `boto3`-specific type annotations. As illustrated by the example provided in §5.2.4, to overcome this limitation, we include these additional type annotations by modifying the AST nodes of the SDK-provided interface objects.

[4]The `boto3` library provides two types of interface objects, i.e., `client` and `resource` [126]. `CloudFlow` supports both of them, and manages the differences through configuration files.

[5]Examples of AST-based extraction of information from source code can be found in a publicly available talk given by the author in November 2024 [59].

5.2.4 CloudFlow Representation

The purpose of this stage is to provide a synchronous-like program execution flow, which we achieve in three steps. First, the previously extracted information is used to populate the relevant event object model[6]. Second, the output provided by the event object model is synthesized as an AST node and then injected as part of an initialization statement. Third, a call to the triggered handler, featuring the event object among its inputs, is AST-synthesized and added to the application code. This three-step process is repeated for each cloud API call that is allowed by the application permissions and that raises events triggering the execution of other handlers.

To further clarify these important steps, Listing 5.1 shows how we instrument the `upload_file` API call in the motivating example presented in §4.2.1. In this case, the initialization statement (lines 9-11) contains the event name, obtained from the infrastructure code, and information on the cloud-based repository (i.e., bucket) extracted from the API call inputs. This is followed by an explicit call to the triggered handler, i.e., `onS3Upload`, at line 12. Note that the instrumentation code is injected *after* the cloud API call site to emulate a standard function call.

```
1  def onSQSMessage(event, context):
2      # After initialization statements
3      uploadFileFolder = 'uploads'
4      uploadFileName = msgBodyDict + '.txt'
5      s3BucketKey = os.path.join(uploadFileFolder, uploadFileName)
6      localFile = os.path.join('/tmp', uploadFileName)
7      s3_client: S3Client = boto3.client('s3')
8      s3_client.upload_file(localFile, os.environ['BUCKET_NAME'],
       s3BucketKey)
9      event = {'Records': [{'eventName': 'ObjectCreated:*', 's3': {'
       bucket':
10         {'name': os.environ['BUCKET_NAME'], 'arn': os.environ['
       BUCKET_NAME'
11         ]}, 'object': {'key': s3BucketKey}}}]}
12     s3uploadhandler.onS3Upload(event, context)
13     return
```

Listing 5.1 Handler with instrumented API call. Lines 9-11 show the initialization statement injected for the event object. The explicit call to the triggered handler is injected on line 12. Note that the `boto3` interface object `s3_client` includes the type annotation `S3Client` added by `CloudFlow` (line 7).

[6]An event object model contains an approximation of the data structure encapsulated in the platform event. The documentation folder of the `CloudFlow` repository, which we intend to open source in the future, includes event object models in JSON format.

5.2.5 Static Analysis Models

`CloudFlow` affords a layer of automation that sits atop Pysa. To the best of our knowledge, the latter is the only open-source tool designed to detect inter-procedural and security-sensitive data flows in Python code. As detailed in §2.5.2, Pysa relies on sources and sinks classified in categories, e.g., user-provided and remote code execution, and included in *models*. These provide additional contextual information by specifying, for instance, function signatures and class definitions. The targeted data flows are detailed via *rules*, where the relevant sources and sinks are listed, along with optional metadata, e.g., intermediate function calls.

Our framework leverages the set of models and rules shipped with Pysa. While these constitute an important starting point, they are generic and need to be complemented with both application and cloud API-specific models. For this reason, `CloudFlow` automatically generates Pysa models that mark as user-provided sources the event objects processed by all the handlers. This approach, which relies on the analysis of the application code AST to extract the handlers' signatures, allows us to analyse two sets of events. The first includes those intended to be controlled by an user, whereas the second contains events that become user-provided because of erroneous or malicious workflow manipulations [103].

As regards the cloud API-specific models, we develop them after identifying `boto3`-specific security-sensitive sinks, for instance the DynamoDB API `scan` [1]. Such models complement those provided by Pysa for generic security-sensitive sinks, such as `os.system` (Listing 4.5). It is noteworthy that our custom Pysa models are automatically disabled if they target cloud APIs disallowed by the application permissions. This is an additional serverless-specific feature of our framework that reduces the number of false positives.

5.3 Evaluation

Previous research shows that security tools can be tested and compared effectively using microbenchmarks [7]. Since existing serverless benchmarks are designed to assess platform performance [164, 87] and cost effectiveness [16], we evaluate `CloudFlow` against the security-oriented microbenchmark suite `CloudBench`, introduced in Chapter 4.

5.3.1 Results

The results of our microbenchmark evaluation are reported in Table 5.1. `CloudFlow` succeeds in identifying the expected data flows in 37 cases out of 40. As further detailed in the ablation study below, these results show that the framework improves the baseline provided in our previously published paper P2. The three failed inter-procedural microbenchmarks, which

Table 5.1 Summary of `CloudFlow` evaluation (EVI=Environment Variable Inspection, INTER=Inter-procedural, INTRA=Intra-procedural, PLA=Plugin Analysis, REA=Resource Analysis, SA=Simple Applications).

Microbenchmarks		**Baseline**		**CloudFlow**		**CloudFlow Feature Contributions**		
Category	**#**	**PASS**	**FAIL**	**PASS**	**FAIL**	**EVI**	**PLA**	**REA**
INTER	30	19	11	27	3	3	4	4
INTRA	9	8	1	9	0	0	1	0
SA	1	0	1	1	0	0	1	0

include one false positive and two false negatives, are also analysed in the remainder of this section.

Ablation study. The number of microbenchmarks passed grows from 27 in the baseline framework to 37 in the `CloudFlow` evaluation, with the highest increase, i.e., eight microbenchmarks, in the inter-procedural category. To explain this result, we note that the baseline framework does not support the following three `CloudFlow` features: environment variable inspection, plugin and resource analysis. We therefore selectively disable them to identify those that, either singly or in combination with others, determine a change in the result of a microbenchmark execution.

As summarized in Table 5.1, the most frequently occurring feature is plugin analysis, which enables detection of the expected data flow in four inter-procedural microbenchmarks, one intra-procedural, and the simple application. We note that the plugin analysis feature is sufficient to successfully execute all of them. Our study also shows that resource analysis is the second most frequent feature, with four occurrences in total. Enabling this feature alone identifies the target data flow in a single case. In the remaining three, both resource analysis and environment variable inspection have to be enabled in `CloudFlow` to pass the microbenchmarks[7].

False positive. We obtain a false positive result with the `api-publish-wrong-bucket-key` microbenchmark. This includes a handler, shown in Listing 5.2, that is triggered when the application receives a HTTP Post request. After inspecting the body of the latter, the code identifies and then uploads a file to a S3 bucket. However, the microbenchmark includes an event filter, and the upload operation raises an event only if the file is stored in a specific folder, configured via the YAML tag `prefix`. Despite inspecting the inputs of the `upload_file` API, `CloudFlow` is unable to obtain a fully resolved value for the variable `s3BucketKey`.

[7]A breakdown of the aggregate figures reported in Table 5.1 can be found in Table B.1.

This, in fact, is initialized through a call to `os.path.join`, and cannot therefore be compared with the relevant folder name in the configuration file.

```
def onHTTPPostEvent(event, context):
    msgBodyDict = event['body']
    uploadFileFolder = 'upload-folder'
    uploadFileName = msgBodyDict + '.txt'
    s3BucketKey = os.path.join(uploadFileFolder, uploadFileName)
    localFile = os.path.join('/tmp', uploadFileName)
    s3_client = boto3.client('s3')
    s3_client.upload_file(localFile, os.environ['BUCKET_NAME'],
   s3BucketKey)
    return
```

Listing 5.2 Handler in `api-publish-wrong-bucket-key`.

We note that our framework analyses cloud API calls by first checking whether the application has the required permissions, and then refines the result by extracting additional information, e.g., event filtering-related, from the API inputs. When obtaining such information is not possible, `CloudFlow` exclusively relies on the API permissions to minimize the number of false negatives. As a result, a non-existing data flow is detected in this case.

False negatives. The microbenchmark `api-put-item-via-file` provides the first false negative of our evaluation. In this case, Pysa is not able to identify the expected data flow due to a taint propagation issue affecting the code in Listing 5.3, included in the handler `onS3Upload`. While the taints from the intended source and sink are correctly propagated to the variables `outputFileObjA` and `outputFileA`, respectively, the two partial data flows are not merged. We note that context manager-specific models, which are now part of `CloudFlow`, are not sufficient to rectify this issue. After contacting the Pysa community for support[8], we confirmed that this is a limitation of the static analysis engine, which treats the two variables as independent. Consequently, no taint is propagated between them. For the expected data flow to be successfully detected, the code in Listing 5.3 would need to be re-structured.

```
outputFileA = os.path.join('/tmp', 'outputFileA.txt')
with open(downloadPath, 'r') as inputFileObj, open(outputFileA, 'w')
    as outputFileObjA:
    inputFileContentsList = inputFileObj.readlines()
    outputFileObjA.write(inputFileContentsList[0])
updateDynamoDBTable(outputFileA, outputFileB)
```

Listing 5.3 Code affected by a taint propagation issue.

[8] https://github.com/facebook/pyre-check/issues/797

The second false negative result is obtained with the `api-publish-boto3-client` microbenchmark, which includes the handler `onS3Upload` also present in the motivating example described in §4.2.1. Surprisingly, only the following three partial data flows are reported by Pysa: (i) between the intended source (located in the handler `onHTTPPostEvent`) and the `with` statement in Listing 4.4, (ii) between the `onS3Upload` input `event` and the final sink (located in the handler `onDynamoDBStream`), and (iii) between the `onS3Upload` input `event` and the `with` statement in Listing 4.4. Our investigation shows that both the forward and the backward taints associated to the variables `downloadPath`, `inputFileObj`, and `inputFileContentsList` are correct. Since all the above partial data flows are identified through the same Pysa rule, we hypothesize that this false negative is caused by a tool post-processing configuration parameter that we were unable to identify[9].

5.4 Discussion

Analysis Framework Scope. As confirmed by the results of our real-world applications analysis presented in Chapter 6, `Cloudflow` is particularly suited to detecting code injection and information leakage vulnerabilities. Their identification relies on models, either automatically generated by `CloudFlow` or provided by Pysa, that classify relevant sources and sinks as security-sensitive. While this technique addresses several of the serverless risk factors described by the CSA [34], others require different approaches. Observability tools, for instance, are necessary to ensure that functions are adequately monitored and log files with security-relevant events are generated. Other risks identified by the CSA include information leakage caused by cross-execution data persistency and insecure third-party dependencies. We observe that the former depends upon the specifics of the serverless platform, and, as such, is challenging to detect statically in a reliable manner. In contrast, insecure dependencies can be identified with specialized static analysis tools [103]. Our framework therefore provides an effective way of complementing existing serverless security tools, especially those that rely on dynamic analysis.

API Documentation Analysis. At present, `CloudFlow` relies on API models obtained from manual analysis of API documentation. The purpose of the analysis is to identify security-relevant inputs, such as the names of cloud databases. Although automated analysis via NLP methods is possible [153], several recent works, e.g., on AWS environments [143] and

[9]For future reference, we also note that experiments performed during the preliminary testing of `CloudFlow` show that Pysa is unable to analyse repositories containing files with the same name. As part of this research, we reported the issue to the Pysa community in April 2024 [58]. However, to the best of our knowledge, only a workaround for the problem is currently available.

Android security [154, 127], still employ manual analysis of a limited set of APIs. This also facilitates validation of the resulting models.

We note that `CloudFlow` requires only a simple API documentation analysis. Consequently, it is possible to automate this process by integrating an API documentation processing framework, e.g., DocFlow [153], into our analysis pipeline. We leave this task to future work.

Generalizability. `CloudFlow` currently identifies security-sensitive data flows in serverless applications implemented in Python. This is, in fact, the only language supported by Pysa, the static analysis tool on which our current implementation relies. However, we emphasize that the development of the proposed methodology was guided by challenges that are language-independent. As a result, `CloudFlow`'s architecture is generic, and can be applied to any programming language.

Specifically, four steps are required to achieve this. The first step (i) is to identify a toolchain that allows parsing the AST and synthesizing code from it. A language-specific static analysis tool with data flow detection capabilities (ii) is also necessary to replace Pysa (e.g., SootUp [74] for Java applications). The third step (iii) is to configure SDK-specific type annotations, if supported by the target language. In our implementation, these are `boto3`-specific. Finally, step (iv) consists of creating documentation-based API models, which depend on the SDK. These can be automatically generated with an API documentation processing framework.

Furthermore, we highlight that `CloudFlow`'s architecture can also be applied to any cloud provider. This requires a preliminary analysis to ascertain how the application code interacts with the cloud services. In our experience, this is routinely implemented via an SDK for simplicity. Once this is confirmed and the target language is chosen, our architecture can be applied by following the four steps described above.

A possible avenue for future work is a JavaScript-focused extension of `CloudFlow`. According to a characterization of the Wonderless dataset [45], JavaScript is the most popular language for the implementation of open-source serverless applications. However, a recent evaluation by Brito *et al.* of nine mainstream tools indicates that static detection of security vulnerabilities in JavaScript code remains problematic [30]. In particular, existing tools struggle with the highly dynamic constructs of the language and the complexity of the dependency system. Consequently, although porting `CloudFlow` to JavaScript is feasible and worth pursuing given the popularity of JavaScript in the serverless domain, whether performance comparable to our Pysa-based `CloudFlow` can be achieved is an open question.

5.5 Related Work

Static Analysis of Serverless Applications. The work illustrated in this chapter builds on the research presented in our paper P2, where we propose a prototype-level static analysis framework that relies on manually-injected handler calls and type annotations. Differently from that prototype, `CloudFlow` achieves automated code instrumentation by parsing and modifying the AST of the inspected application. Indeed, our framework fully automates the analysis process, builds a specific representation to facilitate the detection of relevant data flows, and supports the analysis of environment variables, event filters, and deployment tool plugins.

To support information flow and security analysis, along with automatic generation of documentation and code maintenance tasks, Obetz *et al.* [96] present a novel approach to extending the concept of call graph to serverless environments. The work introduces the idea of *extended service call graph*, which aims to provide a comprehensive description of the structure of an application. To this end, the authors incorporate nodes and edges associated with external platform services available in the cloud-based execution environment. Tests conducted with JavaScript applications retrieved from the AWS Serverless Application Repository [14] show that the approach is feasible, despite its limitations. We highlight that Obetz *et al.*, who also propose an improvement of their graph construction algorithm [95], make use of API-specific functions' summaries to consider the effects of platform services and external libraries. In contrast, we rely on event object models, which are defined once for each supported service and automatically populated by processing the infrastructure and the application code. Furthermore, our main objective is the detection of security-sensitive data flows, not the generation of a comprehensive call graph.

Wang *et al.* [165] develop a performance model for serverless that relies on data flow and code profiling information. The former is obtained with Pysa, whilst the latter is gathered through code instrumentation. Unlike our work, the authors do not process the infrastructure code, and focus their attention on the application's response time rather than its security.

Dynamic Information Flow Frameworks. Research has also been conducted on dynamic information flow frameworks for serverless. These require modified execution environments and, consequently, the deployment of dedicated code on an existing platform. Among them, Valve [38] provides a *workflow-centric* approach to monitoring and authorizing information flows. More specifically, the framework, which extends the open-source serverless platform OpenFaaS [97], monitors policy-based information flows by means of taint labels. Its implementation includes an *agent* and a *controller*. The agent is focused on monitoring API calls, which are recorded as taints, and disk access, with the latter being a necessary step to

prevent information leakage. By contrast, the controller module processes the taints generated by the agent to audit the information flow of the application, and orchestrates the enforcement of the security policies[10]. Similarly, WILL.IAM [131] relies on permissions graphs to implement a serverless-specific access control model. Differently from Trapeze [4], which implements a language-based approach, neither Valve nor WILL.IAM require modifying the application code.

Sirone Jegan *et al.* [145] improve Valve by proposing an extensible security framework for serverless applications called SecLambda. In addition to tracking the control flow of an application, its key feature is that it relies both on global application-related information as well as local function states. These, which are not processed by Valve, are used in SecLambda as an input to a *guard* module, which runs additional code to perform security-specific tasks, e.g., credential protection and rate limiting of serverless functions. To achieve this, such functions are executed within a modified execution environment built on top of the gVisor container runtime [63], which aims at reducing the attack surface of a serverless function, e.g., by controlling the system calls. Similarly to Valve, SecLambda also includes a *controller* module, which is responsible for coordinating and collecting function states from the guards, and for providing the guards with information about the global state of the application. It should be observed that SecLambda is extensible because the function states can be processed by user-defined code. More recently, Sirone Jegan *et al.* have also proposed Kalium [69], which enforces control-flow integrity by monitoring the order of execution of serverless functions.

We underscore that none of the frameworks mentioned in this section analyse serverless applications statically, the key objective of `CloudFlow`.

Serverless Infrastructure Security. Several systems target the infrastructure on which serverless applications rely. Alzayat *et al.* [5] focus their attention on the security issues caused by container reuse, very frequently adopted by FaaS providers to minimize cold-start delays. The devised system, Groundhog, prevents information leakage by restoring a private data-free memory snapshot before the execution of a serverless function. Groundhog has a negligible effect on latency and throughput. Firecracker [2] has similar objectives, but improves the standard container-based strategy with micro virtual machines including a reduced set of resources. In contrast, we focus on the analysis of serverless applications at coding time.

[10] It is worth noting that developer-specified authorization rules can be incorporated into Valve, though these can introduce vulnerabilities if not properly checked.

Trach *et al.* [156] propose the secure serverless platform Clemmys, which is significantly different from both the above-described systems and `CloudFlow`. The authors harden a FaaS platform by relying on the hardware-based shielded execution technology Intel SGX [67], which has been the basis for more generic cloud security research as well [19]. Clemmys is a low-throughput system that guarantees confidentiality and integrity of users' functions executed within the SGX *enclave*, i.e., a secure region of memory address space. However, the deployment of the system implemented by Trach *et al.* is not straightforward, as it requires SGX-specific optimizations and at least one additional software component for cryptographic key management. More generally, we note that the development of hardware-based systems to secure cloud infrastructure, e.g., Marghera [163], is an active area of research. Such systems, in fact, are essential for cloud providers. By contrast, `CloudFlow` is a developer-focused framework to statically analyse serverless applications.

Formal Semantics for Serverless. Jangda *et al.* [68] propose operational semantics consisting of six rules focused on the low-level behaviour of serverless platforms. Such rules, however, do not provide the event-level details necessary for our work. In contrast, Gabbrielli *et al.* [50] present a serverless-specific formal model based on a Serverless Kernel Calculus that includes both stateless functions and stateful platform services. Obetz *et al.* [95] further improve existing in-process and event semantics in order to capture service-specific characteristics, along with function-to-function and function-to-service data transmission. The latter work provides part of the theoretical foundations of our research, which translates those notions into a security-focused static analysis tool.

Serverless Design Patterns. In the first paper focused on serverless architecture and security, Hong *et al.* [66] show six AWS-focused design patterns to build security services, such as virus scanners and compliance checkers. The work was published in 2018, when the technology was still in its infancy. For this reason, we reckon that the key idea proposed by the authors, i.e., the implementation of custom security services based on the illustrated patterns, is now obsolete. The paper, for example, suggests using the event-driven pattern to deploy functions aiming at detecting SQL injection and cross-site scripting. As pointed out by Shields [144, Ch. 11], however, AWS now offers a Web Application Firewall (WAF) that implements this functionality, thus lessening the burden on the platform users. Some of the guidelines provided by Hong *et al.* are still relevant to serverless developers though. For instance, adopting the state machine pattern is beneficial, as the type of cloud service on which it is based, e.g., AWS Step Functions [140], normally includes a mechanism that automatically retries the execution of a function when an error occurs.

More recently, with a view to supporting both researchers and practitioners, Taibi *et al.* [151] review the available white and grey literature, and identify 32 serverless design patterns, organized into five categories. Although the work is not security-focused, some of the described patterns aim to increase the level of security of the deployed applications. Among them, the authorization category and the internal API pattern, where functionality that does not require an external endpoint is confined to dedicated functions [70], are noteworthy examples. Considering the importance of event propagation in the static analysis technique proposed in this thesis, we underscore that one of the pattern categories illustrated by Taibi *et al.* is event-management. Differently from our work, such patterns are not analysed to detect security-sensitive data flows, as they are proposed to address communication problems. Despite this, we note that their description highlights the benefit of having multiple, task-focused functions, along with granularly defined message queues, as these can distribute events more effectively. From a static analysis standpoint, both features pose a challenge. The work by Taibi *et al.*, however, shows that they have to be taken into account in order to improve the current tools and develop new ones.

Static Analysis of Interpreted Languages. Interpreted languages have played a key role in the development of cloud and web applications. Therefore, numerous research contributions on static detection of security vulnerabilities in such languages exist. Many of these, in particular, are JavaScript-specific [30, 149]. For PHP, Xie *et al.* [172] present a novel algorithm that relies on function summaries and symbolic execution to conduct inter-procedural analyses. Pixy [73] increases the precision of data flow analysis by inspecting literals and aliases. Hauzar *et al.* [64] propose an abstract interpretation framework for value and heap analysis. Son *et al.* [148], combine taint analysis with symbolic execution to detect denial of service vulnerabilities and missing authorization checks.

These PHP frameworks are different from `CloudFlow` for two reasons. First, they focus on the dynamic features of PHP. Second, they support security mechanisms used in PHP-based web applications. In contrast, we propose a language-independent approach focused on the characteristics of the serverless paradigm. Given their features, such frameworks could replace Pysa if `CloudFlow` were extended to support PHP. However, according to the Wonderless dataset statistics, this language is not popular in the serverless domain [45].

5.6 Chapter Summary

In this chapter, we present `CloudFlow`, which instantiates a novel approach to statically identifying security-sensitive data flows in serverless applications. `CloudFlow` uses information

extracted from infrastructure and application code to construct a synchronous representation of event-driven serverless applications. The proposed framework then generates application and cloud API-specific static analysis models. Together with the synchronous representation, these can be provided to a general-purpose static analysis tool for data flow detection.

We evaluate `CloudFlow` against our security-oriented microbenchmark suite `CloudBench` (Chapter 4). Our results show that `CloudFlow` passes 92.5% of the microbenchmarks, i.e., 37 out of 40, confirming its effectiveness. The ablation study included in §5.3.1, in addition, illustrates how the presented analysis framework improves the prototype discussed in our paper P2. We finally investigate the failed microbenchmarks, and we identify one false positive and two false negatives. While the former is due to a limitation of `CloudFlow`, the latter are caused by Pysa, the underlying static analysis tool.

Chapter 6

Analysis of the AWSomePy Dataset with CloudFlow

In this chapter, we illustrate the analysis of our dataset conducted with the proposed static analysis framework. To the best of our knowledge, this is the largest security-focused analysis of serverless applications currently available in the literature.

6.1 Introduction

As explained in §5.5, previous research on serverless static analysis does not explore security, instead focusing on call graph generation [96, 95] and performance evaluation [165]. Furthermore, recent large-scale analyses of cloud-based repositories [35, 31] have a narrow scope, as they assess the security of such resources without fully inspecting the applications that rely on them. To bridge this gap, we identify 104 real-world applications in the `AWSomePy` dataset (Chapter 3), and we analyse them with `CloudFlow`. The objectives of our analysis are twofold. First, we aim to gain insight into the types of security vulnerability that can affect such applications. Second, to comprehensively evaluate our framework, we want to ascertain whether it is capable of analysing real-world applications, which implies assessing the caused overhead as well.

This chapter is structured as follows[1]. We begin with the results of the initial dataset triage (§6.2). We then show the identified security-sensitive data flows and vulnerabilities found (§6.3). The latter are further illustrated with three case studies (§6.4). Next, we present our performance evaluation, based on the recorded `CloudFlow` execution times (§6.5). After further discussing the obtained results (§6.6), we elucidate the differences between our work

[1]We remind the reader that additional information on the analysis of the `AWSomePy` dataset conducted with `CloudFlow` is available in Appendix C.

Table 6.1 Summary of the AWSomePy dataset triage. The categorized repositories are listed in the specified tables.

Category	Total No.	Repositories
Template	23	Table C.1
CloudFlow-incompatible	6	Table C.2
Pysa-incompatible	6	Table C.3
Infrastructure code inconsistent	4	Table C.4
Application code inconsistent	2	Table C.5

and previous research contributions (§6.7). A summary of this chapter is included in the last section (§6.8).

6.2 Dataset Triage

Our real-world application analysis is based on the AWSomePy dataset, which consists of 145 Python serverless applications collected in 2022 (§3.2). To ensure that our analysis is performed on applications without inconsistencies and fully compatible with CloudFlow, we conduct an initial triage of this dataset. This triage step filters out 41 applications. As detailed in Table 6.1, we classify 23 of them as templates (not real applications), since they contain examples demonstrating design patterns or the functionality offered by specific cloud services.

We also identify a total of 12 applications that are incompatible with our framework. Specifically, six of them were developed with an obsolete version of Python or relied on a legacy API system instead of the boto3 SDK. The other six applications are Pysa-incompatible because the names of files or folders contain either characters or language keywords not allowed in Python identifiers. As a result, the generated Pysa models are invalid. To remedy this problem, we contacted the Pysa community, but, at the time of writing, it is still unresolved[2].

A further four applications have inconsistent infrastructure code because of missing configuration parameters, references to unavailable application modules, and commented out sections. Similarly, we identify two cases of inconsistent application code caused by Python syntax errors and reliance on code not included in the repository. Note that the details of our dataset triage are reported in §C.1.

[2]https://github.com/facebook/pyre-check/issues/899

6.3 Security-sensitive Data Flows

`CloudFlow` detects a total of 205 security-sensitive data flows in the remaining 104 applications analysed. Specifically, as illustrated in Fig. 6.1, such data flows are identified in 36 applications belonging to this set. The diagram, which reports the applications ranked by the number of security-sensitive data flows, shows that the latter is greater than ten only in three cases, i.e., the applications 032AA, 063AA, and 128AA[3]. Therefore, as confirmed by the cumulative distribution visible in Fig. 6.2, more than 90% of the identified applications contain ten security-sensitive data flows at most.

To gain additional insight into our results, we analyse the size of such applications as well. As expected, the diagram in Fig. 6.3, where the applications are sorted by LOC, shows that the three with the highest number of security-sensitive data flows are among the largest. Their LOC, in fact, range between 2,117 and 2,698. However, the same diagram highlights that there is not a strict correlation between LOC and the amount of data flows detected. Consequently, both the functionality offered by an application and the specifics of its implementation are important factors. Most of the data flows identified in application 032AA, for example, are caused by the presence of HTTP API endpoints[4] or potentially unsafe object deserialization [40]. In the case of application 063AA, instead, code imported dynamically is the reason behind the detection of all the data flows.

It is important to note that a security-sensitive data flow will not always result in a security vulnerability. For instance, code deployed to perform a data transfer between two AWS S3 buckets may be considered an information leakage vulnerability if the transferred information includes personal or sensitive data. Since the nature of the information cannot be deduced via static analysis, it is therefore marked as a security-sensitive data flow that has to be manually reviewed to verify if it leads to a security vulnerability. Because of this, all the data flows identified by `CloudFlow` were manually validated, and then categorized as shown in Table 6.2. Our analysis reveals that 11 of them, present in five different applications, are actual vulnerabilities. We note that four are detected via custom Pysa models automatically generated by our framework, while none of them include code injected by `CloudFlow`.

In contrast, a total of 64 data flows that could not be classified with certainty are included in the category *Potential Vulnerability*. We opt for this classification in two cases. First, when the data flow involves a large number of function calls, which may or may not be event-triggered. Second, when previously deployed and configured cloud-based resources

[3]The mapping between the application identifiers visible in Fig. 6.1 and the `AWSomePy` repositories is specified in Table C.6.

[4]The examples presented in §2.2.2 and §4.2 illustrate cases of security-sensitive data flows where the source is a HTTP request.

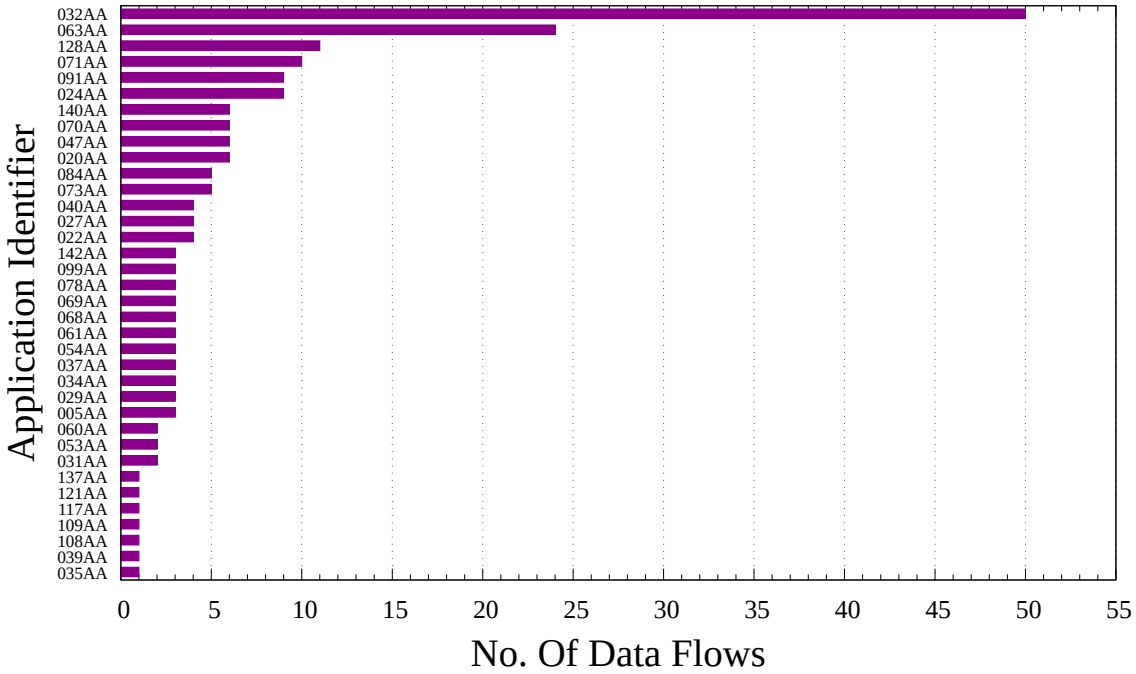

Fig. 6.1 Analysed `AWSomePy` applications vs number of security-sensitive data flows.

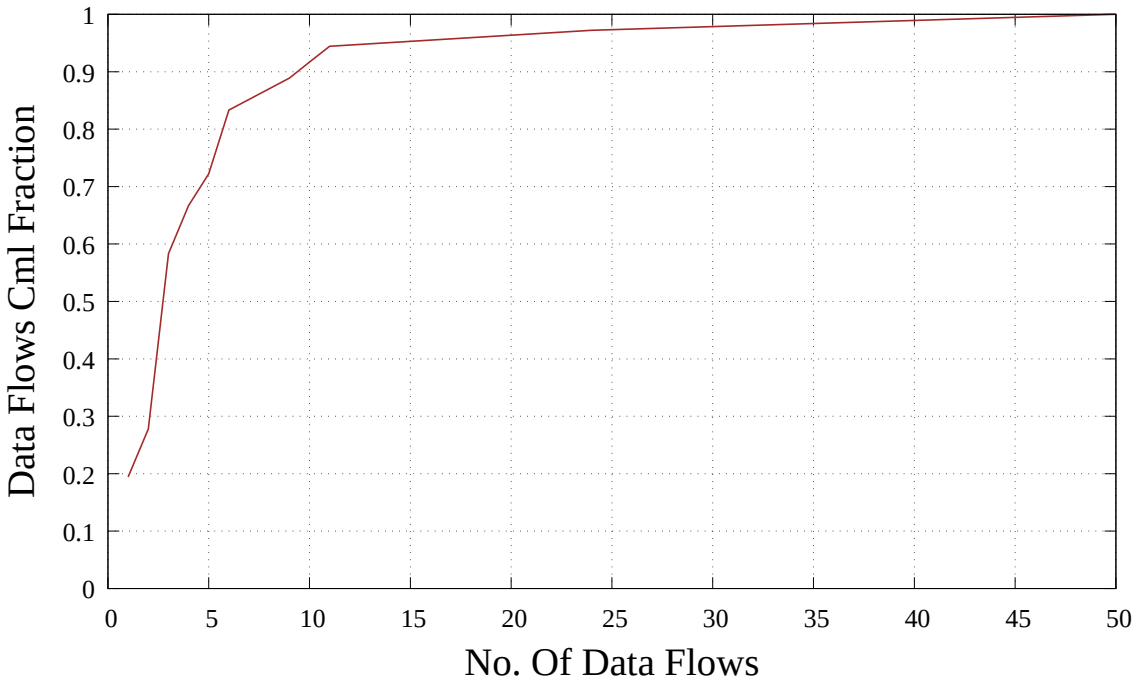

Fig. 6.2 Cumulative distribution of the detected security-sensitive data flows.

are used. The remaining data flows (130) are categorized as *No Vulnerability*, since we find evidence that they are not a security concern.

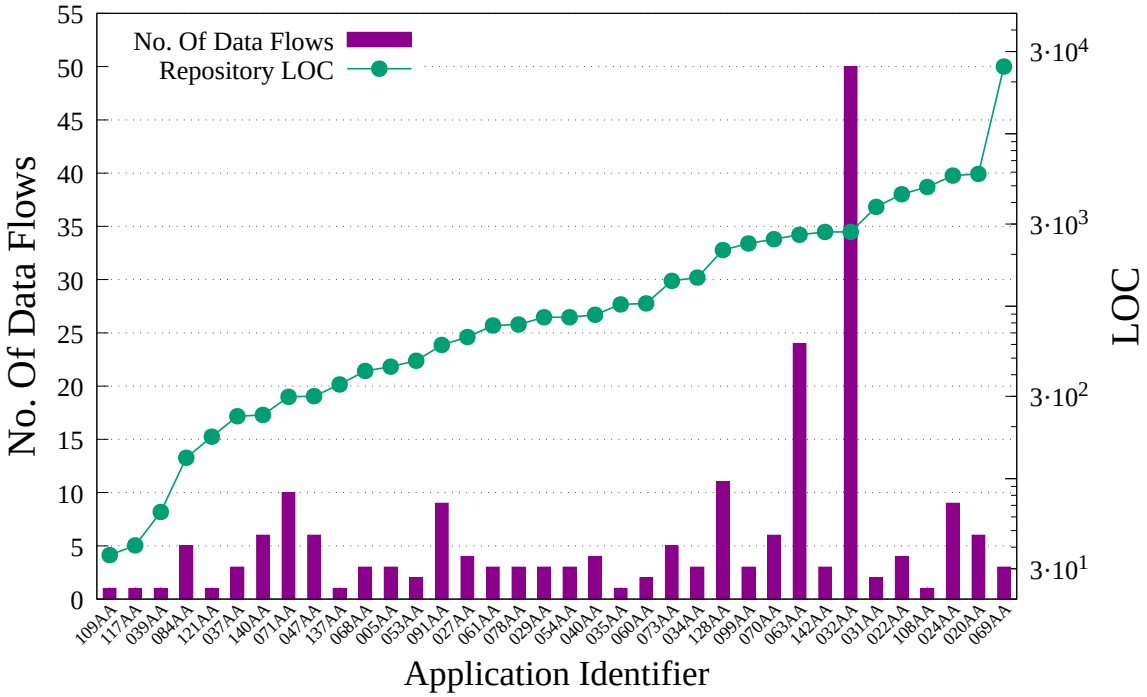

Fig. 6.3 Number of security-sensitive data flows (vertical axis on the left) and LOC (vertical axis on the right) vs analysed `AWSomePy` applications sorted by their LOC. Note the logarithmic scale on the vertical axis on the right.

To gain additional insight into the vulnerabilities, we further classify them into three types as illustrated in Table 6.3. It is noteworthy that three applications exhibit five code injection vulnerabilities caused by a HTTP request-triggered data flow. As shown in the code injection-focused motivating example in §4.2.1, these exist because user-controlled, unsanitized parts of the HTTP requests are used to initialize commands run via standard library functions, such as `subprocess.Popen` and `subprocess.check_output`. Interestingly, we also identify one application where code injection is possible because unchecked command-line (CLI) parameters are used to modify shell scripts executed via `os.system`.

The remaining four vulnerabilities, classified as *Exception Traceback Leakage*, are significantly different from those described above. They are detected by `CloudFlow` because traceback objects, which contain detailed information about raised exceptions, are included in HTTP requests, and hence visible to the user. While not directly exploitable, this pattern should be limited to the testing phase, as it can provide attackers with useful information.

Vulnerability Disclosure. As detailed in §3.2, the `AWSomePy` dataset comprises applications collected from the popular code-hosting platform GitHub. We therefore contacted the maintainers of two vulnerable applications in November 2024 to comply with the GitHub

Table 6.2 Summary of the security-sensitive data flows identified in the `AWSomePy` dataset. The table specifies the total number of data flows for each category along with the number of affected applications (App=Application, INTER=Inter-procedural, INTRA=Intra-procedural, IC=Injected Code).

Category	Total #	App #	INTER	INTRA	IC	CloudFlow Models
Vulnerability	11	5	6	5	0	4
Potential Vulnerability	64	15	56	8	1	30
No Vulnerability	130	31	105	25	1	101

Table 6.3 Summary of the vulnerabilities detected in the `AWSomePy` dataset. The table specifies the total number of vulnerabilities for each category along with the number of affected applications. The column Models reports the number of `CloudFlow`-generated custom Pysa models. (App=Application, INTER=Inter-procedural, INTRA=Intra-procedural, IC=Injected Code).

Vulnerability Type	Total #	App #	INTER	INTRA	IC	Models
Code Injection via CLI	2	1	2	0	0	0
Code Injection via HTTP	5	3	4	1	0	4
Exception Traceback Leakage	4	1	0	4	0	0

process for coordinated vulnerability disclosure [54], but we did not receive a reply prior to the submission of this thesis[5]. As regards the other three, the validation of our results shows that one of those affected by code injection via HTTP is intentionally vulnerable, and was open-sourced by its authors to illustrate common serverless security issues. In the remaining two cases, we were unable to notify the authors of our findings, as the repositories do not include a security policy and, as they are now archived, raising issues is no longer possible.

6.4 Vulnerability Case Studies

To illustrate the effectiveness of the proposed approach, this section describes three of the aforementioned vulnerabilities (§6.3). Since this required manually analysing the corresponding data flows, we also explain how `CloudFlow` facilitates this process.

Code Injection via CLI Case Study. One of the two code injection via CLI vulnerabilities is illustrated in Listing 6.1. The function `main` (line 9) starts by configuring the CLI

[5]For one of these two applications, a GitHub issue was raised to ask for a preferred security contact [57], as the repository did not include a security policy.

parser, and then instantiates the `args` object by calling the method `parse_args` (line 13). As a result, `args` stores the user-specified inputs as attributes, e.g., `args.tfvarsfile`, subsequently passed to the function `jenkins_sync_config` (line 14). The latter initializes the variable `jenkins_address` by calling `_get_tfvars_entry` (line 23), which, in turn, processes the unsanitized command-line input `tfvars_file`. Since the function `_get_tfvars_entry` only extracts information from a user-specified file using a regular expression (line 34), a user-controlled string can become part of `jenkins_address`. This variable is used to initialize `bash_jenkins_to_temp_cmd` (line 27) through a double string replacement within the Bash script stored in `BASH_SCRIPT_JENKINS_TO_TEMP` (line 1). Finally, `bash_jenkins_to_temp_cmd` is executed via `os.system` (line 28). We note that the script stored in `BASH_SCRIPT_JENKINS_TO_TEMP` includes the `sudo` command. This implies that it is supposed to be executed with a high level of privilege, thus making the injection of user-controlled code particularly dangerous.

```
1  BASH_SCRIPT_JENKINS_TO_TEMP = \
2      'ssh-keygen -R {}; ssh -C ubuntu@{} "bash -s" <<EOS \n' \
3      'sudo rm -rf /home/ubuntu/jenkins; mkdir -p /home/ubuntu/jenkins
     ; \n' \
4      'sudo cp -RP --verbose /var/lib/jenkins/* /home/ubuntu/jenkins \
     n' \
5      'sudo chown -R ubuntu.ubuntu /home/ubuntu/jenkins; \n' \
6      'find /home/ubuntu/jenkins/ -type l -delete; \n' \
7      'EOS'
8
9  def main() -> None:
10     parser = argparse.ArgumentParser()
11     # CLI parser configuration
12     ...
13     args = parser.parse_args()
14     jenkins_sync_config(args.mode, args.jenkinsdir, args.varfile,
15                         args.symlinkfile, args.secretsdir,
16                         args.tfvarsfile)
17
18 def jenkins_sync_config(mode, jenkins_dir: str, var_file,
19                         symlink_file, secrets_dir,
20                         tfvars_file) -> None:
21     # Initialization steps
22     ...
23     jenkins_address = 'jenkins.' + _get_tfvars_entry(tfvars_file,
24                                                      'domain')
25     with TemporaryDirectory() as temp_dir:
```

```
26         if mode == 'download':
27             bash_jenkins_to_temp_cmd = BASH_SCRIPT_JENKINS_TO_TEMP.
    format(jenkins_address, jenkins_address)
28             os.system(bash_jenkins_to_temp_cmd)
29
30 def _get_tfvars_entry(tfvars_file, key):
31     with open(tfvars_file, 'r') as fp:
32         for line in fp:
33             if line.startswith(key):
34                 result = re.search('"(.*)"', line).group(1)
35                 return result
36         raise ValueError('Could not find {} in {}'.format(key,
    tfvars_file))
```

Listing 6.1 Code injection via CLI case study.

Code Injection via HTTP Case Study. The code in Listing 6.2 illustrates a case of code injection via HTTP. The application handler defined at line 1 is triggered by a HTTP request. Following this, the event object is parsed to extract a S3 bucket key and a set of options (line 2). The key is used to download a file from the bucket (line 3), whereas the options are validated by calling `param_validation` (line 5). The name of the downloaded file, i.e., `source_filename`, is checked to ensure that it identifies a valid image, and then passed to `image_transform` along with the validated options (line 6). This function initializes the strings within the list `args`, which holds the command executed subsequently with `subprocess.check_output` (line 14).

Our analysis of the auxiliary functions `parse_event` and `is_valid_image` reveals that neither of them sanitizes the file name. Consequently, if the latter contains malicious commands, an attacker can trigger their execution with a HTTP request. `CloudFlow` detects this vulnerability because the handler `event` input is one of the processed sources. This shows the importance of automatically generating application-specific static analysis models (§5.2.5).

```
1 def handler(event, context):
2     s3_key, raw_ops = parse_event(event)
3     source_filename = download_s3_obj(BUCKET, s3_key)
4     if is_valid_image(source_filename):
5         ops = param_validation(raw_ops)
6         output_img = image_transform(source_filename, ops)
7
8 def image_transform(filename, ops):
9     args = ['convert', filename]
10    # Obtain filename extension
```

```
11     # Process ops and init output path
12     output = path + '.' + ext
13     args.append(output)
14     im_result = subprocess.check_output(args)
15     return output
```

Listing 6.2 Code injection via HTTP case study.

Exception Traceback Leakage Case Study. As shown in Table 6.3, one of the applications exhibits four intra-procedural vulnerabilities classified as exception traceback leakage. One of these is visible in Listing 6.3, which includes an event processing function of the application. The code shows that the function runs a set of processing and logging commands after the `try` statement (line 2). If an exception is raised during the execution of such commands, the traceback object `e` is returned unfiltered as part of the `Response` object (line 5). This will include detailed information on the exception, which can be leveraged by an attacker.

```
1 def event_process(missing_event) -> Response:
2     try:
3         # Processing and logging commands
4     except Exception as e:
5         return Response(str(e), status=500)
```

Listing 6.3 Exception traceback leakage case study.

Data Flow Manual Analysis. Our framework generates a summary report specifying the source and sink locations, along with the Pysa rule, for each of the identified data flows. The rules feature self-explanatory descriptions, e.g., *command-line arguments injection may result in RCE*, thus clarifying the reasons behind the detections. `CloudFlow` creates its report by processing the Pysa taint analysis results with the APIs provided by the SAPP tool [132]. To further inspect a given data flow, e.g., by tracking the taints of selected variables or generating a call graph, users can also leverage the Pysa debugging tools [116].

Although these features help identify vulnerabilities, they do not eliminate the need for manual analysis. In our case, it took on average approximately 20 minutes to inspect each data flow. However, we had no prior experience with the analysed applications. We expect the time required by a knowledgeable application's developer would be significantly lower.

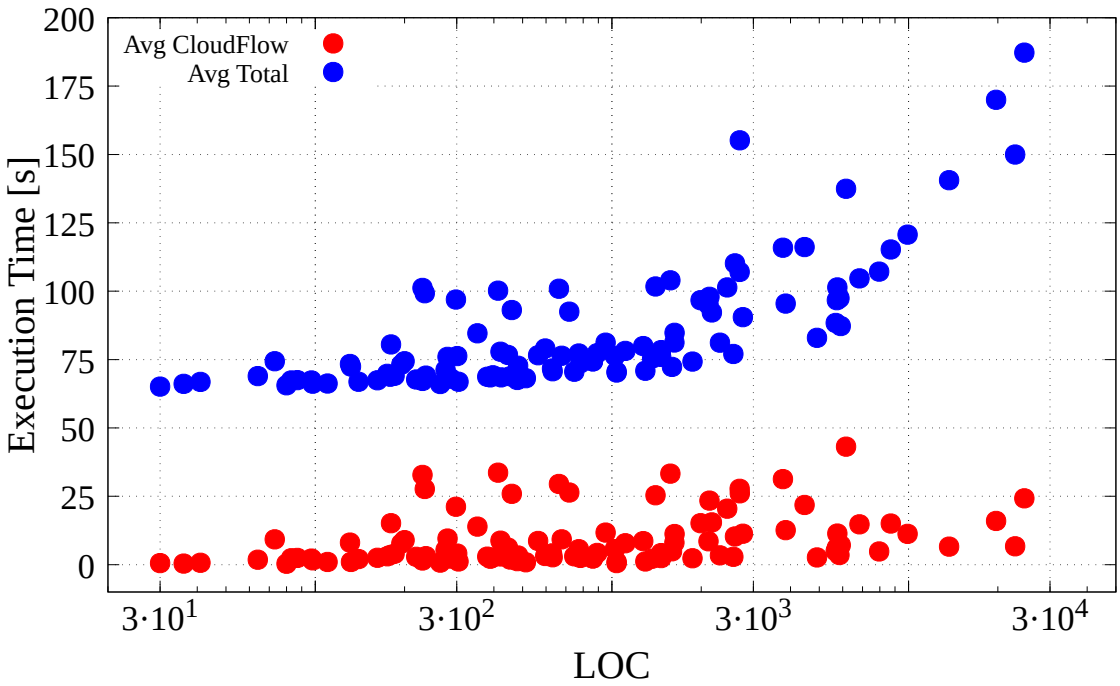

Fig. 6.4 Total and `CloudFlow`-specific execution times vs LOC of the analysed `AWSomePy` repositories. The displayed values are the average of five `CloudFlow` executions. Note the logarithmic scale on the horizontal axis.

6.5 Performance Evaluation

To evaluate the overhead caused by our framework, we report the execution times for the analysed `AWSomePy` applications. Our measurements are shown in Fig. 6.4, where both the total and the `CloudFlow`-specific execution times are displayed as a function of the number of lines of code (LOC) in the repository. Note that the presented values are obtained by averaging the results recorded during five `CloudFlow` executions. All our measurements were performed by running our framework in a Linux Ubuntu 20.04 virtual machine (VM) with 8GB of RAM and a dual-core processor.

We observe that the execution times start increasing when the LOC reaches the approximate value of 3,000. Since the recorded `CloudFlow`-specific values do not exhibit the same trend, the highlighted execution time increase is caused by Pysa. The latter, in fact, performs an automated type inference step[6] and identifies the data flows, two computationally-intensive tasks when carried out on large repositories. Albeit expected, this result confirms

[6]Technically, the automated type inference is performed by Pyre, the static type checker on which Pysa is built [112]. Since the same interface, i.e., the `pyre` command, is used to execute both tools, the presented analysis aggregates the type inference time with the data flow identification time for simplicity.

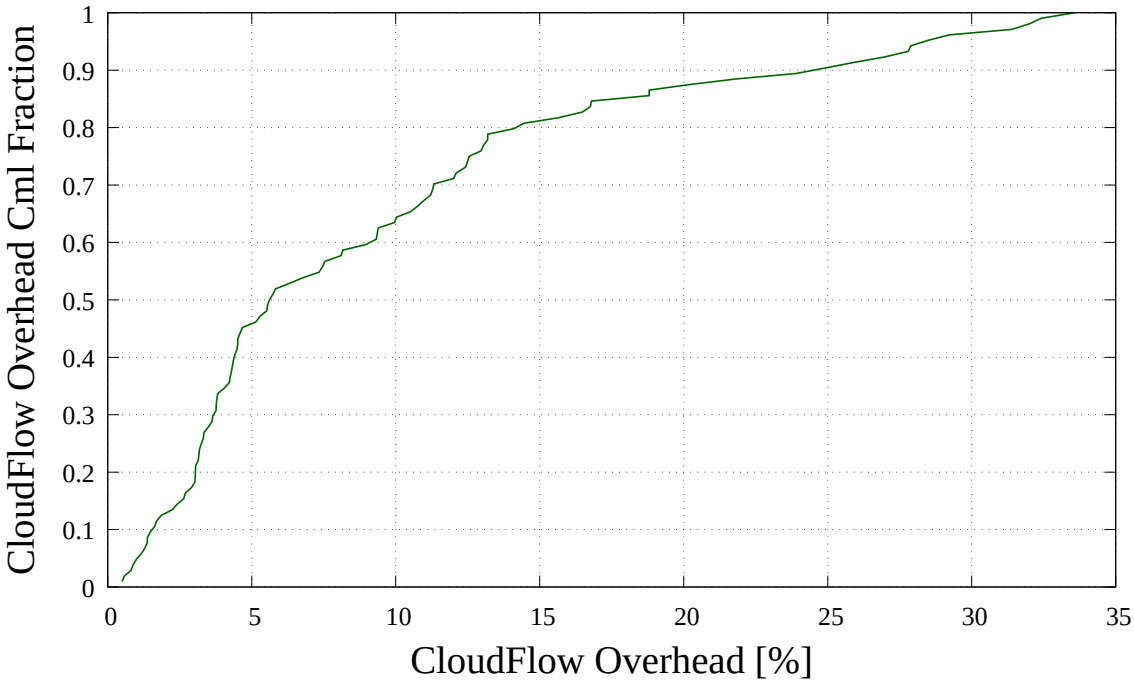

Fig. 6.5 Cumulative distribution of the `CloudFlow` overhead.

the scalability of the proposed approach within the LOC range of the analysed `AWSomePy` repositories.

To better quantify the framework overhead, we also analyse the percentages of the `CloudFlow`-specific execution times w.r.t. the total. Their cumulative distribution, visible in Fig. 6.5, shows that the `CloudFlow` overhead is generally acceptable, as it is 5% or less for 45% of the analysed applications, and it exceeds 25% only for 10% of them. This conclusion is further confirmed by the box plot diagram of Fig. 6.6, which shows the interquartile range, the minimum (0.5%), the maximum (33.6%), and the median (5.7%) of the recorded percentages.

6.6 Discussion

Infrastructure Code Analysis. There are two main limitations to the infrastructure code analysis currently performed by `CloudFlow`. First, our framework identifies and processes only one YAML file for each application under test. However, 21 of the 104 (20.2%) inspected repositories include multiple infrastructure code files. Our analysis indicates that some of these applications comprise several services, which are orchestrated to provide a complex piece of functionality. Other applications, instead, rely on separate `serverless.yml` files to

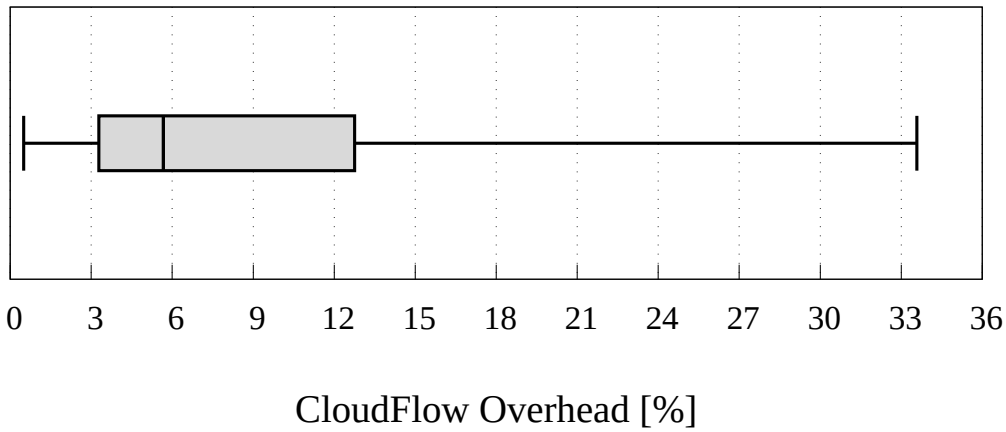

Fig. 6.6 Interquartile range, minimum, maximum, and median of the recorded `CloudFlow` overhead percentages.

specify additional infrastructure components. This implies that the analysis performed by `CloudFlow` in these cases is inevitably partial. To address this, we plan to provide the user with the option of choosing the YAML files to be analysed.

`CloudFlow` extracts application permissions either from specific sections of the infrastructure code file, for instance identified by the tag `iamRoleStatements` (Listing 4.1), or according to the logic of the IAM plugin [139] mentioned in §5.2.2. These are the most widely adopted mechanisms to define application-wide and handler-level permissions, but, as explained while introducing challenge C2, others exist. To evaluate the accuracy of our real-world applications analysis, we quantify the occurrences of such alternative mechanisms in our dataset. As shown in Table 6.4, resource policies, which allow defining permissions for cloud resources in terms of trusted principals, are present in eight applications, and are the most frequently used of these mechanisms. Although not currently supported, resource policy-related information can also be processed by extending our framework. Similarly, the analysis of YAML-specified customer managed policies and AWS managed policies, maintained by the provider to meet the requirements of common use cases, can be integrated as well.

In contrast, processing customer managed policies and roles defined as account resources would require access to the used infrastructure. Although not feasible for a research study, this can be achieved in enterprise settings by deploying a suitably extended version of `CloudFlow`.

Application Code Analysis. Our framework currently supports nine cloud APIs in total, which are shown in Table 6.5 along with the corresponding four AWS services. In four cases, i.e., `put_item`, `put_object`, `send_message`, and `upload_file`, this is achieved

Table 6.4 Alternative mechanisms to define permissions identified in our dataset (arranged alphabetically).

Mechanism	Specification	Apps
AWS Managed Policies	Provider	7
Customer Managed Policies	Account	4
Customer Managed Policies	YAML	7
Resource Policies	YAML	8
Roles	Account	4

Table 6.5 Cloud APIs supported by `CloudFlow` and corresponding AWS services.

API	S3	DynamoDB	SQS	SNS
`delete`	✔			
`list_objects`	✔			
`list_objects_v2`	✔			
`publish`				✔
`put_item`		✔		
`put_object`	✔			
`scan`		✔		
`send_message`			✔	
`upload_file`	✔			

with cloud API call models, whereas we implement custom Pysa models in the other cases. We focus our attention on this set of APIs because, as illustrated in Table 3.4 and Table 3.5, they are among the most frequently used in the `AWSomePy` dataset. Its characterization shows that 46 different services are used, and each of them is accessed via multiple API calls. This implies that the results obtained with our real-world applications leave out other less popular cloud services. As an example, the intentionally vulnerable application mentioned in §6.3 also contains an SQL injection vulnerability that goes undetected due to the absence of models for the AWS RDS service.

`CloudFlow` can be expanded to increase the number of supported services and APIs in a straightforward manner. However, some applications complicate this task by using configuration and management-oriented services to create cloud resources programatically (§3.3.2). For these applications, some important pieces of information become available only at runtime, which implies that the application needs to be analysed by combining dynamic and static techniques. Static analysis can still provide valuable information even when such services are used, but since they introduce data flows that are hard to detect statically we

Table 6.6 AWSomePy files with asynchronous constructs (ACO=Asynchronous Constructs Occurrences).

Application	File	ACO
ProjektTejas_tejas-service	classify.py	2
ProjektTejas_tejas-service	main.py	1
nicolaszein_auth	activation_created.py	1
nicolaszein_auth	reset_password_token_created.py	1
nicolaszein_auth	user_created.py	1

leave to future work the implementation of suitable hybrid strategies.

Asynchronous Constructs. Modern programming languages feature a range of asynchronous programming constructs to support non-blocking I/O operations (e.g., promise objects, asynchronous functions, and asynchronous iterators). These constructs are challenging to analyse statically, as they require specialized frameworks [149]. The objective of CloudFlow is not to support asynchronous constructs but to synthesize and inject infrastructure-dependent handler calls. We then use Pysa to inspect the instrumented application. As with any other general-purpose tool, Pysa relies on over-approximations, e.g., taint broadening [114], for the analysis of asynchronous constructs.

Nonetheless, the validity of the results presented in §6.3 is not significantly affected. As detailed in Table 6.6, our analysis shows that only two of the 104 analysed AWSomePy applications make use of asynchronous Python constructs, i.e., async/await keywords and the asyncio module. Specifically, we find six occurrences of such constructs spread across five application files, with the overall number of Python files being 3,612[7]. We believe that asynchronous constructs are rare in our dataset because their functionality is provided by the cloud services. We have therefore taken a pragmatic approach and consider only patterns more frequently adopted in serverless applications.

In our future work, we intend to improve the precision of our framework by integrating specialized static analysis schemes, such as the JavaScript-specific one proposed by Sotiropoulos *et al.* [149]. We are not aware, however, of alternative schemes for Python, as recent research focused on this language does not take into account asynchronous constructs [90, 130].

[7]The dataset contains four additional files with asynchronous constructs not listed in Table 6.6. Such files were not considered in our analysis because their names, e.g., test_true.py, suggest that they are test files. For completeness, one of these additional files is included in the application jesuejunior_vermilion, whereas the remaining three are part of the application nicolaszein_auth. The total number of asynchronous constructs detected in these files is seven.

6.7 Related Work

Large-scale Analyses of Cloud Resources. Continella *et al.* [35] analyse the security-related configuration of more than 240,000 AWS S3 buckets, one of the most widely adopted cloud storage solutions. The authors identify exposed buckets that can leak sensitive data, e.g., private cryptographic keys and credentials, because of mistakes in their access control policies. Furthermore, the study shows that there exist websites vulnerable to defacement and code injection because they upload static assets from publicly writable buckets. Two tools and a mitigation strategy to address these issues are proposed by the authors.

Cable *et al.* propose Stratosphere [31], which significantly improves the scanner by Continella *et al.* discussed above. Their system relies on passive DNS sources to obtain a large amount of buckets hosted on AWS and other two major providers. The bucket names are used as training data in a machine learning pipeline, which allows identifying more than two million buckets. In essence, while Continella *et al.* focus their attention on buckets with short and simple names, Cable *et al.* observe that the more complex names in their training data have a structure that frequently makes them predictable. Stratosphere finds 5.3 times more buckets with sensitive information compared to the work by Continella *et al.*, and a significant number of AWS buckets where permissions can be changed by an unauthenticated user. Interestingly, the analysis by Cable *et al.* reveals the worsening security problems of AWS S3, due to, in all probability, the complex permission model adopted by Amazon. This, as illustrated in §1.1, is also one of the main challenges (C2) of our research.

We underscore that the discussed works are exclusively focused on cloud storage services. While serverless applications rely on these very frequently (§3.3.2), the static analysis technique presented in this thesis has a broader scope, as it aims to identify security-sensitive data flows by taking into account multiple cloud-based services.

Testing of Serverless Applications. Wen *et al.* [166] highlight that the development of serverless-specific testing tools is an important avenue for future research, as the existing ones cannot be considered mature. To further explore the topic and understand, in particular, the challenges that developers face when testing serverless applications, Lenarduzzi *et al.* [80] interview three industry experts. Despite the limited number of interviewees, the work highlights two important issues relevant to our research. The first is the difference between the local and the deployment environment. While the former can be easily accessed, only the latter includes all the resources available at runtime. This represents a crucial limitation when testing a serverless application statically or with standard unit-tests. Indeed, the frequent use of configuration and management-oriented services, one of the key results of the characterization of `AWSomePy` (§3.3.2), further widens the gap between the two

environments. `CloudFlow` is inevitably affected by this problem, as it can only statically extract the information specified in the infrastructure code.

The complexity of the integration-level tests is the second crucial issue raised in the paper by Lenarduzzi *et al.*. The interviewed experts underscore that mocking cloud-based services effectively is critical yet challenging, a concept also recently reiterated by Rinta-Jaskari *et al.* [129]. To avoid pre-populated test events simulating the service functionality [15, 152], Manner *et al.* [88] propose a semi-automated troubleshooting technique that relies on logging inputs, outputs, and execution context of each serverless function. By contrast, `CloudFlow` makes use of models (§5.2.5) to synthesize an approximated version of the event object, which is then injected in the application code.

Model-based Testing Approaches. Winzinger *et al.* [168] illustrate how dependency graphs, which combine information extracted from application and infrastructure code, can be used to analyse serverless applications. In addition to identifying workflows, the authors show that this model-based approach enables the developer to evaluate the test coverage of an application [169]. Winzinger *et al.* also propose a serverless-specific testing framework focused on data flows [170]. First, these are modeled by categorizing the values returned by the functions in their dataset. Second, the authors rely on their model to devise a code instrumentation strategy aiming at logging data flow-related information. The work has recently been expanded with a methodology and a prototype tool to automatically generate test cases for serverless applications [171]. However, unlike the approach presented in this thesis, all these works rely on models that cannot be fully generated at coding time, as they require runtime information along with static attributes. As a result, data flows can only be analysed dynamically. We highlight that the techniques proposed by Winzinger *et al.* are used for coverage and integration testing, and are not designed to detect security-sensitive data flows. Furthermore, none of them is evaluated against a large dataset.

6.8 Chapter Summary

In this chapter, we present the results of the analysis of `AWSomePy` (Chapter 3) conducted with `CloudFlow` (Chapter 5). After triaging our dataset to identify templates, inconsistencies, and incompatibilities (§6.2), we focus our attention on 104 applications. In total, 205 security-sensitive data flows are detected. Their manual validation reveals that 11 of them are actual vulnerabilities. Specifically, seven allow code injection, whilst four manifest themselves as exception traceback leakage (§6.3). As regards the other security-sensitive data flows, 64

are categorized as potential vulnerabilities, as they cannot be classified with certainty. In contrast, we find evidence that the remaining 130 are not vulnerabilities.

To comprehensively evaluate `CloudFlow`, we also measure its execution times. As detailed in Fig. 6.4, these increase noticeably when the inspected applications contain, approximately, 3,000 LOC or more. A breakdown analysis of the recorded values, however, indicates that this upwards trend is determined by the computationally-intensive tasks performed by Pysa, the underlying static analysis tool. This result, therefore, confirms the scalability of our approach within the explored range of LOC. Finally, it is noteworthy that `CloudFlow` causes a generally acceptable overhead, which we quantify as 5% or less of the total execution time for 45% of the analysed applications (Fig. 6.5).

Chapter 7

Conclusion

In this chapter, we build on the key results of this thesis, summarized in §1.4, to provide directions for future work. We also include our final remarks, which will hopefully be a valuable reference for further research on static analysis of serverless applications.

7.1 Future Work

The results of the `AWSomePy` dataset analysis presented in Chapter 6 show that `CloudFlow` is capable of statically detecting vulnerabilities in real-world serverless applications. However, such applications rely on a high number of services and APIs, along with a wide range of complex mechanisms used to define their permissions. Unavoidably, this implies that the analysis framework proposed in this thesis is affected by some limitations, outlined in §5.4 and §6.6. These are revisited in this section to provide directions for future research.

Cloud Services and APIs. As shown in Table 6.5, `CloudFlow` supports only a limited set of cloud services and APIs. Although these were chosen according to the results of the `AWSomePy` dataset characterization, expanding this set should be considered a priority. Suitable strategies, in particular, will be required to address the challenges posed by configuration and management-oriented services. While providing greater flexibility, these add data flows that are difficult to identify statically. To facilitate this task, an API documentation analysis framework should be integrated into `CloudFlow`. As explained in §5.4, this would simplify the generation of the necessary API models. In addition, analysing the API documentation automatically would enable the extraction of default configuration values for programmatically-created cloud resources. Such values, in turn, would be a starting point for the detection of security-sensitive data flows. In order to increase the accuracy of the results,

CloudFlow should also be able to retrieve the exact version of the SDK by leveraging a software composition analysis tool [103, Ch. 3].

Importantly, extending the set of supported services and APIs should be accompanied by the development of new microbenchmarks. These should be added to CloudBench to facilitate their reuse and improve the coverage of the suite.

Infrastructure Code Analysis. The analysis of the infrastructure code is one of the key steps of the CloudFlow analysis pipeline (Fig. 5.1). This part of a serverless application, in fact, specifies essential parameters, such as permissions and event filters. While the proposed framework already features infrastructure code analysis capabilities, these should be further improved in the future.

Specifically, as detailed in §6.6, CloudFlow does not currently support all the permission mechanisms identified in AWSomePy (Table 6.4). Since resource policies are the most frequently occurring of such mechanisms, their automated analysis should be prioritized. This can be achieved by extending the framework module that extracts permission-related information. Furthermore, we note that CloudFlow is able to analyse only event filters (§2.1.1) for the AWS S3 service. Given that this can lead to inaccurate analyses, identifying approaches to supporting a wider set of such filters is undoubtedly a worthwhile avenue for future research.

Other Programming Languages and Cloud Platforms. The current implementation of CloudFlow is capable of analysing only Python applications. Its architecture, however, is generic and can be applied to other programming languages. The four-step strategy included in §5.4 details how this can be achieved. Thus, this should be a starting point for future extensions of CloudFlow. Supporting the analysis of applications implemented in JavaScript, in particular, is an objective worth pursuing, as this language is widely adopted in the serverless domain [45]. Brito *et al.*, however, highlight that current static analysis tools for JavaScript are affected by significant limitations [30]. Although advanced analysis schemes exist [149], this poses a challenge for the implementation of a JavaScript-focused extension of CloudFlow.

With regard to applying the proposed framework architecture to other cloud platforms, this is possible as well. Notably though, CloudFlow relies on the availability of an SDK, i.e., an interface that applications can leverage to access cloud-based services. While the experience gained during this research suggests that all major serverless environments include an SDK, future work in this area should begin by confirming this for the target platform. The second important step is the development of suitable event object models. As

explained in §4.2.1 and §5.2.4, in fact, these encapsulate complex data structures, which are also platform-specific. Since `CloudFlow` currently relies on event object models obtained manually, methodologies to generate them automatically would be highly beneficial. The identification of such methodologies is therefore an additional research direction that stems from this work.

7.2 Final Remarks

The aim of this section is to share some final remarks on the expected evolution of the serverless paradigm (§7.2.1) as well as our recommendations (§7.2.2). Both parts stem from our experience, and aim at contributing to the development of a serverless ecosystem where static analysis and, more generally, security play a key part.

7.2.1 Evolution of the Serverless Paradigm

Even though the most widely adopted FaaS platform, AWS Lambda, was launched more than a decade ago, it would be incorrect to conclude that serverless technology has now reached its maturity. In their comprehensive literature review, Wen *et at.* [166], in fact, identify 17 potential avenues for future research in this area, with testing tools and data security among the most prominent topics.

Furthermore, since the FaaS model is enterprise-oriented, it is important to consider the trends observed by industry experts. Ibryam mentions three key innovations that are reshaping the serverless paradigm [22]. The first (❶) is the introduction of programmable cloud constructs. These are, in essence, advanced configuration features that extend the functionality offered by existing cloud services. One of them is event filtering, which was considered in the development of `CloudFlow` (§5.2). Other similar features are emerging though, such as event batching and HTTP request routing, now part of the AWS platform. All these features blur the boundaries between application and infrastructure, and ultimately lead to serverless functions with less logic and less code. The second trend (❷) reported by Ibryam is the growing ecosystem of highly specialized multi-cloud services. These are typically developed by start-ups, for instance Fauna [47], focused on building functionality-reach services using those offered by major cloud providers as baselines. As a result, it is plausible that the IaC approach will be replaced by *Composition as Code*, where developers implement applications by combining complex pieces of functionality, i.e., not mere infrastructure components, such as databases and message brokers. This evolution is the third trend (❸), which will likely be accompanied by an increased usage of infrastructure tools featuring an

imperative approach, e.g., Pulumi [107] and the AWS CDK [8]. Declarative infrastructure tools, such as the Serverless Framework used in our work, might not be able, in fact, to support the growing number of required configuration options, due to their limited expressiveness.

Considering the scope of the presented research, it is crucial to understand what these industry trends mean from a static analysis point of view. Importantly, all of them suggest that future serverless applications will likely rely on more powerful abstraction layers and a higher number of configuration options. As underscored by previous academic research [166], these lead to *low-code development*, as the abstractions provided by highly configurable services require simpler serverless functions. While this scenario is undoubtedly beneficial in terms of testing, maintenance and cost, our work shows that this evolution of the serverless paradigm needs to be assessed more critically.

Powerful abstractions and numerous configuration options inevitably imply that models for static analysis, created either manually or automatically, will become more challenging and complex. Consequently, while developers relying on such cloud services might not need advanced coding skills, they will have to gain expertise in the configuration of the necessary infrastructure and, crucially, might have to deploy their applications before testing them comprehensively. Despite the unavoidable limitations of the proposed framework, our research shows that vulnerabilities currently exist in real-world serverless applications. The industry trends discussed above, however, indicate that ease of development and cost are being prioritized. To support the development of a more security-focused evolution of the serverless paradigm, we next provide our recommendations based on the experience gained during this research.

7.2.2 Recommendations

Standardization. The static analysis methodology presented in this thesis (§5.2.1) highlights the challenge of taking into account several cloud-based services and event-triggered code. Specifically, the implementation of `CloudFlow` relies on a number of event object models, despite targeting only one serverless environment. Having identical event objects for different services is, in fact, unattainable, as the specifics of such objects are a consequence of the service features and functionality. However, we would encourage platform developers to share the same basic structure among different event objects. Our research shows that this would have simplified the instantiation of our approach into `CloudFlow`. Wen *et al.* [166] have also called for standardization and interoperability between serverless providers, with cloud events requiring special attention. Considering the growing popularity of multi-cloud options, we can only echo their recommendation. We highlight that such standardization would be instrumental in facilitating the implementation of future applications and in analysing these

statically, thus improving their security posture.

Design Patterns. As illustrated in §5.5, researchers have recently analysed the topic of design patterns for serverless [66, 151]. Notably though, early work in this area is now mostly obsolete due to the evolution of the technology. Although design patterns will inevitably change over time, we recommend that platform providers prioritize their documentation to facilitate their adoption. As illustrated by existing examples [142], relying on design patterns implies using pre-defined structures, which greatly simplify the development of models for static analysis.

Research Datasets. It is important to reiterate that `AWSomePy`, presented in Chapter 3, and the other serverless datasets mentioned in this thesis (§3.5) contain exclusively open-source applications. Relying on datasets with this characteristic is a common practice in software engineering research, as such applications can be obtained from large and readily available collections, e.g., GitHub, used as a reference by numerous developers and communities. Eskandani *et al.*, however, in their analysis of the Wonderless dataset [46], observe that the generalization of their results to closed-source serverless applications would need to be further investigated.

Our work confirms that this aspect is currently neglected in serverless computing research. In particular, we note a stark contrast between the results of the `AWSomePy` and Wonderless characterizations and the data reported by industry experts. Specifically, while the applications included in such datasets feature a low number of handlers, PureSec [108] and the CSA [34] mention that the majority of enterprise-level applications contain at least dozens of functions, with thousands of them also possible in some cases. Such a difference would have a significant impact on the performance of any serverless security tool. The evaluation of `CloudFlow`, for instance, shows a noticeable increase of total execution time for repositories with LOC greater than 3,000 (§6.5). A synergy between industry practitioners and the research community is, therefore, necessary in order to create widely accessible datasets that reflect how closed-source applications are implemented.

References

[1] Abhay Bhargav (2018). Dynamodb injection. [online] https://medium.com/appsecengineer/dynamodb-injection-1db99c2454ac.

[2] Agache, A., Brooker, M., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., and Popa, D.-M. (2020). Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434, Santa Clara, CA. USENIX Association.

[3] Akimova, E. N., Bersenev, A. Y., Deikov, A. A., Kobylkin, K. S., Konygin, A. V., Mezentsev, I. P., and Misilov, V. E. (2021). Pytracebugs: A large python code dataset for supervised machine learning in software defect prediction. In *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*, pages 141–151.

[4] Alpernas, K., Flanagan, C., Fouladi, S., Ryzhyk, L., Sagiv, M., Schmitz, T., and Winstein, K. (2018). Secure serverless computing using dynamic information flow control. *Proc. ACM Program. Lang.*, 2(OOPSLA).

[5] Alzayat, M., Mace, J., Druschel, P., and Garg, D. (2023). Groundhog: Efficient request isolation in faas. EuroSys '23, page 398–415, New York, NY, USA. Association for Computing Machinery.

[6] Amazon Web Services (2021). Aws solutions. [online] https://aws.amazon.com/.

[7] Arzt, S., Rasthofer, S., Fritz, C., Bodden, E., Bartel, A., Klein, J., Le Traon, Y., Octeau, D., and McDaniel, P. (2014). Flowdroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '14, page 259–269, New York, NY, USA. Association for Computing Machinery.

[8] AWS Cloud Development Kit (2025). Define your cloud application resources using familiar programming languages. [online] https://aws.amazon.com/cdk/.

[9] AWS CloudWatch (2021). Amazon cloudwatch documentation. [online] https://docs.aws.amazon.com/cloudwatch/.

[10] AWS Cognito (2021). Amazon cognito documentation. [online] https://docs.aws.amazon.com/cognito/.

[11] AWS DynamoDB (2021). Amazon dynamodb documentation. [online] https://docs.aws.amazon.com/dynamodb/index.html.

[12] AWS IAM (2021). What is iam? [online] https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html.

[13] AWS Lambda (2021). Aws lambda documentation. [online] https://docs.aws.amazon.com/lambda/index.html.

[14] AWS Serverless Application Repository (2021). Aws serverless application repository. [online] https://serverlessrepo.aws.amazon.com/applications.

[15] AWS Tools (2025). Libraries, samples and tools to help go developers develop aws lambda functions. [online] https://github.com/aws/aws-lambda-go/tree/main/events/testdata.

[16] Back, T. and Andrikopoulos, V. (2018). Using a microbenchmark to compare function as a service solutions. In Kritikos, K., Plebani, P., and de Paoli, F., editors, *Service-Oriented and Cloud Computing*, pages 146–160, Cham. Springer International Publishing.

[17] Baldini, I., Castro, P., Chang, K., Cheng, P., Fink, S., Ishakian, V., Mitchell, N., Muthusamy, V., Rabbah, R., Slominski, A., and Suter, P. (2017). *Serverless Computing: Current Trends and Open Problems*, pages 1–20. Springer Singapore, Singapore.

[18] Bartosz Zaczyński (2024). Yaml: The missing battery in python. [online] https://realpython.com/python-yaml/.

[19] Baumann, A., Peinado, M., and Hunt, G. (2014). Shielding applications from an untrusted cloud with haven. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 267–283, Broomfield, CO. USENIX Association.

[20] Bernát Gábor (2018). The state of type hints in python. [online] https://bernat.tech/posts/the-state-of-type-hints-in-python/.

[21] Bhatnagar, S., Li, Z., Song, Z., and Tilevich, E. (2023). Os3: The art and the practice of searching for open-source serverless functions. In *2023 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)*, pages 219–224.

[22] Bilgin Ibryam (2024). Cloud-computing in the post-serverless era: Current trends and beyond. [online] https://www.infoq.com/articles/cloud-computing-post-serverless-trends/.

[23] Borges, H. and Tulio Valente, M. (2018). What's in a github star? understanding repository starring practices in a social coding platform. *Journal of Systems and Software*, 146:112–129.

[24] Boto3 Documentation (2022). Lambda client invoke api. [online] https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/lambda.html#Lambda.Client.invoke.

[25] Boto3 Documentation (2023). AWS SDK for Python (Boto3) to create, configure, and manage AWS services. [online] https://boto3.amazonaws.com/v1/documentation/api/latest/index.html.

[26] Boto3 DynamoDB Stubs Documentation (2025). Dynamodb module. [online] https://youtype.github.io/boto3_stubs_docs/mypy_boto3_dynamodb/.

[27] Boto3 DynamoDB Stubs Package (2025). Project description for package mypy-boto3-dynamodb. [online] https://pypi.org/project/mypy-boto3-dynamodb/.

[28] Bravenboer, M. and Smaragdakis, Y. (2009). Strictly declarative specification of sophisticated points-to analyses. In *Proceedings of the 24th ACM SIGPLAN Conference on Object Oriented Programming Systems Languages and Applications*, OOPSLA '09, page 243–262, New York, NY, USA. Association for Computing Machinery.

[29] Brikman, Y. (2019). *Terraform: Up & Running*. O'Reilly Media, Inc., Sebastopol, CA, USA, second edition.

[30] Brito, T., Ferreira, M., Monteiro, M., Lopes, P., Barros, M., Santos, J. F., and Santos, N. (2023). Study of javascript static analysis tools for vulnerability detection in node.js packages. *IEEE Transactions on Reliability*, 72(4):1324–1339.

[31] Cable, J., Gregory, D., Izhikevich, L., and Durumeric, Z. (2021). Stratosphere: Finding vulnerable cloud storage buckets. In *Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses*, RAID '21, page 399–411, New York, NY, USA. Association for Computing Machinery.

[32] Chester, J. (2021). *Knative in Action*. Manning Publications, Shelter Island, NY, USA, first edition.

[33] Cloud Native Computing Foundation (2021). Cncf wg-serverless whitepaper v1.0. [online] https://raw.githubusercontent.com/cncf/wg-serverless/master/whitepapers/serverless-overview/cncf_serverless_whitepaper_v1.0.pdf.

[34] Cloud Security Alliance (2019). The 12 most critical risks for serverless applications. [online] https://cloudsecurityalliance.org/blog/2019/02/11/critical-risks-serverless-applications/.

[35] Continella, A., Polino, M., Pogliani, M., and Zanero, S. (2018). There's a hole in that bucket! a large-scale analysis of misconfigured s3 buckets. In *Proceedings of the 34th Annual Computer Security Applications Conference*, ACSAC '18, page 702–711, New York, NY, USA. Association for Computing Machinery.

[36] Datadog Website (2021). The state of serverless. [online] https://www.datadoghq.com/state-of-serverless-2021/.

[37] Datadog Website (2023). The state of serverless. [online] https://www.datadoghq.com/state-of-serverless/.

[38] Datta, P., Kumar, P., Morris, T., Grace, M., Rahmati, A., and Bates, A. (2020). Valve: Securing function workflows on serverless computing platforms. In *Proceedings of The Web Conference 2020*, WWW '20, pages 939–950, New York, NY, USA. Association for Computing Machinery.

[39] Datta, P., Polinsky, I., Inam, M. A., Bates, A., and Enck, W. (2022). ALASTOR: Reconstructing the provenance of serverless intrusions. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 2443–2460, Boston, MA. USENIX Association.

[40] Davide Mastromatteo (2020). Security concerns with the python pickle module. [online] https://realpython.com/python-pickle-module/#security-concerns-with-the-python-pickle-module.

[41] Davis, J., Thekumparampil, A., and Lee, D. (2017). Node.fz: Fuzzing the server-side event-driven architecture. In *Proceedings of the Twelfth European Conference on Computer Systems*, EuroSys '17, page 145–160, New York, NY, USA. Association for Computing Machinery.

[42] Elshamy, A., Alquraan, A., and Al-Kiswany, S. (2023). A study of orchestration approaches for scientific workflows in serverless computing. In *Proceedings of the 1st Workshop on SErverless Systems, Applications and MEthodologies*, SESAME '23, page 34–40, New York, NY, USA. Association for Computing Machinery.

[43] Enck, W., Gilbert, P., Chun, B.-G., Cox, L. P., Jung, J., McDaniel, P., and Sheth, A. N. (2010). TaintDroid: An Information-Flow tracking system for realtime privacy monitoring on smartphones. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*, Vancouver, BC. USENIX Association.

[44] Endo, A. T. and Møller, A. (2020). Noderacer: Event race detection for node.js applications. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, pages 120–130.

[45] Eskandani, N. and Salvaneschi, G. (2021). The wonderless dataset for serverless computing. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, pages 565–569.

[46] Eskandani, N. and Salvaneschi, G. (2023). The uphill journey of faas in the open-source community. *Journal of Systems and Software*, 198:111589.

[47] Fauna Website (2025). The distributed serverless database. [online] https://fauna.com/.

[48] Frank, F., Alfke, M., Franceschi, A., Pastor, J. S., and Uphillis, T. (2017). *Puppet: Mastering Infrastructure Automation*. Packt Publishing, Birmingham, UK, first edition.

[49] Fu, X., Lee, D., and Jung, C. (2018). nadroid: statically detecting ordering violations in android applications. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization*, CGO '18, page 62–74, New York, NY, USA. Association for Computing Machinery.

[50] Gabbrielli, M., Giallorenzo, S., Lanese, I., Montesi, F., Peressotti, M., and Zingaro, S. P. (2019). No more, no less. In Riis Nielson, H. and Tuosto, E., editors, *Coordination Models and Languages*, pages 148–157, Cham. Springer International Publishing.

[51] Geir Arne Hjelle (2019). Python type checking (guide). [online] https://realpython.com/python-type-checking/.

[52] GitHub Repository (2021). Serverless aws pseudo parameters. [online] https://github.com/svdgraaf/serverless-pseudo-parameters.

[53] GitHub REST API Documentation (2022). Get a repository api. [online] https://docs.github.com/en/rest/repos/repos?apiVersion=2022-11-28#get-a-repository.

[54] GitHub Security Vulnerabilities Disclosure (2024). About coordinated disclosure of security vulnerabilities. [online] https://docs.github.com/en/code-security/security-advisories/guidance-on-reporting-and-writing-information-about-vulnerabilities/about-coordinated-disclosure-of-security-vulnerabilities.

[55] Giuseppe Raffa (2022). Module boto3 - invalid model issue. [online] https://github.com/facebook/pyre-check/issues/601.

[56] Giuseppe Raffa (2023). Module boto3 - taint propagation issue. [online] https://github.com/facebook/pyre-check/issues/795.

[57] Giuseppe Raffa (2024a). Coordinated security vulnerability disclosure - preferred contact. [online] https://github.com/caffeinetv/snappy/issues/20.

[58] Giuseppe Raffa (2024b). Pyre / pysa unable to analyse files with same name in different folders. [online] https://github.com/facebook/pyre-check/issues/823.

[59] Giuseppe Raffa (2024c). Serverless computing and python: Security challenges and ongoing research. [online] https://www.youtube.com/watch?v=KVbpvDdL_PQ.

[60] Gollmann, D. (2011). *Computer Security*. John Wiley & Sons, Chichester, UK, third edition.

[61] Google Cloud Platform (2021). Accelerate your transformation with google cloud. [online] https://cloud.google.com/.

[62] Guha, A., Saftoiu, C., and Krishnamurthi, S. (2010). The essence of javascript. In D'Hondt, T., editor, *ECOOP 2010 – Object-Oriented Programming*, pages 126–150, Berlin, Heidelberg. Springer Berlin Heidelberg.

[63] gVisor (2021). gvisor is an application kernel for containers that provides efficient defense-in-depth anywhere. [online] https://gvisor.dev/.

[64] Hauzar, D. and Kofron, J. (2015). Framework for Static Analysis of PHP Applications. In Boyland, J. T., editor, *29th European Conference on Object-Oriented Programming (ECOOP 2015)*, volume 37 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 689–711, Dagstuhl, Germany. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[65] Hillard, D. (2023). *Publishing Python Packages: Test, Share, and Automate Your Projects*. Manning Publications, Shelter Island, NY, USA, first edition.

[66] Hong, S., Srivastava, A., Shambrook, W., and Dumitras, T. (2018). Go serverless: Securing cloud via serverless design patterns. In *10th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 18)*, Boston, MA. USENIX Association.

[67] Intel SGX (2021). Intel software guard extensions. [online] https://software.intel.com/content/www/us/en/develop/topics/software-guard-extensions.html.

[68] Jangda, A., Pinckney, D., Brun, Y., and Guha, A. (2019). Formal foundations of serverless computing. *Proc. ACM Program. Lang.*, 3(OOPSLA).

[69] Jegan, D. S., Wang, L., Bhagat, S., and Swift, M. (2023). Guarding serverless applications with kalium. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4087–4104, Anaheim, CA. USENIX Association.

[70] Jeremy Daly (2018). Serverless microservice patterns for aws. [online] https://www.jeremydaly.com/serverless-microservice-patterns-for-aws.

[71] Jeremy Daly (2020). Event injection: Protecting your serverless applications. [online] https://www.jeremydaly.com/event-injection-protecting-your-serverless-applications/.

[72] JMeter (2025). Apache jmeter. [online] https://jmeter.apache.org/.

[73] Jovanovic, N., Kruegel, C., and Kirda, E. (2006). Pixy: a static analysis tool for detecting web application vulnerabilities. In *2006 IEEE Symposium on Security and Privacy (S&P'06)*, pages 6 pp.–263.

[74] Karakaya, K., Schott, S., Klauke, J., Bodden, E., Schmidt, M., Luo, L., and He, D. (2024). Sootup: A redesign of the soot static analysis framework. In Finkbeiner, B. and Kovács, L., editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 229–247, Cham. Springer Nature Switzerland.

[75] Katzer, J. (2020). *Learning Serverless*. O'Reilly Media, Inc., Sebastopol, CA, USA, first edition.

[76] Kim, J. and Lee, K. (2019a). Functionbench: A suite of workloads for serverless cloud function service. In *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*, pages 502–504.

[77] Kim, J. and Lee, K. (2019b). Practical cloud workloads for serverless faas. In *Proceedings of the ACM Symposium on Cloud Computing*, SoCC '19, page 477, New York, NY, USA. Association for Computing Machinery.

[78] Knative (2025). Open-source enterprise-level solution to build serverless and event driven applications. [online] https://knative.dev/docs/.

[79] Kummita, S., Piskachev, G., Späth, J., and Bodden, E. (2021). Qualitative and quantitative analysis of callgraph algorithms for python. In *2021 International Conference on Code Quality (ICCQ)*, pages 1–15.

[80] Lenarduzzi, V. and Panichella, A. (2021). Serverless testing: Tool vendors' and experts' points of view. *IEEE Software*, 38(1):54–60.

[81] Leonard Richardson (2023). Beautiful soup documentation. [online] https://www.crummy.com/software/BeautifulSoup/.

[82] Lepiller, J., Piskac, R., Schäf, M., and Santolucito, M. (2021). Analyzing infrastructure as code to prevent intra-update sniping vulnerabilities. In Groote, J. F. and Larsen, K. G., editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 105–123, Cham. Springer International Publishing.

[83] Li, W., Ming, J., Luo, X., and Cai, H. (2022). PolyCruise: A Cross-Language dynamic information flow analysis. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 2513–2530, Boston, MA. USENIX Association.

[84] Li, X., Leng, X., and Chen, Y. (2023). Securing serverless computing: Challenges, solutions, and opportunities. *IEEE Network*, 37(2):166–173.

[85] Madsen, M., Tip, F., and Lhoták, O. (2015). Static analysis of event-driven node.js javascript applications. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2015, page 505–519, New York, NY, USA. Association for Computing Machinery.

[86] Magusic, B. (2023). *Azure Security*. Manning Publications, Shelter Island, NY, USA, first edition.

[87] Maissen, P., Felber, P., Kropf, P., and Schiavoni, V. (2020). Faasdom: a benchmark suite for serverless computing. In *Proceedings of the 14th ACM International Conference on Distributed and Event-Based Systems*, DEBS '20, page 73–84, New York, NY, USA. Association for Computing Machinery.

[88] Manner, J., Kolb, S., and Wirtz, G. (2019). Troubleshooting serverless functions: a combined monitoring and debugging approach. *SICS Software-Intensive Cyber-Physical Systems*, 34:99–104.

[89] Microsoft Azure (2021). Power your vision on azure. [online] https://azure.microsoft.com/en-gb/.

[90] Monat, R., Ouadjaout, A., and Miné, A. (2020). Static Type Analysis by Abstract Interpretation of Python Programs. In Hirschfeld, R. and Pape, T., editors, *34th European Conference on Object-Oriented Programming (ECOOP 2020)*, volume 166 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 17:1–17:29, Dagstuhl, Germany. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[91] Morris, K. (2020). *Infrastructure as Code*. O'Reilly Media, Inc., Sebastopol, CA, USA, second edition.

[92] Mypy Website (2025). The mypy static type checker. [online] https://www.mypy-lang.org/.

[93] Nafise Eskandani (2021). Wonderless dataset github repository. [online] https://github.com/prg-grp/wonderless.

[94] Nupponen, J. and Taibi, D. (2020). Serverless: What it is, what to do and what not to do. In *2020 IEEE International Conference on Software Architecture Companion (ICSA-C)*, pages 49–50.

[95] Obetz, M., Das, A., Castiglia, T., Patterson, S., and Milanova, A. (2020). Formalizing event-driven behavior of serverless applications. In Brogi, A., Zimmermann, W., and Kritikos, K., editors, *Service-Oriented and Cloud Computing*, pages 19–29, Cham. Springer International Publishing.

[96] Obetz, M., Patterson, S., and Milanova, A. (2019). Static call graph construction in AWS lambda serverless applications. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*, Renton, WA. USENIX Association.

[97] OpenFaaS (2021). Serverless functions, made simple. [online] https://www.openfaas.com/.

[98] OpenLambda (2021). An open source serverless computing platform. [online] https://github.com/open-lambda/open-lambda.

[99] OpenWhisk (2021). Open source serverless cloud platform. [online] https://openwhisk.apache.org/.

[100] OWASP Serverless Top 10 Project (2021). Owasp serverless top 10. [online] https://owasp.org/www-project-serverless-top-10/.

[101] OWASP Serverless Top 10 Report (2018). Owasp top 10: Serverless interpretation. [online] https://raw.githubusercontent.com/OWASP/Serverless-Top-10-Project/master/OWASP-Top-10-Serverless-Interpretation-en.pdf.

[102] Palo Alto Networks (2019). Palo alto networks completes acquisition of puresec. [online] https://www.paloaltonetworks.com/company/press/2019/palo-alto-networks-completes-acquisition-of-puresec.

[103] Podjarny, G. and Tal, L. (2019). *Serverless Security*. O'Reilly Media, Inc., Sebastopol, CA, USA, first edition.

[104] PortSwigger Website (2021). Major jobs website left sensitive client data exposed for months. [online] https://portswigger.net/daily-swig/major-jobs-website-left-sensitive-client-data-exposed-for-months.

[105] PortSwigger Website (2022a). Insecure amazon s3 bucket exposed personal data on 500,000 ghanaian graduates. [online] https://portswigger.net/daily-swig/insecure-amazon-s3-bucket-exposed-personal-data-on-500-000-ghanaian-graduates.

[106] PortSwigger Website (2022b). Turkish flight operator pegasus airlines suffers data breach. [online] https://portswigger.net/daily-swig/turkish-flight-operator-pegasus-airlines-suffers-data-breach.

[107] Pulumi (2021). Cloud engineering for everyone. [online] https://www.pulumi.com/.

[108] PureSec (2019). The ten most critical risks for serverless applications v1.0. [online] https://github.com/puresec/sas-top-10.

[109] Pydantic Website (2025). Pydantic documentation. [online] https://docs.pydantic.dev/latest/.

[110] Pygount (2023). Pygount command line tool documentation. [online] https://pygount.readthedocs.io/en/latest/index.html.

[111] Pyre (2024). Getting started with pyre. [online] https://pyre-check.org/docs/getting-started/.

[112] Pyre Inference Feature (2025). Coverage increasing strategies (pyre infer). [online] https://pyre-check.org/docs/pysa-coverage/#pyre-infer.

[113] Pysa (2023). Pysa overview. [online] https://pyre-check.org/docs/pysa-basics/.

[114] Pysa Advanced Topics (2023). Taint broadening. [online] https://pyre-check.org/docs/pysa-advanced/#taint-broadening.

[115] Pysa Codebase (2023). Collection of default pysa rules. [online] https://github.com/facebook/pyre-check/blob/main/stubs/taint/core_privacy_security/taint.config.

[116] Pysa Debugging Tools (2023). Debugging tools. [online] https://pyre-check.org/docs/pysa-tips/#debugging-tools.

[117] Pysa Documentation (2023). Feature annotations. [online] https://pyre-check.org/docs/pysa-features/.

[118] Python Built-in Function eval (2025). eval documentation. [online] https://docs.python.org/3/library/functions.html#eval.

[119] Python Enhancement Proposal 3107 (2025). Pep 3107 – function annotations. [online] https://peps.python.org/pep-3107/.

[120] Python Enhancement Proposal 483 (2025). Pep 483 – the theory of type hints. [online] https://peps.python.org/pep-0483/.

[121] Python Enhancement Proposal 484 (2025). Pep 484 – type hints. [online] https://peps.python.org/pep-0484/.

[122] PyYAML (2025). Pyyaml library documentation. [online] https://pyyaml.org/wiki/PyYAMLDocumentation.

[123] Raffa, G., Alis, J. B., O'Keeffe, D., and Dash, S. K. (2023). Awsomepy: A dataset and characterization of serverless applications. In *Proceedings of the 1st Workshop on SErverless Systems, Applications and MEthodologies*, SESAME '23, page 50–56, New York, NY, USA. Association for Computing Machinery.

[124] Raffa, G., Blasco, J., O'Keeffe, D., and Dash, S. K. (2024). Towards inter-service data flow analysis of serverless applications. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 654–658.

[125] Raffa, G., Blasco, J., O'Keeffe, D., and Dash, S. K. (2025). CloudFlow: Identifying security-sensitive data flows in serverless applications. In *34th USENIX Security Symposium (USENIX Security 25)*, Seattle, WA. USENIX Association.

[126] Ralu Bolovan (2018). Python, boto3, and aws s3: Demystified. [online] https://realpython.com/python-boto3-aws-s3/.

[127] Rasthofer, S., Arzt, S., and Bodden, E. (2014). A machine-learning approach for classifying and categorizing android sources and sinks. In *NDSS 2014*, volume 14.

[128] Reps, T., Horwitz, S., and Sagiv, M. (1995). Precise interprocedural dataflow analysis via graph reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '95, page 49–61, New York, NY, USA. Association for Computing Machinery.

[129] Rinta-Jaskari, E., Allen, C., Meghla, T., and Taibi, D. (2022). Testing approaches and tools for aws lambda serverless-based applications. In *2022 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)*, pages 686–692.

[130] Salis, V., Sotiropoulos, T., Louridas, P., Spinellis, D., and Mitropoulos, D. (2021). Pycg: Practical call graph generation in python. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 1646–1657.

[131] Sankaran, A., Datta, P., and Bates, A. (2020). Workflow integration alleviates identity and access management in serverless computing. In *Annual Computer Security Applications Conference*, ACSAC '20, pages 496–509, New York, NY, USA. Association for Computing Machinery.

[132] SAPP Tool (2025). Post processor for facebook static analysis tools. [online] https://github.com/facebook/sapp.

[133] Sarkar, A. and Shah, A. (2018). *Learning AWS*. Packt Publishing, Birmingham, UK, second edition.

[134] Semgrep AppSec Platform (2025). Perform cross-file analysis. [online] https://semgrep.dev/docs/semgrep-code/semgrep-pro-engine-intro.

[135] Semgrep CE (2025). Semgrep community edition (ce) philosophy. [online] https://semgrep.dev/docs/contributing/semgrep-philosophy.

[136] Semgrep Documentation (2025). Data-flow analysis engine overview. [online] https://semgrep.dev/docs/writing-rules/data-flow/data-flow-overview.

[137] Semgrep Website (2021). Static analysis at ludicrous speed - find bugs and enforce code standards. [online] https://semgrep.dev/.

[138] Serverless Example With Step Function (2024). Community signup serverless service with lambda, python, step function and dynamodb. [online] https://www.serverless.com/examples/slack-signup-serverless.

[139] Serverless Framework IAM Plugin (2024). Serverless iam roles per function plugin. [online] https://www.serverless.com/plugins/serverless-iam-roles-per-function.

[140] Serverless Framework Step Functions Plugin (2024). Serverless step functions plugin. [online] https://www.serverless.com/plugins/serverless-step-functions.

[141] Serverless Framework Website (2021). Zero-friction serverless development. [online] https://www.serverless.com/.

[142] Serverless Land Website (2025). Serverless patterns collection. [online] https://serverlessland.com/patterns.

[143] Shevrin, I. and Margalit, O. (2023). Detecting Multi-Step IAM attacks in AWS environments via model checking. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6025–6042, Anaheim, CA. USENIX Association.

[144] Shields, D. (2022). *AWS Security*. Manning Publications, Shelter Island, NY, USA, first edition.

[145] Sirone Jegan, D., Wang, L., Bhagat, S., Ristenpart, T., and Swift, M. (2020). Guarding Serverless Applications with SecLambda. *arXiv e-prints*, page arXiv:2011.05322.

[146] Sivakumaran, P. and Blasco, J. (2019). A study of the feasibility of co-located app attacks against BLE and a Large-Scale analysis of the current Application-Layer security landscape. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1–18, Santa Clara, CA. USENIX Association.

[147] Somu, N., Daw, N., Bellur, U., and Kulkarni, P. (2020). Panopticon: A comprehensive benchmarking tool for serverless applications. In *2020 International Conference on COMmunication Systems & NETworkS (COMSNETS)*, pages 144–151.

[148] Son, S. and Shmatikov, V. (2011). Saferphp: finding semantic vulnerabilities in php applications. In *Proceedings of the ACM SIGPLAN 6th Workshop on Programming Languages and Analysis for Security*, PLAS '11, New York, NY, USA. Association for Computing Machinery.

[149] Sotiropoulos, T. and Livshits, B. (2019). Static Analysis for Asynchronous JavaScript Programs. In Donaldson, A. F., editor, *33rd European Conference on Object-Oriented Programming (ECOOP 2019)*, volume 134 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 8:1–8:29, Dagstuhl, Germany. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[150] Steven Arzt (2017). Droidbench - a micro-benchmark suite to assess the stability of taint-analysis tools for android. [online] https://github.com/secure-software-engineering/DroidBench.

[151] Taibi, D., El Ioini, N., Pahl, C., and Niederkofler, J. R. S. (2020). Patterns for serverless functions (function-as-a-service): A multivocal literature review. In *Proceedings of the 10th International Conference on Cloud Computing and Services Science - CLOSER*, pages 181–192. INSTICC, SciTePress.

[152] Thomas Schoffelen (2024). A library of example event payloads for aws lambda. [online] https://github.com/tschoffelen/lambda-sample-events/tree/master/events/aws.

[153] Tileria, M., Blasco, J., and Dash, S. K. (2024). Docflow: Extracting taint specifications from software documentation. ICSE '24, New York, NY, USA. Association for Computing Machinery.

[154] Tileria, M., Blasco, J., and Suarez-Tangil, G. (2020). WearFlow: Expanding information flow analysis to companion apps in wear OS. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, pages 63–75, San Sebastian. USENIX Association.

[155] Tiobe Website (2024). Tiobe index. [online] https://www.tiobe.com/tiobe-index/.

[156] Trach, B., Oleksenko, O., Gregor, F., Bhatotia, P., and Fetzer, C. (2019). Clemmys: Towards secure remote execution in faas. In *Proceedings of the 12th ACM International Conference on Systems and Storage*, SYSTOR '19, pages 44–54, New York, NY, USA. Association for Computing Machinery.

[157] Ustiugov, D., Petrov, P., Kogias, M., Bugnion, E., and Grot, B. (2021). Benchmarking, analysis, and optimization of serverless function snapshots. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'21)*. ACM.

[158] Vallée-Rai, R., Gagnon, E., Hendren, L., Lam, P., Pominville, P., and Sundaresan, V. (2000). Optimizing java bytecode using the soot framework: Is it feasible? In Watt, D. A., editor, *Compiler Construction*, pages 18–34, Berlin, Heidelberg. Springer Berlin Heidelberg.

[159] van Eyk, E., Scheuner, J., Eismann, S., Abad, C. L., and Iosup, A. (2020). Beyond microbenchmarks: The spec-rg vision for a comprehensive serverless benchmark. In *Companion of the ACM/SPEC International Conference on Performance Engineering*, ICPE '20, page 26–31, New York, NY, USA. Association for Computing Machinery.

[160] van Eyk, E., Toader, L., Talluri, S., Versluis, L., Uță, A., and Iosup, A. (2018). Serverless is more: From paas to present cloud computing. *IEEE Internet Computing*, 22(5):8–17.

[161] vHive (2021). Open-source framework for serverless experimentation. [online] https://github.com/ease-lab/vhive.

[162] Viafore, P. (2021). *Robust Python: Write Clean and Maintainable Code*. O'Reilly Media, Inc., Sebastopol, CA, USA, first edition.

[163] Volos, S., Fournet, C., Hofmann, J., Köpf, B., and Oleksenko, O. (2024). Principled microarchitectural isolation on cloud cpus. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 183–197, New York, NY, USA. Association for Computing Machinery.

[164] Wang, L., Li, M., Zhang, Y., Ristenpart, T., and Swift, M. (2018). Peeking behind the curtains of serverless platforms. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 133–146, Boston, MA. USENIX Association.

[165] Wang, R., Casale, G., and Filieri, A. (2022). Enhancing performance modeling of serverless functions via static analysis. In Troya, J., Medjahed, B., Piattini, M., Yao, L., Fernández, P., and Ruiz-Cortés, A., editors, *Service-Oriented Computing*, pages 71–88, Cham. Springer Nature Switzerland.

[166] Wen, J., Chen, Z., Jin, X., and Liu, X. (2023). Rise of the planet of serverless computing: A systematic review. *ACM Trans. Softw. Eng. Methodol.*, 32(5).

[167] Widyasari, R., Sim, S. Q., Lok, C., Qi, H., Phan, J., Tay, Q., Tan, C., Wee, F., Tan, J. E., Yieh, Y., Goh, B., Thung, F., Kang, H. J., Hoang, T., Lo, D., and Ouh, E. L. (2020). Bugsinpy: a database of existing bugs in python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations o Software Engineering*, ESEC/FSE 2020, page 1556–1560, New York, NY, USA. Association for Computing Machinery.

[168] Winzinger, S. and Wirtz, G. (2019). Model-based analysis of serverless applications. In *2019 IEEE/ACM 11th International Workshop on Modelling in Software Engineering (MiSE)*, pages 82–88.

[169] Winzinger, S. and Wirtz, G. (2020). Applicability of coverage criteria for serverless applications. In *2020 IEEE International Conference on Service Oriented Systems Engineering (SOSE)*, pages 49–56.

[170] Winzinger., S. and Wirtz., G. (2021). Data flow testing of serverless functions. In *Proceedings of the 11th International Conference on Cloud Computing and Services Science - CLOSER*, pages 56–64. INSTICC, SciTePress.

[171] Winzinger, S. and Wirtz, G. (2022). Automatic test case generation for serverless applications. In *2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, pages 77–84.

[172] Xie, Y. and Aiken, A. (2006). Static detection of security vulnerabilities in scripting languages. In *Proceedings of the 15th Conference on USENIX Security Symposium - Volume 15*, USENIX-SS'06, USA. USENIX Association.

[173] YAML (2021). Yaml ain't markup language. [online] https://yaml.org/.

Appendix A

CloudBench Characterization

In this appendix, we include additional information on the developed microbenchmarks. In particular, individual URLs are provided for those already publicly available to support future research work.

A.1 Preliminary Microbenchmark Collection

As explained in §4.3, the `CloudBench` suite was developed by expanding the preliminary collection of microbenchmarks included in the paper P2. These are publicly available, and Table A.1 provides their names, categorization (i.e., inter-procedural or intra-procedural), and individual URLs.

Table A.1 Preliminary collection of microbenchmarks used to develop the `CloudBench` suite (INTER=Inter-procedural, INTRA=Intra-procedural).

Microbenchmark	INTER	INTRA	URL
`api-publish-wrong-bucket-key`	✔	✗	GitHub Link
`api-put-item-boto3-client`	✔	✗	GitHub Link
`api-put-item-via-file`	✔	✗	GitHub Link
`api-put-item-wrong-table`	✔	✗	GitHub Link
`api-put-object-boto3-client`	✔	✗	GitHub Link
`api-put-object-bucket-assign`	✔	✗	GitHub Link
`api-scan-boto3-client`	✗	✔	GitHub Link
`api-scan-table-assign`	✗	✔	GitHub Link
`api-send-message-boto3-client`	✔	✗	GitHub Link
`owasp-serverless-injection`	✗	✔	GitHub Link

A.2 CloudBench Inter-procedural Microbenchmarks

The purpose of this section is to provide additional information on the inter-procedural microbenchmarks included in `CloudBench`. Specifically, to further characterize each of them, we detail the target cloud service, i.e., the service that provides the targeted API, and the other services used (Tables A.2, A.3, A.4, and A.5).

Table A.2 Cloud service characterization. `CloudBench` inter-procedural microbenchmarks with S3 as target service (DDB=DynamoDB, TS=Target Service, ✔=Yes, ✗=No).

Microbenchmark	TS	S3	DDB	SQS	SNS
`api-put-object-boto3-client`	S3	✔	✗	✗	✗
`api-put-object-bucket-assign`	S3	✔	✗	✗	✗
`api-put-object-iam-plugin`	S3	✔	✗	✗	✗
`api-put-object-partial-import`	S3	✔	✗	✗	✗
`api-put-object-two-handlers`	S3	✔	✗	✗	✗
`api-upload-file-boto3-client`	S3	✔	✗	✗	✗
`api-upload-file-bucket-assign`	S3	✔	✗	✗	✗
`api-upload-file-iam-plugin-custom`	S3	✔	✗	✗	✗
`api-upload-file-iam-plugin-inherit`	S3	✔	✗	✗	✗
`api-upload-file-iam-plugin`	S3	✔	✗	✗	✗
`api-upload-file-kw-args`	S3	✔	✗	✗	✗
`api-upload-file-object-assign`	S3	✔	✗	✗	✗
`api-upload-file-partial-import`	S3	✔	✗	✗	✗
`api-upload-file-two-handlers`	S3	✔	✗	✗	✗

Table A.3 Cloud service characterization. `CloudBench` inter-procedural microbenchmarks with DynamoDB as target service (DDB=DynamoDB, TS=Target Service, ✔=Yes, ✗=No).

Microbenchmark	TS	S3	DDB	SQS	SNS
`api-put-item-boto3-client`	DDB	✔	✔	✗	✗
`api-put-item-dict-var`	DDB	✔	✔	✗	✗
`api-put-item-kw-args`	DDB	✔	✔	✗	✗
`api-put-item-misspelt-service`	DDB	✔	✔	✗	✗
`api-put-item-table-assign`	DDB	✔	✔	✗	✗
`api-put-item-via-file`	DDB	✔	✔	✗	✗
`api-put-item-wrong-table-os-environ`	DDB	✔	✔	✗	✗
`api-put-item-wrong-table-os-getenv`	DDB	✔	✔	✗	✗
`api-put-item-wrong-table-str`	DDB	✔	✔	✗	✗
`api-put-item-wrong-table`	DDB	✔	✔	✗	✗

Table A.4 Cloud service characterization. `CloudBench` inter-procedural microbenchmarks with SQS as target service (DDB=DynamoDB, TS=Target Service, ✔=Yes, ✗=No).

Microbenchmark	TS	S3	DDB	SQS	SNS
`api-send-message-boto3-client`	SQS	✔	✔	✔	✗
`api-send-message-kw-arg`	SQS	✔	✔	✔	✗
`api-send-message-queue-assign`	SQS	✔	✔	✔	✗

Table A.5 Cloud service characterization. `CloudBench` inter-procedural microbenchmarks with SNS as target service (DDB=DynamoDB, TS=Target Service, ✔=Yes, ✗=No).

Microbenchmark	TS	S3	DDB	SQS	SNS
`api-publish-boto3-client`	SNS	✔	✔	✗	✔
`api-publish-no-file-system`	SNS	✔	✗	✗	✔
`api-publish-wrong-bucket-key`	SNS	✔	✗	✗	✔

Appendix B

CloudFlow Architecture and Evaluation

In this appendix, we include additional details on the architecture and the evaluation of `CloudFlow`. Specifically, the appendix supplements the description of the framework (§5.2) and the results of its evaluation (§5.3.1).

B.1 CloudFlow Architecture

`CloudFlow` comprises a collection of task-focused modules. As illustrated in Fig. B.1, these are coordinated by an *Analysis Manager*, a high-level component that relies on the functionality offered by the modules to perform the required processing steps. After completing these, the Analysis Manager launches Pysa [113], the general-purpose static analysis tool used to inspect the instrumented application. All the modules of our framework are integrated into a command-line utility, which we intend to open-source to support future research work.

To further document the proposed analysis framework, we also include its workflow in Fig. B.2. This details how the application under test is instrumented before performing the data flow analysis. The workflow includes the processing steps described in §5.2, and supplements the high-level diagram of Fig. 5.1.

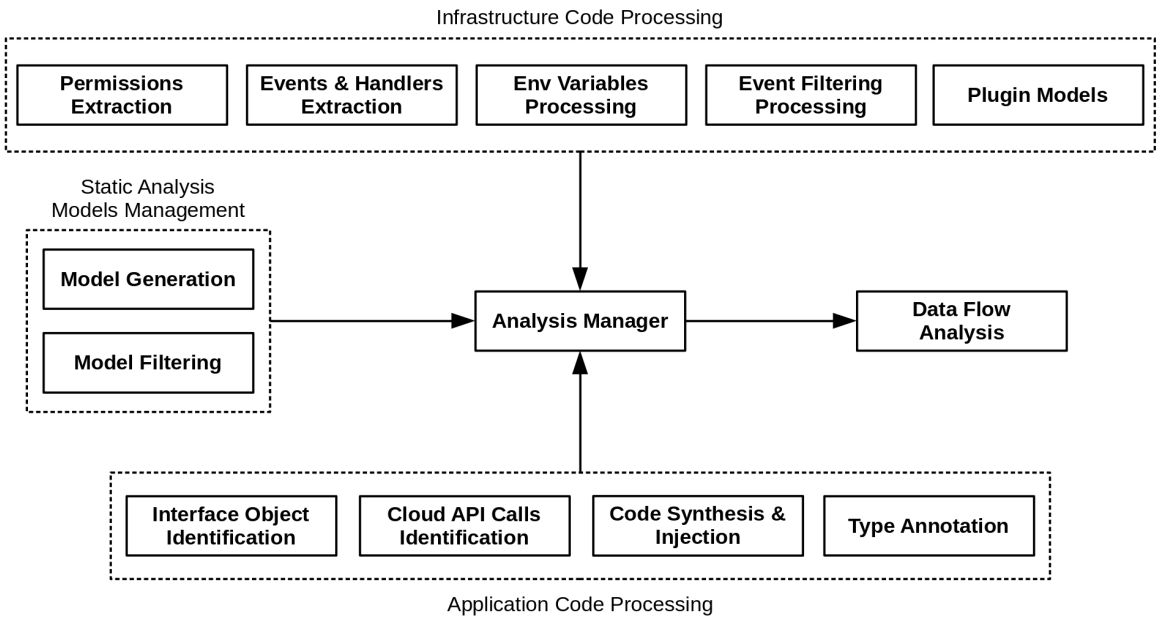

Fig. B.1 `CloudFlow` modular architecture.

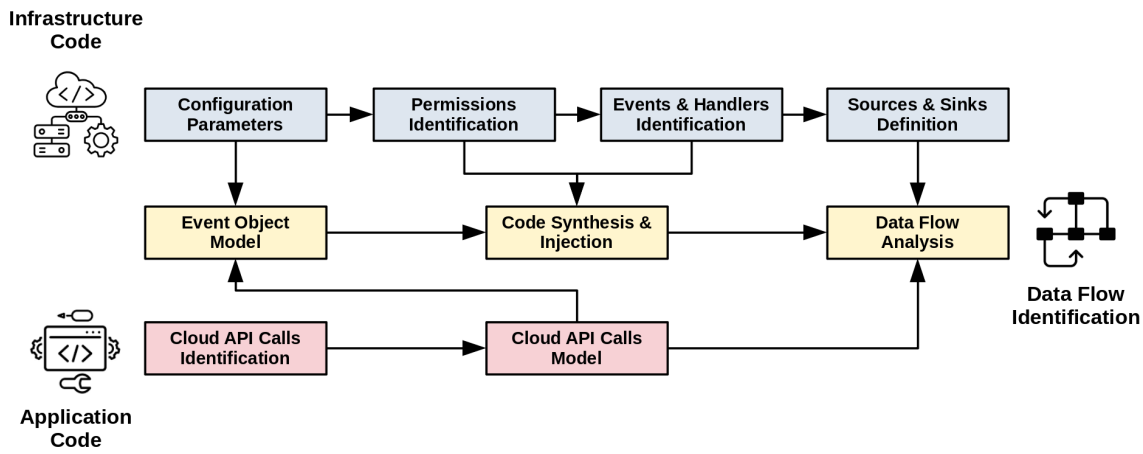

Fig. B.2 `CloudFlow` detailed workflow.

B.2 Ablation Study Results

The detailed results of the ablation study presented in §5.3.1 are reported in Table B.1. The table shows all the `CloudBench` microbenchmarks that failed in the baseline evaluation. The features that enabled their successful execution with `CloudFlow` are specified as well. We note that Table B.1 includes the ablation study results at microbenchmark level, thus supplementing the aggregate figures reported in Table 5.1.

Table B.1 Microbenchmark-level results of `CloudFlow` ablation study. The column CloudFlow specifies the CloudFlow evaluation results (EVI=Environment Variable Inspection, PLA=Plugin Analysis, REA=Resource Analysis, ✔=Decisive feature, ✗=Non decisive feature).

Microbenchmark	EVI	PLA	REA	CloudFlow
`api-put-object-iam-plugin`	✗	✔	✗	PASS
`api-upload-file-iam-plugin-custom`	✗	✔	✗	PASS
`api-upload-file-iam-plugin-inherit`	✗	✔	✗	PASS
`api-upload-file-iam-plugin`	✗	✔	✗	PASS
`api-put-item-via-file`	✗	✗	✗	FAIL
`api-put-item-wrong-table-os-environ`	✔	✗	✔	PASS
`api-put-item-wrong-table-os-getenv`	✔	✗	✔	PASS
`api-put-item-wrong-table-str`	✗	✗	✔	PASS
`api-put-item-wrong-table`	✔	✗	✔	PASS
`api-publish-boto3-client`	✗	✗	✗	FAIL
`api-publish-wrong-bucket-key`	✗	✗	✗	FAIL
`owasp-serverless-injection-iam-plugin`	✗	✔	✗	PASS

Appendix C

AWSomePy Dataset Analysis

In this appendix, we report the detailed results of the dataset triage presented in §6.2. We also include further information on the security-sensitive data flows discussed in §6.3.

C.1 AWSomePy Dataset Triage

This section includes a set of tables, i.e., Table C.1, C.2, C.3, C.4, and C.5, that list the `AWSomePy` repositories filtered out. Each table is dedicated to one of the identified categories.

Table C.1 `AWSomePy` repositories categorized as Template (TEM).

Repository	Cat.
aws-samples_serverless-patterns	TEM
ryfeus_stepfunctions2processing	TEM
brianz_serverless-design-patterns	TEM
PacktPublishing_Serverless-Design-Patterns-and-Best-Practices	TEM
stevehouel_serverless-todo-api	TEM
AdventielFr_terraform-aws-lets-encrypt-renew-certificates	TEM
labsyspharm_minerva-cloud	TEM
yuki-mt_scripts	TEM
teremterem_swipe-thought-bot	TEM
bikranz4u_aws-lambda	TEM
dota247_AWS	TEM
mduesterhoeft_websocket-step-function	TEM
Kiran01bm_aws-related	TEM
farrukh90_lambda	TEM
vortexkd_serverless_experiments	TEM
wyutong1997_SmartMedicineDispenser	TEM
reklawth_aws-python	TEM
dmillikan_aws_automation	TEM
rspishock_automating-aws-with-python	TEM
tanalam2411_aws	TEM
c5apple_slstest	TEM
komushi_youtube-dl	TEM
ryfeus_test-jupyter-binder	TEM

Table C.2 `AWSomePy` repositories categorized as `CloudFlow`-incompatible (CFI).

Repository	Cat.
awslabs_amazon-pollexy	CFI
brianz_dilbert-slack-bot	CFI
peakshift_fovea	CFI
aaronshim_alexa-skills	CFI
gplassard_aws-cloudformation-custom-ressource-github-webhook	CFI
SmartPracticeschool_SPS-2126-Virtual-Office-Assistant-Powered-by-AWS-Lex	CFI

Table C.3 `AWSomePy` repositories categorized as Pysa-incompatible (PSI).

Repository	Cat.
epsagon_epsagon-python	PSI
keithrozario_Gov-TLS-Audit	PSI
hprutvisagar_aws-pricing-tools	PSI
mruston0_albumshuffler	PSI
dthompson989_rancor-py-beta	PSI
phouse512_piper	PSI

Table C.4 `AWSomePy` repositories categorized as infrastructure code inconsistent (ICI).

Repository	Cat.
FBosler_fb-twitter-bot	ICI
nbabra_subStop-c1-hackathon	ICI
yohanmishkin_reminder-back	ICI
BloomTech-Labs_labby-functions	ICI

Table C.5 `AWSomePy` repositories categorized as application code inconsistent (ACI).

Repository	Cat.
servian_aws-auto-remediate	ACI
evvwevv_ImageUploader	ACI

C.2 Identified Security-sensitive Data Flows

This section includes Table C.6, which specifies the mapping between the `AWSomePy` repositories with security-sensitive data flows and the identifiers visible in Fig. 6.1.

Table C.6 `AWSomePy` repositories with security-sensitive data flows.

App Id	Repository	No. Of Data Flows
032AA	paulharte_autobrew	50
063AA	halprin_retrospective	24
128AA	carlosonunez_tripit-apis	11
071AA	zacharygraziano_bigtext	10
091AA	tlskins_tennis-swing-cutter	9
024AA	o-o-overflow_dc2020q-scoreboard	9
140AA	mawinkler_c1-app-sec-insekurestore	6
070AA	Gilbertly_kickoffai-sls-dataml	6
047AA	harveyrendell_strava-reporter	6
020AA	apache_incubator-mxnet-ci	6
084AA	EOS-Phos_processor	5
073AA	the-deep_serverless	5
040AA	caffeinetv_snappy	4
027AA	snuffkin_qctokyo	4
022AA	mozilla_subhub	4
142AA	ONSdigital_es-imputation-sg	3
099AA	zhammer_adventure-anywhere	3
078AA	DemocracyClub_wdiv-s3-trigger	3
069AA	malariagen_sims-backbone	3
068AA	pearcem0_whos-out	3
061AA	vincentclaes_serverless_crypto_analysis	3
054AA	ohyeahjoe_apcompsciproject	3
037AA	hobuinc_codm	3
034AA	MXNetEdge_mxnet-infrastructure	3
029AA	zhammer_playlist-souffle	3
005AA	bahrmichael_aws-scheduler	3
060AA	warpcomdev_sentiment	2
053AA	demidov91_belarusbank	2
031AA	brazil-data-cube_cube-builder-aws	2
137AA	LCOGT_photonranch-status	1
121AA	Global-Witness_eu-gas-detector	1
117AA	KevinJMcGarry_cloudwatch_notifier	1
109AA	seo-3_yakisaba	1
108AA	hecforga_cocum	1
039AA	zamirajaupaj_serverless-rekognition-polly	1
035AA	GeoscienceAustralia_dea-orchestration	1